

研 究 生 毕 业 论 文

(申请工程硕士学位)

论 文 题 目 基于模型驱动的实时与离线混合检索系统
的设计与实现

作 者 姓 名 _____

学科、专业名称 _____

研 究 方 向 _____

指 导 教 师 _____

2026 年 4 月 23 日

学 号：

论文答辩日期：XXXX 年 XX 月 XX 日

指 导 教 师： (签字)

Design and Implementation of a Model-Driven Hybrid Real-Time and Offline Retrieval System

by

Supervised by

A dissertation submitted to
the graduate school of
in partial fulfilment of the requirements for the degree of
MASTER OF ENGINEERING
in

April 23, 2026

研究生毕业论文中文摘要首页用纸

毕业论文题目： 基于模型驱动的实时与离线混合检索系统

专业 级硕士生姓名：

指导教师（姓名、职称）：

摘 要

在信息爆炸和数据多样化的时代，如何高效地从海量信息中快速获取精准的答案，已经成为各类检索系统面临的关键挑战。传统的检索系统往往依赖于基于关键词的匹配，虽然能提供一定的搜索结果，但常常忽略了用户意图的深层理解，导致检索效果不理想。近年来，随着自然语言处理和深度学习技术的快速发展，基于模型驱动的检索系统逐渐成为研究热点，但在工程化应用中，如何将复杂的技术和模型高效地集成，并支持高效、低延迟的检索与数据处理，依然面临诸多挑战。

本论文设计并实现了一套以 Search Agent 为核心的端到端检索流水线系统，旨在解决上述挑战。系统依托 MCP Server 和搜索引擎后端技术架构构建，整体架构划分为前端交互层、搜索引擎处理层和数据处理层三个层次，通过 MCP Server 进行中间层的通信和数据传输，确保了系统的高效与稳定。在线搜索模块基于 Bin Search 引擎，结合 Python 爬虫实现内容的实时爬取和解析，提升了搜索精度与时效性；离线搜索模块则利用 Lucene 和 Faiss 实现文本和向量检索，确保垂直领域数据的高效处理。系统部署于企业私有云环境，并通过微服务架构提供高可用性和灵活的扩展能力。

系统包含以下三项核心功能模块：

（1）Prompt 解析模块：系统首先利用预训练模型对用户 Prompt 进行语义分词与关键词提炼，将自然语言意图转化为可执行的检索请求和任务指令。通过这一模块，用户的自然语言输入得以精准转化为机器可理解的查询请求。

（2）实时/离线检索模块：该模块整合了实时在线检索和离线垂域检索两个子系统，支持高效的检索与数据获取。首先，针对实时检索，系统以 bin search 为代表，能够处理热点定时检索、超时与并发控制等任务。当检索结果满足展示条件时，触发基于 Python 的实时爬取。实时爬取模块专注于抓取策

略、请求限速、动态超时、实时解析与断点容错，确保能够高效获取新鲜数据，并以低延迟实时下发至数据处理模块。其次，针对离线检索，该模块采用关键词驱动的文本倒排检索与语义向量检索，面向小红书、知乎等垂域数据源，获取候选内容。在离线检索的实现中，优化了索引质量、批量导入与向量化策略，确保了高效的离线检索性能。

(3) 召回融合重排模块：该模块结合了跨通道召回融合、重排与数据处理的核心功能，旨在提升检索结果的相关性与响应时延。通过 TopK 合并策略和跨通道召回融合，系统在保证召回覆盖的同时优化了排序结果的准确性。数据处理方面，构建了包括实时解析、清洗、质量评估的分层流水线，结合规则与模型的抽取与去噪，确保高效地将爬取内容落库到倒排索引与向量库。索引层实现了倒排索引的增量构建与合并，同时优化了向量化模型和近似搜索参数，从而平衡了召回覆盖与响应时延。

系统测试结果表明，实时爬取与实时解析在受控的超时与并发策略下能够在可接受的延迟范围内提供新鲜内容，爬取成功率达到了 95%，且抓取到可检索内容的延迟控制在 1 秒内。跨通道召回融合与分层重排在延迟约束下有效提升了整体排序效果，Top1 排名的准确率提高了 15%，同时响应延迟保持在 100ms 以内。

关键词：模型驱动提示解析；Search Agent；实时检索；离线检索；实时爬取；混合召回融合；倒排索引；向量索引

研究生毕业论文英文摘要首页用纸

THESIS: Design and Implementation of a Model-Driven Hybrid
Real-Time and Offline Retrieval System

SPECIALIZATION: _____

POSTGRADUATE: _____

MENTOR: _____

Abstract

In the era of information explosion and data diversification, the ability to efficiently extract accurate answers from massive information has become a key challenge for various retrieval systems. Traditional retrieval systems often rely on keyword-based matching, which, while providing some search results, frequently overlook a deeper understanding of user intent, leading to unsatisfactory retrieval outcomes. In recent years, with the rapid development of natural language processing and deep learning technologies, model-driven retrieval systems have gradually become a research hotspot. However, in engineering applications, how to efficiently integrate complex technologies and models to support high-performance, low-latency retrieval and data processing still presents numerous challenges.

The thesis designs and implements an end-to-end retrieval pipeline system centered around a Search Agent, aimed at addressing the aforementioned challenges. The system is built on the technical architecture of the MCP Server and search engine backend, with the overall structure divided into three layers: the frontend interaction layer, search engine processing layer, and data processing layer. The communication and data transmission between layers are managed through the MCP Server, ensuring system efficiency and stability. The real-time search module is based on the Bin Search engine, integrating Python web crawlers for real-time content fetching and parsing, thereby improving search accuracy and timeliness. The offline search module utilizes Lucene and Faiss for text and vector-based search, ensuring efficient processing of domain-specific data. The system is deployed in an enterprise private cloud environment, and its high availability and scalability are supported by a microservices architecture.

The system includes the following three core functional modules:

(1) Prompt Parsing Module: The system first utilizes pre-trained models to perform semantic tokenization and keyword extraction on the user's prompt, transforming the natural language intent into executable retrieval requests and task instructions. This module allows the user's natural language input to be accurately converted into machine-understandable queries.

(2) Real-time/Offline Retrieval Module: This module integrates real-time online search and offline domain-specific search subsystems to support efficient retrieval and data acquisition. For real-time retrieval, the system, represented by Bin Search, handles tasks such as scheduled hot-topic retrieval, timeout and concurrency control. When the retrieval results meet the display conditions, it triggers real-time content scraping based on Python. The real-time scraping module focuses on fetching strategies, request throttling, dynamic timeouts, real-time parsing, and fault tolerance, ensuring the efficient acquisition of fresh data and its low-latency delivery to the data processing module. For offline retrieval, the module employs keyword-driven inverted text search and semantic vector search to target data sources like XHS and Zhihu, obtaining candidate content. In implementing offline retrieval, optimizations are made to indexing quality, batch imports, and vectorization strategies, ensuring high-efficiency performance in offline search.

(3) Recall Fusion and Re-ranking Module: This module integrates cross-channel recall fusion, re-ranking, and data processing to enhance the relevance of retrieval results and response latency. Through the TopK merging strategy and cross-channel recall fusion, the system optimizes the accuracy of ranking results while maintaining recall coverage. In terms of data processing, a multi-layered pipeline is constructed, including real-time parsing, cleaning, and quality evaluation. By combining rule-based and model-based extraction and denoising techniques, the system ensures efficient storage of crawled content into both inverted indexes and vector databases. The index layer implements incremental construction and merging of inverted indexes, while also optimizing the vectorization model and approximate search parameters to balance recall coverage and response latency.

The system test results indicate that Real-time scraping and parsing, under controlled timeouts and concurrency strategies, can deliver fresh content within an accept-

able latency range, with a scrape success rate of 95%, and the delay for fetching searchable content is controlled within 1 second. Cross-channel recall fusion and layered re-ranking effectively improve the overall sorting performance within latency constraints, increasing the accuracy of the Top-1 ranking by 15%, while keeping the response delay under 100ms.

keywords: Model-driven prompt parsing, Search Agent, Real-time retrieval, Offline retrieval, Real-time crawling, Hybrid recall fusion, Inverted index, Vector index

目 录

目 录	vii
插图清单	xi
附表清单	xiii
第一章 绪论	1
1.1 研究背景与意义	1
1.2 研究发展现状	2
1.2.1 信息检索与混合检索的发展	2
1.2.2 大模型在提示解析与检索驱动中的作用	3
1.2.3 实时检索与离线垂域检索的工程需求与应用场景	4
1.2.4 查询算法的分类和发展现状	4
1.3 研究目标与任务	6
1.4 论文结构安排	7
第二章 技术综述	9
2.1 开发与性能工具	9
2.1.1 版本控制与代码托管: Git	9
2.1.2 线上变更管理平台: MCM	10
2.1.3 日志与调试工具: Log4j	10
2.1.4 配置与密钥管理: Lion	11
2.1.5 后端开发框架: MDP 美团开发平台	12
2.1.6 缓存与加速: Redis	12
2.1.7 消息队列服务: Mafka	13
2.1.8 远程过程调用: RPC	13
2.2 数据与服务器监控平台	14
2.2.1 指标监控与观测性能: Prometheus	14
2.2.2 告警与 SLO/SLI 管理: Alertmanager	15
2.2.3 安全监控与审计: Auditd	15
2.3 混合检索中使用到的算法	16

2.3.1 倒排索引与文本检索算法: Lucene	16
2.3.2 向量检索与近似最近邻算法: Faiss	17
2.3.3 重排与排序模型: feature-based LTR	18
2.3.4 实时爬取与解析相关算法	19
2.3.5 索引更新与增量建库策略: bulk	20
2.4 本章小结	20
第三章 实时与离线混合检索系统的需求分析与设计	21
3.1 系统总体概述	21
3.2 系统需求分析	23
3.2.1 系统用例分析	23
3.2.2 Prompt 解析与 Query 生成	24
3.2.3 MCP 协调与参数传递	24
3.2.4 实时在线检索 (bin search) 与爬取触发	26
3.2.5 离线垂域检索 (文本检索/向量检索)	28
3.2.6 索引入库与数据处理模块	29
3.2.7 非功能性需求分析	31
3.3 系统总体设计	32
3.3.1 系统逻辑视图	32
3.3.2 系统过程视图	34
3.3.3 系统开发视图	35
3.3.4 系统物理视图	37
3.4 系统模块设计	39
3.4.1 系统层次与模块划分	39
3.4.2 Prompt 解析模块设计	40
3.4.3 实时检索与爬取模块设计	44
3.4.4 离线检索与索引模块设计	48
3.4.5 召回融合与重排模块设计	51
3.5 数据库与索引存储设计	54
3.6 本章小结	61
第四章 实时与离线混合检索系统的设计与实现	63
4.1 Prompt 解析模块的实现	63
4.2 实时检索与爬取模块的实现	65

目 录	ix
4.3 离线检索与索引模块的实现.....	67
4.4 召回融合与重排模块的实现.....	69
4.5 本章小结.....	71
第五章 系统测试	73
5.1 系统测试概述	73
5.2 单元测试	73
5.3 功能测试	75
5.3.1 Prompt 解析模块功能测试	75
5.3.2 实时检索与爬取模块功能测试	76
5.3.3 离线检索与索引模块功能测试	77
5.3.4 召回融合与重排模块功能测试	78
5.4 非功能测试	79
5.5 本章小结	80
第六章 总结与展望.....	81
6.1 论文工作总结	81
6.2 后续工作展望	82
致 谢	83
参考文献	85

插图清单

3-1	系统总体概述图	21
3-2	系统详细分解图	22
3-3	系统整体用例图	23
3-4	系统逻辑视图	33
3-5	数据爬取清洗解析入库时序图	35
3-6	系统开发视图	36
3-7	系统物理视图	38
3-8	Prompt 解析模块设计流程图	41
3-9	Prompt 解析模块设计设计类图	42
3-10	Prompt 解析模块设计时序图	43
3-11	实时检索与爬取模块流程图	44
3-12	实时检索与爬取模块设计类图	45
3-13	实时检索与爬取模块时序图	47
3-14	离线检索与索引模块流程图	48
3-15	离线检索与索引模块设计类图	49
3-16	离线检索与索引模块时序图	50
3-17	召回融合与重排模块设计流程图	51
3-18	召回融合与重排模块设计设计类图	52
3-19	召回融合与重排模块设计时序图	54
3-20	关系型数据数据库设计 er 图	59
4-1	Prompt 解析模块的实现	64
4-2	实时检索与爬取模块的实现	66
4-3	离线检索与索引模块的实现	68
4-4	召回融合与重排模块的实现	70
5-1	单元测试代码	74

附表清单

- 3-1 Prompt 解析与 Query 生成用例描述 25
- 3-2 MCP 协调与参数传递用例描述 26
- 3-3 实时在线检索（bin search）与爬取触发模块用例描述 27
- 3-4 离线垂域检索（文本检索/向量检索）模块用例描述 29
- 3-5 索引入库与数据处理模块用例描述 30
- 3-6 用户查询表：存储用户输入的查询信息..... 55
- 3-7 搜索结果表：存储搜索引擎返回的结果..... 56
- 3-8 爬取的数据表：存储从爬虫抓取的数据..... 56
- 3-9 内容库表：存储所有的内容，包括文本、图像等 57
- 3-10 处理后的数据表：存储经过清洗和质量评估后的数据 58

- 4-1 Prompt 解析模块接口描述（Search Agent） 63
- 4-2 实时检索与爬取模块接口描述（Search Agent 实时模块） 65
- 4-3 离线检索与索引模块接口描述（Search Agent 离线模块） 67
- 4-4 召回融合与重排模块接口描述（Search Agent 召回融合与重排） ... 69

- 5-1 系统单元测试结果表 75
- 5-2 Prompt 解析模块功能测试报告 76
- 5-3 实时检索与爬取模块功能测试报告 77
- 5-4 离线检索与索引模块功能测试报告 78
- 5-5 召回融合与重排模块功能测试报告 78

第一章 绪论

本章简要介绍本项目的研究背景、目标以及主要内容。项目旨在开发一个高效的 Search Agent 系统，该系统通过连接 MCP server 和后端搜索引擎，实现对用户查询的智能处理与精准搜索。系统包括两大核心功能模块：实时在线搜索和离线搜索。在线搜索模块支持基于实时数据的动态爬取与内容展示，离线搜索则专注于垂直领域的数据处理与检索。通过关键词提炼、文本检索与向量检索相结合，系统能够有效提升搜索精度与响应速度。

1.1 研究背景与意义

随着互联网内容规模和多样性的快速扩展，用户对信息获取的即时性、准确性和场景相关性的要求不断提高。在这个大数据时代，信息的生成速度与更新频率已经远超传统检索系统的处理能力。特别是用户从搜索引擎、社交媒体、新闻网站等平台获取信息的需求日益增长，传统基于关键字的检索方法在面对高维语义查询、非结构化内容和跨媒体信息时逐渐显露出局限性 [1]。这些方法主要依赖词汇匹配或字符串匹配技术，尽管在某些场景下能有效检索到符合条件的结果，但它们对语义理解的能力较弱。尤其在需要捕捉用户深层意图和跨领域知识的情境中，基于关键字的检索方法往往无法提供满意的检索效果。此外，这些方法对于处理复杂查询、长尾问题以及多模态数据时显得力不从心，导致信息覆盖面不足和用户体验不佳。

在此背景下，语义向量检索、预训练大模型以及混合检索策略应运而生，并为提升检索覆盖与相关性提供了新的可能。语义向量检索通过将文本、图像等数据转化为向量空间中的点，能够更好地捕捉到信息之间的语义关联，突破了传统基于关键字的匹配限制。通过使用深度学习技术训练的大型预训练模型，系统可以实现更为精确的语义理解，从而大幅提高信息检索的质量与效率 [2]。混合检索策略则结合了传统检索与基于语义向量的检索方法，利用两者的优势，进一步增强了检索结果的精准度与覆盖面，尤其在大规模数据环境中，能够显著优化用户的检索体验 [3]。

与此同时，商业与工程实践中存在两类互补的数据来源，这对现代检索系统的架构设计提出了更高的要求。第一类是需要保证时效性和新鲜度的实时在线内容，这类内容的更新频繁且与时事新闻、社交动态等紧密相关，因此其检索要求极高的响应速度与及时更新能力 [4]。第二类是具有专业垂域价值的离线数据集合，这类数据一般来源于学术论文、行业报告、专业文献等，包含了大量的结构化或半结构化的信息。虽然这些数据的时效性较差，但其内容的深度和专业性使得它们在某些特定场景下具有不可替代的价值。

然而，实时在线数据和离线垂域数据在检索需求、索引策略与召回机制上存在显著差异。实时数据要求系统具有高效的抓取与更新能力，以应对内容的快速变动，并且在时间限制下提供准确的结果；而离线数据则更侧重于高质量的索引与深度检索，系统需要在保证召回覆盖的同时，尽可能提高检索效率。这就要求检索系统能够同时应对两类数据的特点，综合考虑时效性与专业性，提出合适的检索策略。

本研究提出并实现以 Search Agent 为核心的工程化方案，旨在通过模型驱动的 prompt 解析将自然语言提示稳定、准确地转化为检索指令 [5]，进而在 MCP 协调下高效触发实时在线检索或离线检索通道 [6]，并在必要时发起实时爬取以增强结果的新鲜度与信息深度。该工作具有重要工程与学术意义：一方面，从工程视角，它覆盖了从提示理解、检索调度、实时爬取、实时解析、到数据清洗与入库的端到端流程，解决了多通道协同、时延控制与容错调度等实用问题，为生产环境下的检索平台提供可落地的实现路径；另一方面，从方法论角度，结合模型驱动的语义解析与混合检索策略 [7]，可以显著提升召回覆盖与排序质量，探索在低延迟约束下的召回融合与分层重排策略，对后续检索系统的研究与优化具有借鉴意义。因此，构建一个工程化且可部署的模型驱动实时—离线混合检索 Search Agent，不仅能满足实际业务需求，也具有推动检索系统向更语义化、更实时化方向发展的学术价值。

1.2 研究发展现状

1.2.1 信息检索与混合检索的发展

信息检索领域自经典倒排索引与布尔检索起步，经历了概率检索模型、基于语言模型的检索方法，到近年来的基于深度学习与表示学习的语义检索体系演进 [8]。倒排索引在处理大规模文本检索时以其高效的召回与成熟的工程实现

成为主力，而向量检索（尤其是基于深度语义表示的近似最近邻搜索）在语义匹配与模糊查询场景下展现出显著优势 [9]。由此催生的混合检索架构，通常通过并行或分层方式同时利用倒排索引与向量索引来获得既有精确匹配又有语义覆盖的候选集。近年的研究与工程实践还重点关注召回融合策略，如分数级融合、RRF 等算法已被广泛应用，而基于原始分数的动态加权融合策略可进一步提升；索引更新与增量构建策略以降低索引维护成本并保证数据新鲜度，其中流式向量量化与双索引架构通过分离操作更新与检索查询来实现近实时的索引变更可见性 [10]；以及在有限资源下的检索效率优化，例如基于乘积量化的近似最近邻搜索压缩技术，通过聚类划分空间与量化大幅降低存储开销，以及通过即时重算嵌入而非存储的存储高效型索引方案 [11]。

在工程化方向，检索系统趋向模块化与服务化，出现了分离的索引服务、检索召回服务、重排服务与上下游数据流水线 [12]。实时检索场景推动了流式索引、近实时更新与低延迟 API 的发展，而离线垂域数据的采用则强调批处理编目、质量筛选与语义向量化的一致性。总体来看，学术界在召回模型、向量表示与重排模型上持续推进，工业界则在系统可用性、可扩展性与运维方面积累了大量经验，混合检索正逐步由实验室成果向生产系统演进。

1.2.2 大模型在提示解析与检索驱动中的作用

预训练大模型在自然语言理解、生成及抽取任务上展示出强大的通用能力，使其在将用户自然语言提示转化为检索动作中具备天然优势。通过对 prompt 进行语义分词、意图识别、实体抽取与关键词提炼，模型能够为检索器提供更符合用户语义需求的检索词或扩展查询，从而改进召回的相关性。在工程实践中，模型驱动的 prompt-to-query 模块 [13] 通常需要具备稳定性、可解释性与可控性，这意味着在使用大模型时应结合规则化后处理、置信度评估与模板约束以降低噪声引入风险 [14]。

此外，大模型可用于对检索结果的初步筛选或摘要生成，为是否触发下一步实时爬取或展示决策提供支持。模型还能够在质量评估阶段对爬取内容进行语义一致性检查与关键信息抽取 [15]，从而提升二次清洗的自动化水平。然而，将大模型引入实时系统也带来了挑战，包括推理延迟、资源消耗与模型输出的可靠性问题 [16]。为此，实际工程常采用混合策略：在延迟敏感路径上使用轻量化或蒸馏模型，而在非实时或离线环节使用更强的模型以保证质量；同时引入缓存、异步化与并行化机制以缓解延迟瓶颈。

1.2.3 实时检索与离线垂域检索的工程需求与应用场景

实时检索强调低延迟与高新鲜度 [17]，应用于热点事件监测、即时问答与短时话题聚合等场景。工程上需要解决的问题包括实时索引或近实时索引更新 [18]、并发与超时控制 [19]、快速判定是否需要发起深度爬取 [20] 以及保证爬取与解析在可接受延迟内完成 [21]。实时爬取除了常规的反爬、限速与容错外，更要求解析模块能处理流式数据并将结构化信息及时下发至后续清洗与入库流程，从而缩短从采集到可检索之间的时延。此外，实时检索系统还需处理并发与超时控制、快速判定是否需要发起深度爬取以及保证爬取与解析在可接受延迟内完成。实时爬取除了常规的反爬、限速与容错外，更要求解析模块能处理流式数据并将结构化信息及时下发至后续清洗与入库流程，从而缩短从采集到可检索之间的时延。在爬取侧，基于流式处理框架构建的低延迟可扩展爬虫可将 URL 抓取与解析作为持续流处理 [22]；在实时数据摄入侧，基于 CDC 的变更数据捕获管道可将源数据库的增、删、改操作实时流式同步至向量数据库，避免批量更新的滞后问题。多模态实时数据系统也展示了跨文本、向量、空间等不同数据模态的连续查询与高吞吐摄入能力。

离线垂域检索则更多面向内容丰富、具有较高语义价值或社区特征的数据源，如小红书、知乎等。离线通道的关键在于高质量索引构建、向量化一致性、批量数据清洗与标签化，并保证在检索时能够提供领域内高度相关的候选项 [23]。离线数据通常需要更复杂的质量评估机制、去噪与样式适配，以及周期性更新策略以兼顾新旧数据的平衡 [24]。现实应用中，实时与离线通道往往需要协同工作：实时通道补充新鲜度，离线通道提供深度与专业性。因此工程上需设计统一的调度与融合机制，使两类通道在延迟预算内实现最优召回与排序效果 [25]，同时保证系统的可运维性与扩展性。

1.2.4 查询算法的分类和发展现状

查询算法作为信息检索领域的核心技术，旨在通过高效的搜索策略从海量数据中获取与用户查询最相关的结果。随着大数据和人工智能技术的飞速发展，查询算法的研究与应用也经历了显著的变革。尤其是在倒排、向量和重排等技术的结合与发展下，查询算法的性能得到了显著提升。本文将重点回顾倒排、向量和重排技术的研究进展，并分析它们在查询算法中的应用现状与未来趋势。

倒排技术是传统信息检索中最为基础且关键的组成部分之一。倒排索引通过建立词项到文档的映射关系，大大提高了查询过程中的效率。在倒排索引的构建中，首先会对所有文档进行分词、去除停用词并记录词频，从而生成一个倒排表格，方便快速查询。该技术的优势在于其结构的紧凑性和查询效率，特别是在处理大规模数据集时，倒排索引能够显著减少搜索时间。虽然倒排技术在早期已取得较大成功，但随着数据规模的不断扩大，传统的倒排索引方法在处理复杂查询、多语种信息和多模态数据时逐渐显现出局限性。因此，近年来，研究者开始探索更为高效的倒排结构，例如基于压缩技术的倒排索引，以及多层次、分布式的倒排索引方法，旨在进一步提高查询效率和存储优化。

与倒排索引相辅相成的是向量空间模型，该模型通过将文本表示为向量形式，进而计算文档与查询之间的相似度。向量空间模型在信息检索中扮演着重要角色，尤其是在查询结果排序和相关性度量方面。在该模型中，每个文档被表示为一个高维向量，向量的维度对应于文档中的词汇。通过计算查询向量与文档向量之间的相似度，查询算法能够筛选出与用户查询最为相关的文档。然而，传统的向量空间模型也面临着数据稀疏性、向量维度过高等问题，导致查询效率和效果的下降。为了解决这些问题，近年来，研究者提出了基于降维、主题模型以及词嵌入技术的向量表示方法，这些方法能够在一定程度上缓解数据稀疏性问题并提升查询精度。

在倒排索引和向量模型的基础上，倒排向量重排技术逐渐成为提升查询精度的重要研究方向。倒排向量重排旨在通过对初步查询结果进行重新排序，进一步提高检索结果的相关性。在传统的信息检索中，倒排索引能够快速检索出相关文档，但往往无法准确区分文档之间的细微差异，导致检索结果的准确性不足。为此，研究者引入了重排机制，结合机器学习、深度学习等方法，通过学习文档和查询之间的复杂关系，调整倒排结果的排序。最初的倒排重排方法多依赖于传统的排序算法，如支持向量机、决策树等，这些方法通过手工设计特征进行训练，尽管在某些场景下取得了较好效果，但其对特征工程的依赖较大，且对复杂查询的处理能力有限。

近年来，深度学习技术的引入为倒排向量重排带来了新的突破。基于深度神经网络的倒排重排方法能够自动学习更加复杂的特征，避免了传统方法中繁琐的特征工程。例如，基于卷积神经网络和循环神经网络的模型，能够捕捉到文档与查询之间的语义关系，提高重排的准确性。此外，强化学习的应用也为倒排重排提供了新的思路，模型通过与环境的交互不断调整重排策略，优化查

询结果的质量。基于深度学习和强化学习的倒排向量重排技术，在多个实际应用中，如搜索引擎、推荐系统和问答系统等，展现出了优越的性能。

1.3 研究目标与任务

本研究的总体目标是设计并实现一个以模型驱动的 Search Agent 为核心、能够在 MCP 协调下对接实时在线与离线检索后端的工程化检索流水线，满足低延迟响应、高检索质量与可部署运维的要求。MCP 通过统一的客户端—服务器协议实现了 AI 代理与多种数据源之间的通信标准化，为检索系统提供了可扩展的架构基础 [26]。在 MCP 生态中，ETOM 评估框架进一步对多跳端到端工具协调能力进行系统诊断，发现检索错误占所有失败原因的近一半，凸显了检索环节在 MCP 协调架构中的瓶颈地位 [27]。为实现该目标，论文将完成以下具体任务：

首先，构建 prompt-to-query 模块，采用预训练模型结合规则化后处理方法实现对用户自然语言提示的语义分词、关键词提炼与检索意图判别，并给出置信度评估与可控的输出格式以便于下游调用。在意图检测方面，SAID 框架通过同时建模文本信息与查询—答案关系结构，在小样本场景下显著提升了查询意图检测的准确性。在查询改写方面，AdaRewriter 采用测试时自适应策略，利用轻量级奖励模型在推理阶段从候选改写中筛选最优查询，在多个对话式检索数据集上取得了显著优于现有方法的性能 [28]。

其次，设计实时在线检索通道的时序与控制策略，包含热点定时检索、超时与并发控制、检索结果的实时下发机制，以及在关键词指示需要详细展示时触发的 Python 实时爬取模块，爬取模块需包含动态限速、超时处理、实时解析与断点容错能力。在实时数据采集架构方面，FediLive 框架面向去中心化社交网络构建了高并发、高容错、易扩展的跨实例爬取与数据预处理系统，通过动态限速、轮询去重、ID 冲突解决及多线程调度等技术，在有限计算资源下实现了大规模实例的全平台采集，为实时爬取模块的设计提供了可复用的工程方案。

第三，构建离线检索通道的索引与向量化管线，完成离线数据的采集、批量清洗、向量化建库与倒排索引的增量更新策略。在数据清洗环节，LOCA 框架通过“增强—审阅”循环自动补全缺失的逻辑步骤并区分科学原理与推导过程，能够将科学语料中的错误率从 20% 降低至 2% 以下，展示了大语言模

型在自动化数据清洗与质量评估中的潜力 [29]。在向量索引的增量更新方面，LiveVectorLake 提出了双层时间知识库架构，将热层向量索引与冷层列式版本存储分离，实现了当前知识的毫秒级检索与历史版本的点时间追溯。

第四，搭建端到端的数据处理流水线，包括实时解析、二次清洗、质量评估、ETL 与入库，并实现基础监控、日志与故障恢复机制以保证系统的工程可用性。

通过上述任务的实施，最终在吞吐与延迟、检索与排序质量、新鲜度以及工程指标上验证系统的实用性与可部署性。

1.4 论文结构安排

本文结构如下所述：

第一章为绪论，主要介绍研究背景与意义，结合当前信息检索技术的发展趋势，分析混合检索与大语言模型结合的研究价值。同时，对国内外研究现状进行梳理，重点涵盖信息检索技术演进、大模型在提示解析与检索驱动中的作用，以及实时检索与离线垂域检索的工程需求。在此基础上明确论文的研究目标与主要任务，并对全文结构进行总体说明。

第二章为技术综述，系统梳理本文所涉及的关键技术与工具，包括开发与性能工具、数据监控体系以及混合检索中的核心算法，如 Lucene 倒排索引、Faiss 向量检索及学习排序模型等，为后续系统设计提供技术基础。

第三章为系统需求分析与总体设计，是全文核心部分之一。首先从系统功能与业务流程出发，对 Prompt 解析、实时检索、离线检索、索引构建等模块进行需求分析；随后从逻辑视图、过程视图、开发视图与物理视图四个层面进行系统架构设计，并详细说明各模块划分及其交互关系，同时完成数据库与索引存储方案设计。

第四章为系统实现，具体阐述各核心模块的实现方法，包括 Prompt 解析、实时检索与爬取、离线检索与索引构建以及召回融合与重排机制，重点体现系统工程落地过程与关键技术细节。

第五章为系统测试，从单元测试、功能测试及非功能测试三个层面，对系统的正确性、稳定性与性能进行验证，确保系统满足设计预期。

第六章为总结与展望，对全文工作进行归纳，并对未来可能的优化方向与研究拓展进行讨论。

第二章 技术综述

系统架构包括多个核心模块，如版本控制、日志管理、后端开发框架、缓存加速、消息队列服务等，这些模块构成了系统的技术基础。此外，针对数据监控和安全性问题，本系统还集成了 Prometheus、Alertmanager 等监控工具，以实现全程的运维监控与故障预警。在混合检索方面，系统使用 Lucene 进行文本检索、Faiss 进行向量检索，并结合特征排序算法实现倒排重排。最后，针对实时数据处理需求，系统还提供了增量建库和实时爬取相关算法的支持。第二章将详细介绍这些技术和工具的应用及其在系统中的角色。

2.1 开发与性能工具

2.1.1 版本控制与代码托管：Git

版本控制是工程可重复性、协作与发布管理的基础。本项目采用 Git[30] 作为分布式版本控制系统，结合集中化的代码托管平台实现代码审查、分支策略和 CI/CD 的触发点。针对 Search Agent 这样既有后端服务又有爬虫脚本、模型推理代码与配置文件的复杂项目，推荐实施明确的分支策略：主分支保持可生产态，开发分支作为集成流，功能分支按任务切分，修复分支用于紧急修复。代码提交须配合规范化的提交信息模板与变更说明，PR 流程应强制代码审查，并在合并前通过静态检查、单元测试与安全扫描。

在具体工程中，建议将不同模块采用子目录或单独仓库管理，根据变更边界决定是否采用 mono-repo，以便在 CI/CD 中精确触发相关流水线。对于模型与大体积静态资源，可以使用 Git LFS 或专门的模型仓库，并在代码仓库中引用版本号以保证可重复性。与版本控制紧密结合的是代码审计与回滚策略：所有生产发布都应打 tag，保留构建产物与发布日志，发生问题时能够快速回滚；对于数据库迁移或索引变更，应设计滚动回退方案与兼容逻辑，避免无法回滚的破坏性迁移。

此外，Git 与 CI 的集成应支持自动化质量门：静态检查、单元/集成测试、

容器镜像构建与扫描，以及性能回归的快速 smoke 测试。对 Search Agent 这种对延迟敏感的服务，CI 中增加轻量化的延迟/吞吐基线测试有助于早期发现性能回归。最后，强制在 PR 描述中添加影响范围和回归测试点，便于 reviewers 关注关键风险点并减少生产事故。

2.1.2 线上变更管理平台：MCM

线上变更管理平台 MCM 承担从代码到生产的发布闭环，协调构建、验收、灰度、回滚与审计。对于 Search Agent 和关联数据流水线而言，MCM 的角色尤为关键：它不仅负责发布微服务镜像或配置，还需管理与 MCP、爬取任务调度、消息队列主题、索引迁移等交叉依赖的变更。工程实践上，MCM 应提供可视化的发布流水线，并支持多维度的审批流，以满足线上变更的合规性与可追溯性。

在变更策略方面，MCM 要支持金丝雀发布与按流量比例的灰度策略，以便逐步放开 Search Agent 的新逻辑，在小范围内观察对延迟、错误率和爬取负载的影响。平台需能与服务发现与流量控制层联动，自动调整路由权重并在异常时回退。此外，MCM 应集成回滚与补偿操作的自动化脚本：当发现模型输出异常或爬取压力导致 downstream 队列堆积时，可一键回滚到上一个稳定版本并自动清理或重排待处理任务。

安全与合规性也是 MCM 的核心功能之一。平台应记录每一次变更的签名、审批记录、变更影响范围以及回滚操作日志，便于事后追溯。对涉及敏感配置的变更，MCM 可与密钥管理系统结合，控制运行时权限并在变更前后做安全检查。最后，MCM 应支持灰度实验与 A/B 测试的集成，允许通过相同平台管理检索质量实验并收集指标以决定是否推广。

2.1.3 日志与调试工具：Log4j

日志是分布式系统观察性（observability）的基础。本项目以 Log4j[31] 作为服务端日志记录的统一方案，约定日志格式、日志级别与采集规范，确保在复杂请求链路中能够快速定位问题。对 Search Agent 服务而言，应在关键路径上放置结构化日志，包含统一的 trace_id、span_id、request_id、用户上下文、时间戳、耗时与状态码等字段，便于后续通过 ELK/EFK 堆栈进行聚合查询与可视化。

日志级别的设置需兼顾诊断需求与性能/存储成本：INFO 记录业务成功路径与关键事件，WARN/ERROR 记录异常和可疑情况，DEBUG/TRACE 用于开发与问题定位但默认线上关闭，必要时通过动态配置按服务实例或按请求进行临时降级以排查问题。对于高吞吐组件，应采用异步日志写入与批量发送机制以降低 I/O 阻塞；并配置日志滚转、压缩与生命周期管理，防止磁盘被日志淹没。

在日志隐私与合规方面，必须避免将敏感 PII 以明文写入日志。需要在日志层或前置过滤器中做脱敏/哈希处理，并对某些敏感字段设置短期保留策略。调试工具方面，结合 Log4j 输出与分布式追踪，可以实现请求链路的端到端还原：当某个检索请求触发了爬取并最终入库，可以通过 `trace_id` 快速找到每一步的时间消耗、异常与上下游消息。为提高调试效率，建议在日志中统一输出关键指标，并在异常触发时自动将相关上下文推送到事故分析系统，以便快速恢复。

2.1.4 配置与密钥管理：Lion

配置管理与密钥管理是保证分布式服务可靠运行的重要基础。以企业级配置中心：Lion，它应支持动态配置下发、灰度配置、版本回滚、配置回滚与不同环境（dev/test/prod）隔离。对于 Search Agent，配置项包括 MCP 服务地址与超时、后端检索路由权重、爬虫并发/超时策略、模型版本与推理参数、消息队列 topic 配置、索引写入策略等。动态下发能力使得在不重启服务的前提下调整限流阈值或切换模型版本成为可能，极大提升线上应对突发的灵活性。

密钥管理方面，Lion 应与企业的密钥/凭证管理（Vault、KMS）集成，提供加密存储与运行时解密能力。对涉及第三方抓取的认证信息、数据库/向量库写权限、API Key 等，必须通过密钥管理系统进行周期性轮换与访问审计。服务在启动阶段仅获取必要的运行凭证，且建议使用短期凭证（STS）与最小权限原则来减少泄露风险。配置下发时需区分静态配置与动态配置，并对敏感配置增加审批流程与变更回溯。

工程实践上，配置中心应支持按服务实例或分组进行灰度配置，并与 MCM 联动以实现发布时的全链路一致性。配置变更也应触发自动化回归检查，并在变更前后采集关键指标以判断变更效果。最后，配置与密钥访问应纳入权限管理体系（RBAC），并记录访问与变更日志以便审计与合规。

2.1.5 后端开发框架：MDP 美团开发平台

企业级开发平台:MDP 常为后端服务提供约定的工程模板、统一的服务框架、内置中间件和开箱即用的能力。在本项目中，借助 MDP 之类的平台可以大幅降低样板代码、统一接入企业级网关与服务治理体系，使 Search Agent 的开发更专注于业务逻辑而非基础设施建设。MDP 通常提供微服务启动模版、统一的配置读取器、服务注册与发现、熔断限流拦截器、拦截链式的鉴权与审计插件，以及自动化的健康检查与重启策略。

对于模型推理与高并发调用场景，MDP 的中间件也能提供线程池、异步调用封装与限流策略，帮助避免单个组件因模型推理阻塞而拖垮整个服务。在与 MCP、消息队列或外部爬取服务交互时，可利用平台的 SDK 实现一致的重试/退避/超时策略与调用链追踪，保证调用方与被调用方在失败场景下有一致的恢复行为。MDP 平台还可能内置灰度与特性开关支持，与 MCM 与 Lion 协同实现灵活发布与配置控制。

在工程落地上，使用 MDP 也需注意对外部依赖的隔离与可替换性设计：例如 Search Agent 的模型服务调用应以接口抽象，避免平台框架升级导致业务逻辑难以迁移；爬虫任务管理、消息订阅等也应设计为可插拔模块，以便在不同运行环境或大规模扩容时进行独立扩展。最后，平台化带来的监控与指标收集能力应被充分利用：将 Search Agent 的关键指标打点并上报到统一监控体系，以便实现自动化告警与容量预判。

2.1.6 缓存与加速：Redis

Redis[32] 作为内存型键值存储，常被用于缓存热点数据、会话管理与轻量级的消息协调。在 Search Agent 场景中，Redis 可发挥多重作用：首先作为热点 Cache，缓存近期或高频查询的检索结果或聚合后的摘要，显著降低对后端检索与爬取的频繁调用，减少延迟与成本；其次可用于存储短期状态以支持幂等与断点续传；再次，Redis 的 Pub/Sub 或 Streams 模式可作为轻量级的事件通道，用于连接 Search Agent 与实时解析/入库消费者的低延迟通知。

在使用 Redis 时要注意缓存一致性问题：采用 Cache-Aside 是常见模式，但在写入/更新索引或入库后，需要合理设置失效策略与主动失效机制以避免脏读。对于实时性强且在短时间内频繁更新的数据，应设置较短的 TTL 或采用基于版本号的验证机制。Redis 内存管理与 key 设计对性能至关重要。对大规模热

点缓存，还需考虑分片策略与本地缓存结合，减少网络往返。

并发场景下，Redis 还可提供分布式锁与原子操作（Lua 脚本）来协调并发爬取或索引重建任务。但应谨慎使用分布式锁：锁超时、网络分区会引起死锁或资源竞争，推荐以幂等任务设计与乐观并发控制为主，分布式锁作为补充。最后，Redis 的监控（内存使用、慢查询、key 过期/eviction 率）需要纳入整体可观测体系，以预防缓存击穿或内存溢出对系统的连锁影响。

2.1.7 消息队列服务：Mafka

在分布式检索与数据处理流水线中，消息队列 Mafka 承担解耦、削峰与可靠传输的职责。Search Agent 与 MCP、爬虫、实时解析服务、二次清洗模块、索引入库组件之间通过主题进行异步通信：例如将触发的爬取任务写入爬虫任务队列，爬取结果写入解析队列，解析后写入清洗与入库队列。消息队列支持消费者组扩容来增加吞吐，同时通过分区保证消息有序性需求。

消息语义影响业务设计。大多数爬取及解析场景可接受至少一次传递语义，但入库环节若要求幂等性，应在消息中携带唯一 ID 并在消费者侧实现幂等写入。消息格式应采用 schema，并配合 schema registry 以支持兼容变更。消息保留策略（retention）与压缩策略应根据业务需求设定：实时通道可能需要较短保留但更低延迟，离线通道希望长期保留以便回放重跑。

对于高并发与高吞吐场景，需关注消息堆积与滞后：当爬取速率超过后端解析或清洗吞吐时，会出现队列积压，系统需触发回压（backpressure）或自动降级以避免拉跨下游服务。Mafka 还应与监控系统集成，暴露主题的 lag、吞吐与消费速率，支持告警。最后，消息系统的安全、多租户隔离与运维也是长期稳定运行须考虑的运维成本。

2.1.8 远程过程调用：RPC

RPC[33] 是微服务间通信的主要方式。Search Agent 与 MCP、检索后端、模型推理服务、索引写入服务等之间通常采用 gRPC、Thrift 或企业自研 RPC 框架实现高性能的二进制协议调用。RPC 设计需关注接口定义（IDL）、向后兼容性、序列化效率、超时与重试策略、幂等性与熔断机制。对延迟敏感的调用路径，应设置较短的超时并实现快速失败和快速降级，避免单一慢节点拖垮整体请求。

负载均衡与服务发现是 RPC 系统的另一个关键点：客户端应支持简单轮询、加权轮询与基于延迟的智能路由，配合服务注册中心实现可用实例选择。链路级别需要集成分布式追踪以便观测请求跨服务的耗时分布。错误处理方面，需要区分可重试的暂时性错误与不可重试的业务错误，重试策略要配合指数退避及最大重试次数，以防止雪崩效应。对于流式交互，RPC 框架应支持流式或双向流通信。

安全性方面，RPC 通信需要 TLS 加密、鉴权、访问控制列表和限流来防止滥用。版本管理与灰度发布也应在接口层面设计兼容策略，并在 MCM 管理的灰度过程中逐步切换新接口。最后，RPC 的性能与稳定性直接影响用户体验，需在开发阶段引入 RPC 的基准测试、超时注入与故障演练以提前发现潜在风险。

以上各节围绕 Search Agent 的工程实现场景展开，既涵盖工具链使用要点，也强调了在项目中实际落地的设计与运维考虑，为后续第 3 章系统设计与第 8 章部署运维章节提供了技术基础与选型依据。若你需要，我可以基于这些内容继续生成每节的引用资料、图示示意或与具体开源组件的对比表。

2.2 数据与服务器监控平台

2.2.1 指标监控与观测性能：Prometheus

Prometheus[34] 是目前云原生监控的事实标准，其以拉模式采集指标、时序数据库存储指标、并配套 PromQL 用于聚合与告警计算。对于 Search Agent 这样包含 prompt 解析、模型推理、检索后端、实时爬取、解析清洗与入库等多模块的系统，Prometheus 提供了端到端可观测能力：从主机与容器层的资源指标，到应用层的业务指标，再到中间件的队列/滞后指标都可统一采集和分析。

工程实践上，应在各服务中通过官方 client 或现成 exporter 暴露结构化指标。建议按照统一命名规范与标签约定设计度量。延迟类使用 histogram 以支持精确的分位数计算与误差控制；计数器用于请求量与错误计数；Gauge 用于当前队列长度、内存使用等瞬时值。标签设计需控制基数，常用标签包含 service、instance、env、backend、model_version、region。

Prometheus 的配置应针对不同指标采取不同抓取间隔：核心请求链路使用短抓取间隔，但部分低频或高维指标可用较长间隔以降低压力。对于短寿命或批量作业，可使用 Pushgateway 或将任务状态写入 Kafka 并由 long-

running consumer 将指标转换为时序指标。为支持长期查询与告警历史，推荐与 Thanos/Cortex 等长期存储方案结合，实现多实例 HA、全局聚合与跨地域查询。

在实践中，应编写 recording rules 将常用的聚合预计算成新的时间序列，以减少 PromQL 的计算成本和告警延迟；同时设计丰富的 Grafana dashboard：总体服务健康面板、爬虫面板、索引面板以及中间件面板。最后要规划指标保留策略、压缩与 downsampling，以及 Prometheus 的 HA，并对 Prometheus 自身资源、scrape timeout、目标超高基数做容量预估。

2.2.2 告警与 SLO/SLI 管理：Alertmanager

Alertmanager[35] 与 Prometheus 告警规则配合使用，承担告警聚合、抑制、分组与路由到不同通知渠道。在工程化检索系统中，应先定义清晰的 SLI 与 SLO，再将 SLO 映射为 Prometheus 告警规则。示例 SLI/SLO 包括：可用性、尾延迟、实时通道新鲜度、爬取成功率 > 99%、索引可见延迟 < 60s 等。

基于这些 SLO，构建分级告警策略：Severity=critical 的告警应触发即时 paging 并包含明确的 runbook；Severity=high 发送到运维/平台频道并创建 incident ticket；低优先级发送到团队 Slack 频道作为观察信号。

告警规则应避免噪声（noise）和抖动：引入速率/滑动窗口避免瞬间抖动触发告警；使用 Alertmanager 的抑制规则来避免告警风暴。告警消息应包含：简短问题描述、受影响实例/服务、触发时间、关键指标与当前数值、影响范围、推荐快速修复步骤与回滚入口。为提升响应效率，每个告警要绑定明确的值班与升级策略，并在告警文档中写明“首要检查项”。

在告警运行化方面，建立告警演练和 SLA 报告周期，使用 Alertmanager 的 silences 管理计划内维护窗口。结合 A/B 实验与 MCM 的灰度发布机制，将变更评估纳入告警策略：如新 prompt-to-query 策略上线后，应短期提高监控灵敏度并设定临时告警阈值以快速发现偏差。为实现长期告警与 SLO 报告，建议将告警事件与 incident 管理系统联动，并在每次 incident 后产出 RCA 与改进措施。

2.2.3 安全监控与审计：Auditd

安全审计是保障系统合规性与可追溯性的基石。Auditd 在内核层捕获关键系统事件，并将审计日志写入本地或转发至集中审计系统。对 Search Agent 项

目而言，应重点审计与敏感资源交互有关的操作：模型与 `embedding` 文件的访问、密钥/凭证文件的读取、索引库与对象存储（S3）写入权限的变更、爬虫运行时的异常进程启动以及特权端口的绑定等。在规则层面，应通过 `auditctl` 定义细粒度规则，生成的审计事件需要通过 `Filebeat` 转发至安全信息事件管理或 ELK 平台，结合 SIEM 规则做实时检测。

审计日志需要保证完整性与不可篡改性：建议将日志写入不可变存储或使用 `append-only` 的集中日志集群，并启用签名或哈希校验以便事后核验。针对高价值资产，应实现访问审批与多因素验证，并在审计系统中生成告警。同时，审计策略需考虑隐私合规性，避免在审计日志中直接保存用户完整 `prompt` 或包含 PII 的抓取内容，必要时做脱敏或加密处理，并将脱敏策略纳入审计流程。

运维方面，应将 `auditd` 与整体监控体系协同：当审计检测到异常时，不仅在 SIEM 中生成安全事件，还应触发 `Prometheus` 告警以联动运维与安全团队。审计结果应定期进行安全审查，并作为事故分析的重要证据。最后，审计策略需要与组织的合规要求对齐，设定日志保留期、访问控制与审计报告模板，并在重大变更前进行安全评估与审计规则更新。

2.3 混合检索中使用到的算法

2.3.1 倒排索引与文本检索算法：Lucene

倒排索引 `Lucene` 是传统文本检索系统的基石，其核心思想是为每个词项维护一个 `posting` 列表，记录词项出现的文档 `id`、词频、位置信息等辅助字段。基于倒排索引的文本检索将查询解析为一个或多个 `term` 查询，通过合并 `posting` 列表、计算匹配文档的相关性得分得到候选集合并排序。`Lucene` 是行业内最成熟的全文检索库之一，它在倒排索引结构、管理、压缩编码、查询解析与评分模型实现方面提供了完整且高效的实现，常作为离线索引或近实时索引的底层引擎，以及被 `Elasticsearch/OpenSearch` 等封装成分布式搜索服务。

工程实现层面的关键点包括：分词器与分析链决定了中文/英文、短语/口语化 `prompt` 的表示；字段映射影响功能与存储成本；索引组织采用 `segment` 机制，通过内存缓冲写入、定期 `flush`、后台 `merge` 来平衡写入吞吐与查询性能。对于实时通道，需要支持近实时能力：`Lucene` 的 `refresh` 可以在秒级内暴露新写入文档，但频繁 `refresh` 会导致 `segment` 激增并带来合并压力，因此需要在可搜索延迟与系统稳定性之间权衡。

检索质量由多个环节共同决定。Query-side 的扩展能显著提升召回；同时需要注意短文本特有的问题：短 query 导致稀疏信号，需要通过 query expansion、pseudo-relevance feedback 或基于模型的 query rewriting 提供更完整的语义表达。Rank-side 的基线评分一般作为召回与前期排序手段，后续再由更强的 reranker 精排。

在分布式场景下，还需考虑分片路由、副本策略、查询 fan-out 与聚合延迟。工程上常采用 hybrid 策略：倒排索引用于精确匹配与高吞吐基础召回，向量检索用于语义召回，两者并行执行并在合并阶段做 score normalization 与去重。对于你项目，离线垂域数据（XHS、知乎）用倒排索引承担文本检索通道；Search Agent 在向倒排系统下发查询时应携带模型提取的实体/意图/扩展词以提高匹配命中率，并在实时通道中将检索结果作为触发爬取或二次解析的判断依据。

实践建议包括：合理设置 refresh 策略以兼顾实时性与稳定性；高频查询做热点缓存以降低后端压力；对索引字段与分词器进行严格 schema 管控以保证离线批量导入与在线增量写入时的一致性；以及对查询日志做长期分析以指导同义词表、停用词和 query rewriting 的迭代。

2.3.2 向量检索与近似最近邻算法：Faiss

Faiss[36] 语义检索依赖 embedding 将文本或段落映射为稠密向量，随后通过最近邻搜索找到语义相似的内容。对于大规模数据，精确最近邻计算代价高昂，近似最近邻（ANN）算法因此成为主流。Faiss 是工业界广泛使用的向量检索库，提供了多种索引结构与 ANN 算法实现，包括基于图的 HNSW、基于倒排的 IVF 以及基于量化的 PQ/OPQ 等，且支持 CPU/GPU 加速。

在算法选择上存在明显权衡：HNSW 能在较高召回率下提供低延迟查询，是在线低延迟场景的优选，但内存占用较大；IVF-PQ 适用于海量向量且能通过压缩降低内存，但需要训练步骤并通过 nprobe 参数控制召回/延迟 trade-off。工程上常采用分层策略：用较廉价的倒排或低维 ANN 做初步召回，再对小候选集执行精确距离计算或 cross-encoder 精排。

向量索引的工程要点包括：embedding 的稳定性（同一模型与预处理规则在离线和线上必须一致）、嵌入维度与量化配置、索引构建与增量更新策略、分片与副本策略以及查询参数调优。实时性挑战在于新抓取内容如何尽快可检索：Faiss 本质上对批量构建更友好，在线插入支持但可能带来碎片或性能开

销，因此常见方法包括异步批量插入、增量热数据放到内存表/小型 HNSW，而定期将热数据合并回主索引。

另外需要关注向量距离的归一化与 **score calibration**。在混合检索中，文本检索与向量检索输出的分数尺度不同，直接合并容易导致偏向某一路径。常用做法是对分数做 **min-max** 归一化、或采用学习型融合器来融合倒排与向量召回。去重逻辑也需在合并阶段处理，以避免重复条目影响重排效果。

资源层面，Faiss 支持 GPU 加速但引入 GPU 资源管理与成本问题；若使用专门的向量数据库，可获得运维便利性，但可能在配置灵活性和成本上有所权衡。对你的项目，离线垂域数据更适合在向量索引中做语义召回，实时抓取的新内容可先写入短期内存索引以保证新鲜度，随后进行离线合并重建以维持稳定性与压缩率。

2.3.3 重排与排序模型：feature-based LTR

现代检索系统普遍采用两阶段架构：第一阶段召回获取较大候选集，第二阶段重排对候选进行精细排序以改善最终呈现质量。重排模型通常分为基于特征的机器学习模型与基于 Transformer 的 **cross-encoder**。基于特征的 LTR[37] 在工程中应用广泛，原因是推理延迟低、可解释性强、易于特征扩展与线上部署，适合作为在线低延迟重排的主力。

构造丰富的 **query-document** 特征、上下文/用户特征、行为特征以及质量特征。对于爬取内容，还应加入爬取可靠性特征与结构化抽取质量分。训练 LTR 需要可靠的标注。在实时/离线混合场景中，点击信号可能偏向离线内容或对新鲜内容不友好，因此需要进行样本重加权或采用多任务学习来缓解偏差。LightGBM 等树模型支持高效并行训练与低延迟推理，可部署为在线服务或通过模型导出为静态 **ensembling**。对 **top-K** 候选使用 **cross-encoder** 做二次精排可进一步提升质量，但其推理成本较高，应限制在非常小的候选集上。为降低延迟，可使用蒸馏技术将 **cross-encoder** 的知识蒸馏到轻量模型。模型需要定期重训练以适应数据分布变化。对新上线的检索策略或新语义特征，需要设计冷启动策略。

重排需在整体延迟预算内完成，通常采用分层重排：第一阶段用轻量 LTR 做快速排序、第二阶段对 **topN** 做 **heavier rerank**。对实时爬取触发的展示场景，要求快速返回预览及异步补全，这对重排系统的异步处理能力提出要求。部分特征计算昂贵，工程上通过预计算、特征缓存、按需计算和流水线化降低

成本。使用树模型便于解释，并在 CI/CD 引入模型性能回归检测以防止上线降级。当候选来自倒排与向量通道时，需对不同通道的原始分数做归一化或使用学习型融合器将多通道特征一并输入 LTR 模型，从而实现统一评分与去重。在你的 Search Agent 中，推荐采用基于特征的 LTR 做在线主重排，保留 cross-encoder 在离线或半实时的顶端精排任务。特征应覆盖检索源、爬取/解析质量指标、以及 prompt-derived intent 特征，以在混合召回的候选上做出合理排序。

2.3.4 实时爬取与解析相关算法

实时爬取要求在短时间内抓取目标页面并抽取结构化信息以满足展示或入库需求，其与传统批量爬取有明显不同：更强的时效性、并发控制要求以及对解析可靠性的更高要求。实时爬取管线由任务调度、下载器、页面解析/正文抽取、结构化抽取与质量校验组成。

任务调度与优先级：热点定时检索产生的任务常具有严格时效窗口，调度系统需按优先级、并发配额与目标站点速率限制分配抓取资源 [38]。常用算法包括优先队列，以及基于令牌桶/漏桶的速率限制器来保证对目标域的友好度。对于高优先级的展示触发请求，采用快速路径而非深度抓取。

静态页面可用高效的 HTTP 客户端下载，而现代站点常含大量 JS 渲染内容，需要 headless 浏览器或使用 SSR/API 替代方案。Headless 浏览器成本高，工程上常采取策略：先尝试纯 HTTP 抓取并用快速可行的 heuristic 决定是否降级到浏览器渲染；或使用轻量化浏览器池与预渲染缓存以降低成本。抓取到 HTML 后需去除 boilerplate，常用算法包括基于规则的 readability，以及基于 ML 的正文判别 [39]。实体抽取与结构化字段抽取可用传统规则/正则，也可用命名实体识别（NER）、关系抽取模型或表格解析器 [40]。对实时解析，优先使用高精度低延迟的混合方案：规则先行、模型校验。为避免重复入库与冗余解析，需在抓取前后进行 URL 规范化、指纹计算以及近似去重 [41]。SimHash 可以高效检测相似页面，适用于大规模实时流；对于更精确的去重，可结合向量相似度阈值做二次确认。实时爬取必须严格控制超时，并设计重试与退避策略 [42]。任务应具备幂等性与断点续传，同时在失败场景下将错误上下文作为日志与报警信息返回给 Search Agent 以便决策。实时爬取与解析产生的数据需进行自动质量评估：字段完整度、正文长度、语义一致性检测、来源可信度评分 [43]。对低质量结果可以设置降权或异步补救（重试、人工审核）。此外，

实时路径应配合打点与监控以便在 MCM 中设定 SLO 并触发回退策略 [44]。

2.3.5 索引更新与增量建库策略：bulk

索引更新策略决定了数据新鲜度、资源消耗与系统稳定性。两类常见策略是批量 bulk 导入时更新，各有利弊。Bulk 适用于离线大规模重建：通过批处理对清洗后的数据进行统一向量化，优点是构建效率高、压缩比好、索引结构优化良好；缺点是新数据可搜索延迟高，不利于时效性要求。NRT 更新能在秒级或分钟级使新文档可搜索，但频繁小批次写入会导致碎片化、内存/存储开销增长与合并压力。

对批量稳定数据采用定期 bulk 重建，对高优先级实时数据使用 near-real-time 或短期内存索引。具体实现方式包括：将实时写入保存为小索引，查询的同时搜索主索引与 delta 索引，定期将 delta 合并进主索引。这种“双索引”模式已在现代向量数据库中得到验证，其核心思路是用轻量增量索引承接实时写入，与主索引分离以降低写入放大，同时保证新数据立即可查 [45]。这样既保证了低延迟暴露，又能在合并时做压缩与优化。写入队列 + 异步批入：实时写入先落在消息队列，由消费者做批量聚合后以合适大小批次提交到索引，既降低了写放大也保障了吞吐。对于 Faiss 类型索引，常把新向量写入轻量内存索引用于短期可搜索，后台周期性地将这些热向量并入主索引并重训练 [46]。使用索引版本号或别名配合蓝绿部署（blue-green），在 bulk 重建完成后切换别名原子化对外生效，从而避免查询时的不一致。对于 NRT 更新，需关注写入的原子性与幂等性，使用唯一 ID 与去重幂等策略。bulk 重建消耗计算与 IO 峰值，应安排在低峰窗口并支持预取/分片并行；NRT 更新需要更精细的 merge policy 设置与资源配额以避免长时间 GC 或磁盘碎片化 [47]。

2.4 本章小结

本章围绕 Search Agent 系统的关键支撑技术展开系统性梳理，从工程工具体系、监控平台以及核心检索算法三个层面构建了完整的技术基础框架。这些工具共同构成了系统稳定运行与高效开发的基础环境。通过上述技术的系统梳理，不仅明确了各类技术在整体系统中的功能定位，也为后续系统架构设计与模块实现提供了坚实的理论与工程基础。

第三章 实时与离线混合检索系统的需求分析与设计

3.1 系统总体概述

本章节首先从总体层面描述 Search Agent 系统的定位、功能边界与端到端工作流，为后续各模块的需求与设计奠定基础。Search Agent 的核心目标是在 MCP 协调下，作为“提示（prompt）→ 检索/爬取 → 解析清洗 → 索引入库 → 结果返回”这一端到端流水线的执行与调度者。系统需要同时对接两类检索后端：一类是实时在线检索（以 bin search 为代表），具备热点定时检索与对检索结果触发 Python 实时爬取的能力；另一类是离线垂域检索（如小红书、知乎），以批量同步的方式提供丰富垂域内容，支持基于倒排索引的文本检索与基于向量索引的语义检索。Search Agent 的职责既包括将用户自然语言 prompt 转化为可执行检索指令，也包括根据业务策略在实时/离线通道间路由、触发爬取、控制并发与超时、管理任务的生命周期及错误降级，并负责将抓取或检索得到的数据以消息方式下发至数据处理流水线完成清洗与入库。

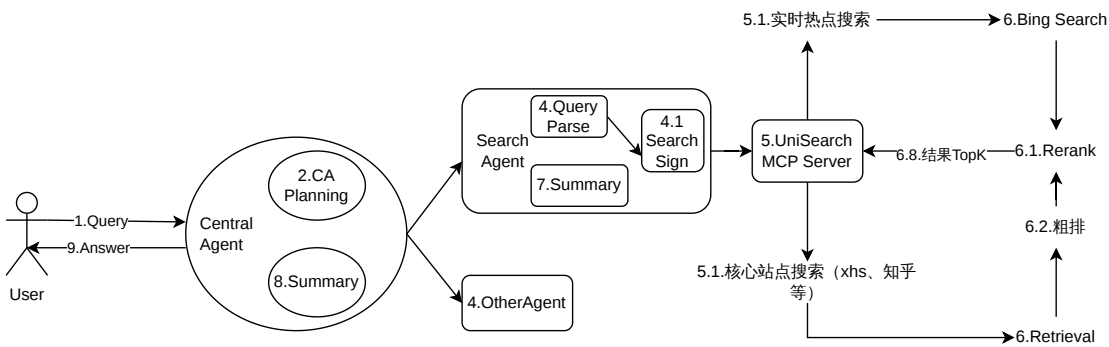


图 3-1: 系统总体概述图

图 3-1是系统的总体概述工作流，图中说明了系统的概述流程。图 3-2是系统的详细介绍图。结合两张图可以梳理清楚整个系统：用户通过上游系统发起带有 prompt 的检索请求，MCP 将请求路由至 Search Agent；Search Agent 先

系统的关键非功能目标包括：端到端延迟在业务可接受范围内（对交互类请求 P95 延迟例如 < 300ms，P99 < 1s，为触发爬取的场景允许异步补全），高吞吐，高可用（SLA 99.9% 以上）、可扩展、可观测、以及合规安全。总体架构应采用分层、模块化设计以便拆分职责、便于单元化部署与灰度发布，同时在关键路径上采用缓存、异步化与降级策略以保证稳定性与响应性。

3.2 系统需求分析

3.2.1 系统用例分析

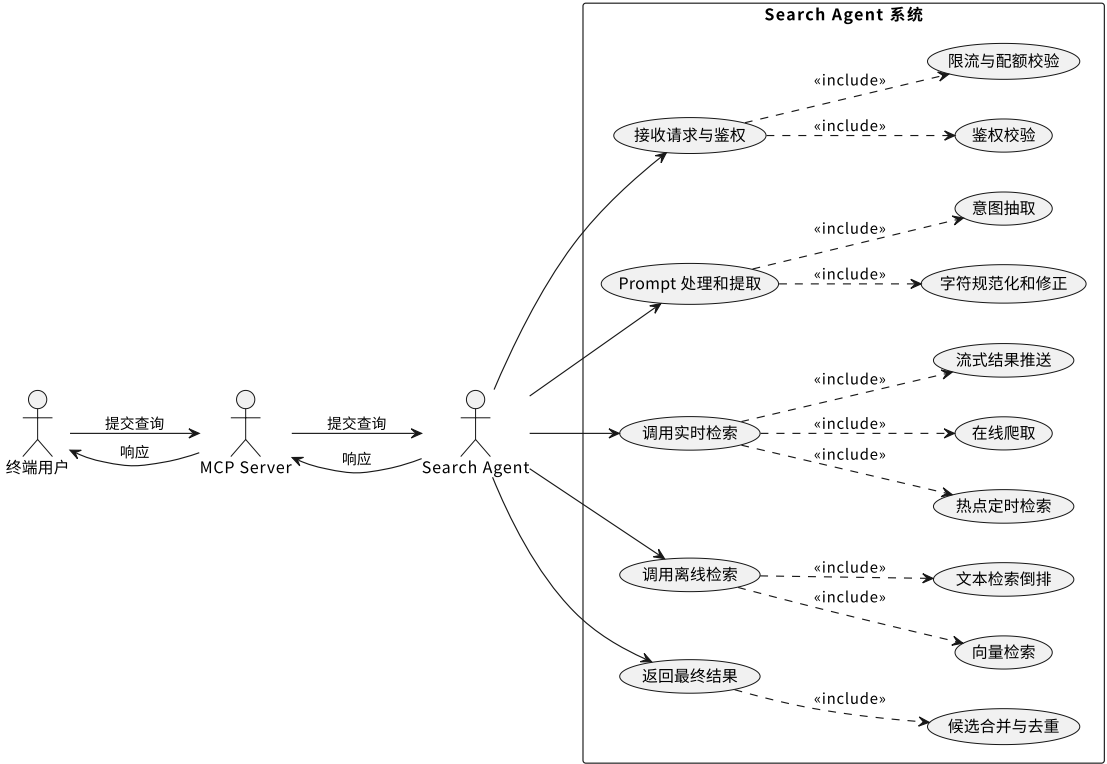


图 3-3: 系统整体用例图

图 3-3是本系统的总体用例图。本系统围绕智能检索与内容获取构建完整链路，其总体需求涵盖查询理解、任务调度、在线获取、离线召回以及数据处理与索引构建五个核心模块。首先，在 Prompt 解析与 Query 生成阶段，需要对用户输入进行语义解析与意图识别，提取关键实体、约束条件与上下文信息，并生成结构化查询表达，以支撑后续多通道检索。MCP 协调与参数传递模块

则负责统一调度各子系统，将解析后的查询映射为标准化参数，并根据策略将请求分发至不同检索通道，支持动态路由、策略控制以及多源结果融合的基础能力，同时保证参数传递的一致性与可追踪性。在实时在线检索与爬取触发方面，系统需具备低延迟，当现有索引无法满足需求时，自动触发爬虫任务进行补充抓取，实现对新鲜内容的覆盖，且需要控制抓取频率与资源消耗，保证系统稳定性。离线垂域检索模块侧重于高质量内容的深度召回，结合文本检索与向量检索提升语义匹配能力，依赖高质量索引与领域数据沉淀，强调召回的准确性与内容价值。最后，索引入库与数据处理模块承担数据清洗、去重、结构化处理与索引构建任务，是支撑整个系统性能与效果的基础，需保证数据处理流程标准化、可扩展，并支持增量更新与索引重建，以持续优化检索效果与系统整体表现。

3.2.2 Prompt 解析与 Query 生成

Prompt 解析模块是 Search Agent 的前置入口，其职责不仅是从自然语言中抽取关键词，更要识别用户意图、拆分多意图句子、生成可供各检索通道消费的结构化检索请求，并对是否触发爬取、检索范围与过滤条件给出明确建议。技术实现需融合规则与模型：规则层用于处理确定性逻辑（敏感词、白名单/黑名单、固定命令），模型层采用轻量化 encoder 或 sequence-to-sequence 模型做语义分词、实体识别与 query 重写。

输出必须包含明确的字段：关键词列表、扩展查询建议、意图标签、触发爬取布尔值与置信度分数，以便 MCP 与后端执行精确路由。响应时延是关键约束，交互路径要求 prompt 解析尽可能在几十到几百毫秒完成，故工程上需部署蒸馏模型、推理缓存与请求批处理策略；对于复杂解析可走异步增强路径，由离线更强模型定期校准解析器并提供回流修正。解析结果需可追溯（记录模型版本、解析耗时、置信度），并支持灰度发布与回滚，避免模型或规则迭代导致大范围质量回退。针对短文本与口语化输入，应加强同义词扩展、拼写容错与实体链接能力，以提升倒排与向量检索的召回覆盖。

3.2.3 MCP 协调与参数传递

MCP 是系统的控制面，负责接收来自上游的请求、下发任务至 Search Agent 或爬虫池，并在异步任务完成后执行回调通知。对 MCP 的需求包括对同

表 3-1: Prompt 解析与 Query 生成用例描述

名称	Prompt 解析与 Query 生成
参与者	用户、前端应用、MCP、Search Agent、Prompt Parser、实时检索适配器、离线检索适配器（倒排/向量）、爬虫调度器、日志/审计服务
触发条件	用户在前端提交自然语言检索请求或查询指令，请求被 MCP 接收并转发到 Search Agent。
前置条件	MCP 与 Search Agent 服务正常；Prompt Parser 的在线推理池可用并通过鉴权；实时与离线检索后端可访问；爬虫池处于允许状态（依据白名单与配额）。请求已附带必要上下文。
后置条件	生成标准化的 QueryPackage 并记录解析元数据；根据策略路由至实时或离线通道（或并行）；当满足展示详情条件时触发爬取任务并写入任务队列；将解析与路由决策写入审计日志；将初步检索候选或占位结果返回给 MCP 供上游或前端展示，必要时通过异步回调补全详情。
正常流程	MCP 接收请求并鉴权，携带 trace、上下文与优先级下发任务 -> Search Agent 对 prompt 做规范化预处理并调用 Prompt Parser 轻量语义解析 -> Prompt Parser 返回关键词、意图、展示指示、重写 query、置信度与模型版本，规则引擎进行合规与安全校验并在必要时覆写或标记人工复核 -> Search Agent 根据解析结果与全局策略（配额、白/黑名单、系统负载）路由请求（离线/实时/并行、向量优先或触发爬取）。

步与异步两类请求模式的统一支持、对任务状态生命周期的管理、以及对优先级与配额的全局调度能力。在同步场景，MCP 要求在预设延迟预算内等待并返回融合后的召回结果；在触发爬取并需异步补全的场景，MCP 需为任务分配唯一 task_id 并提供任务状态查询、超时处理与回调 URL，以便前端或上游系统根据状态决定是否展示占位信息或发起主动刷新。参数传递方面，MCP 必须支持强类型接口，透明传递 trace_id、模型版本、超时阈值、priority 等元信息，同时对任务的重试与幂等要求提供机制。

MCP 还承担资源配额控制，按域名/来源对爬取速率与并发上限进行限制，并在系统压力上升时提供全局熔断与降级策略。安全上，MCP 应实现鉴权与审计，记录每次参数变更与任务触发，确保满足合规审计要求。为提升可用性，MCP 需要水平扩展与 HA 设计，并将任务状态与元数据持久化以便故障恢复与历史回放。

表 3-2是 MCP 协调与参数传递用例描述，MCP 相当于是一个“负载均衡”的组件，为用户和 Agent 之间进行牵线搭桥，减少因为链路复杂带来的额外开销和维护成本。

表 3-2: MCP 协调与参数传递用例描述

名称	MCP 协调与参数传递
参与者	MCP（协调平台）、前端/上游调用方、Search Agent、Prompt Parser、实时检索适配器、离线检索适配器（倒排/向量）、爬虫调度器、配额/白灰黑名单服务、消息队列、审计与监控服务
触发条件	上游系统或用户通过前端提交检索请求到 MCP，或 Search Agent/其他组件发起需要 MCP 协调的异步任务。
前置条件	MCP 与各下游组件鉴权与网络连通正常；配置中心中存在任务协议（schema）、默认超时和重试策略、配额配置与回调约定；请求包含必要元数据。
后置条件	请求被合法路由并携带完整参数透传到目标组件；任务状态在 MCP 中登记并可查询；异步任务注册了回调或事件订阅；所有关键参数变更与路由决策记录审计日志与监控埋点以支持回放与问责。
正常流程	MCP 接收请求并鉴权校验、限流，生成 task_id/trace_id 及超时与优先级 -> 将请求及元数据透传给 Search Agent 并记录路由，同步走等待路径，异步登记任务 -> 对需多下游调用的场景按预定义序列或并行策略拆分子任务并下发，调用均携带 trace 与鉴权与超时上下文 -> 与配额/白灰黑名单服务校验敏感操作权限，配额不足或合规禁止时拒绝或降级并记录计量 -> 同步路径聚合下游返回或超时后降级响应上游，异步路径维护任务状态机并在完成 -> 发生异常时执行重试/退避/降级/回滚策略，记录故障与决策写入审计日志并触发告警 -> 全流程埋点关键指标并汇总日志/trace 到监控与回放系统以支持线上定位與策略优化。

3.2.4 实时在线检索（bin search）与爬取触发

实时在线检索通道以低延迟与高新鲜度为核心目标，承担热点定时检索与对即时请求的快速响应。该通道需支持定期的热点扫描，将热点词推入检索队列并维持短时 cache 以应对突发流量。同时，Search Agent 必须在检索后根据 prompt 的解析结果与检索 snippet 的质量判断是否触发 Python 爬取以获得页面详情或补充信息。

触发条件既要保证用户体验（提高展示丰富度），又要控制系统与目标站点的压力，故需要策略性阈值、白名单/黑名单与实时配额。触发后的爬取由爬虫池异步执行，Search Agent 在主请求路径上应采取占位返还与异步补全策略：先返回检索候选或简短摘要以满足首屏响应需求，再在爬取完成后通过 MCP 推送更新以补全详情。

表 3-3: 实时在线检索（bin search）与爬取触发模块用例描述

名称	实时候选召回与爬取触发（bin search 路径）
参与者	Search Agent、MCP、Prompt Parser、实时检索适配器（bin search client）、爬虫调度器、爬虫池、配额与白灰黑名单服务、监控与审计服务
触发条件	Search Agent 接收到来自 MCP 的已解析 QueryPackage，并判断需在实时通道进行低延迟候选召回。
前置条件	实时检索适配器与后端 bin search 服务在 SLA 内可用；爬虫调度器及爬虫池处于允许接收任务的状态；配置中心含最新的域名白/黑名单与域级配额；Search Agent 与各适配器间鉴权与网络连通性正常；请求携带 trace_id 和剩余超时预算。
后置条件	实时检索适配器返回候选集合及 snippet；在满足触发条件且合规/配额允许下，爬取任务成功下发至爬虫调度器并入队；Search Agent 将初步候选用于首屏展示并订阅爬取回调以异步补全；所有路由与触发决策记录入审计日志并埋点上报监控。若爬取未被允许或失败，则按降级策略返回可替代展示。
正常流程	MCP 下发带 QueryPackage 到 Search Agent，Agent 检查剩余超时与优先级 -> 在短超时窗并行调用 bin search 适配器获取候选 -> 对候选做快速质量评估：snippet 密度、来源可信度（白/灰/黑名单、历史分）、与“需展示详情”匹配度，并读取域抓取可用性与域级配额 -> 若需抓取且域名允许，向配额服务申请并在获批后构造幂等爬取任务入队并记录审计-> 爬虫遵守严格超时/重试与退避策略，解析成功将结构化结果及 parse_confidence、extraction_errors 写入数据处理队列，失败记录原因并告警 -> 全流程埋点爬取率、配额使用、成功率、耗时与降级等指标，达到阈值时触发自动限流或人工干预。

实时检索通道对延迟与吞吐的双重约束要求在工程实现上做好缓存优先、并行调用倒排与向量后端、以及在高并发时进行部分降级（如返回缓存或减少重排深度）。此外，触发爬取前务必执行合规检查（robots、版权）与来源可

信度评估，避免对受限制资源进行自动化抓取。

实时爬取模块需要在严格 SLA 下快速抓取目标页面并抽取结构化信息，以支撑展示或后续入库。该模块应在下载、渲染、解析三个阶段设定明确的超时阈值，并在发生超时时采取退避或降级策略。初步抓取优先采用轻量 HTTP 请求获取静态内容；若页面依赖大量 JS 渲染且需要完整 DOM 才能提取目标信息，则在探测到渲染必要性时切换到 headless 浏览器执行渲染，但应限制渲染并发与时间以控制成本。解析流程采用规则与模型融合的方法，先用基于规则的 readability 类库快速抽取正文，再以轻量 NER/IE 模型或正则规则进行字段抽取与标准化。解析结果需附带质量元数据（parse_confidence、extraction_errors、source_score），以便后续清洗模块决定是否进行二次清洗或人工审核。为保证幂等性与避免重复，爬取任务在下发前应做 URL 规范化与指纹检查，任务执行应支持断点续传与重试上限。系统监控需覆盖成功率、超时率、平均/分位延迟、解析错误分布与队列堆积情况，并在阈值触达时触发自动降级（如暂停新的爬取任务或延长超时）以保护整体稳定性。安全与合规性包括对目标站点的请求频率控制、敏感信息过滤与爬取日志审计。

3.2.5 离线垂域检索（文本检索/向量检索）

离线检索通道面向小红书、知乎等社区垂域数据，强调语义丰富性与内容深度。离线数据通过定期批量采集或授权数据源同步进入 ETL 管道，经过清洗、去重、标签化与向量化后，分别构建倒排索引与向量索引以支持文本检索与语义检索。由于离线通道对实时性要求相对宽松，重点在于保证索引质量与语义一致性：embedding 模型版本的管理、预处理规则的统一、以及向量索引构建参数都需严格控制并记录以支持索引回滚或重建。离线数据的增量更新策略应兼顾新鲜度与构建成本，常见做法是在索引侧采用 delta 索引或周期性 bulk 重建并通过 alias 切换实现无缝上线。离线通道还需提供用于训练与评估的标注数据集，支持重排模型与召回融合的离线评估，进而为线上 Search Agent 的决策提供基准。质量控制包括对来源可信度、互动指标（点赞、评论）和文本完整性的评估，并将质量分作为索引字段供检索与重排模型使用。

此外，离线检索通道还需要构建完善的数据血缘与版本管理体系，对每一批数据的来源、处理流程以及索引构建过程进行全链路记录，以便在出现效果波动时能够快速定位问题并进行回溯分析。在特征工程层面，可以引入多粒度语义表示，如段落级与文档级向量，以提升长文本检索效果。同时，通过构建

多路召回机制，将关键词匹配结果与语义召回结果进行融合，进一步提升召回覆盖率与相关性。在评估环节，应结合点击率、停留时长等行为数据构建离线指标体系，并与人工标注结果进行对齐，以形成稳定的评估基准。针对不同垂域场景，还可以设计差异化标签体系与领域词表，从而增强模型对专业语境的理解能力，最终提升整体检索质量与用户体验

表 3-4: 离线垂域检索（文本检索/向量检索）模块用例描述

名称	离线垂域检索（文本检索与向量检索）用例
参与者	Search Agent、MCP（请求协调）、Prompt Parser、离线倒排索引服务、向量检索服务、索引管理/元数据服务（Index Manager）、特征/模型服务、监控与审计服务
触发条件	Prompt Parser 或 Search Agent 判断该请求应使用离线通道（例如需高语义覆盖、垂域社区来源 XHS/知乎优先或用户指定离线来源），由 MCP 将请求下发至 Search Agent 并路由到离线检索模块。
前置条件	离线倒排索引与向量索引已按计划构建并对外提供查询接口；索引版本（index_version）和 embedding_model_version 可查询并与请求兼容；检索服务健康且鉴权通过；请求包含 trace_id、期望候选数与超时预算。
后置条件	返回整合后的候选集合（包含来源、score、index_version 与质量元数据）；将低质量或无结果的请求及其原始上下文记录到回放/告警系统以便后续补采或索引改进；检索全程元数据写入审计日志以便离线分析。
正常流程	Search Agent 接收 QueryPackage 并选离线通道，计算剩余超时与优先级 -> 并行发起关键词文本检索与向量检索，指定 index/embedding 版本与检索参数 -> 倒排与向量服务在超时内返回候选，超时则回退到另一路或使用缓存 -> 对候选按 doc_id/内容指纹去重并归一化/融合得分形成候选池 -> 延迟允许时运行轻量 LTR 并对 top-N 启用精排，否则返回 LTR 结果 -> 以统一格式回传 MCP，记录检索指标并提交回放样本以支持离线分析与索引优化。

3.2.6 索引入库与数据处理模块

数据处理模块负责将来自实时检索、爬取与离线导入的原始内容转化为可搜索、合规与高质量的索引文档。实时管道要求低延迟处理：对爬取产物先做轻量化解析与去噪（HTML 去壳、编码修复、简单实体抽取），并快速计算质量分与指纹后入队进行二次清洗。

表 3-5: 索引入库与数据处理模块用例描述

名称	索引入库与数据处理
参与者	爬虫/检索后端（产出原始内容）、消息队列、数据处理模块、索引构建服务（倒排索引构建器、向量化与向量索引器）、索引管理/元数据服务、Search Agent、监控与审计服务
触发条件	爬虫或检索后端将原始抓取内容或离线导入批次写入消息队列或对象存储，触发数据处理流水线进行实时解析/批量清洗并最终入库构建倒排或向量索引。
前置条件	数据处理服务与索引服务可用并通过鉴权；消息队列或对象存储中存在待处理的原始数据或批次；索引构建参数在索引管理服务中可查；系统具有足够的写入配额与资源配额以执行入库任务。
后置条件	清洗后的结构化文档通过幂等写入入倒排索引或向量索引；索引元数据更新并持久化；生成入库日志与审计记录以便回放与溯源；对低质量或异常文档标记并触发人工复核或重处理任务。
正常流程	数据来源写入消息队列/对象存储并登记元数据-> 实时/批量消费者拉取并快速解析，生成结构化记录 -> 结构化记录二次清洗，质量评估产出，低质或疑似敏感标记人工复核或降权 -> 通过阈值的记录按路径入库：文本写入倒排索引，需向量化的送 Embedding 并写入向量索引，写入需幂等 -> 索引构建完成后更新索引元数据并通过 alias 原子切换使新数据可见，记录耗时与成功率用于监控与告警 -> 入库失败或质量不达标写入回放并触发重处理，全部处理版本与关键元数据记录审计以支持可追溯与合规。

二次清洗在离线或异步流程中进行更复杂的处理，包括语义一致性校验、重复检测、实体规范化、时间与作者字段标准化、以及使用模型判别的内容可信度评估。质量评估体系需结合规则和机器学习：规则覆盖格式校验与明确违禁项，模型用于语义相似性、内容完整性和敏感信息检测。处理流程应记录完整的数据血缘与处理版本，以支持审计与问题回溯。清洗后的数据需按照不同索引需求生成不同格式的入库包：倒排索引侧注重字段标准化与分词质量，向量索引侧注重 embedding 的稳定性与元数据一致性。对低质量或可疑条目，系统应支持自动降权、暂缓入库或人工复核机制，并将复核结果用于改进自动化质量判定模型。

索引入库阶段要求高可用、可扩展且具备可观测的写入体系。对于倒排索

引，需要设计批量导入与近实时刷新的混合策略，保证离线数据批量导入的压缩与查询性能，同时允许对重要实时数据采取 `delta flush` 以满足短期可搜索性。对于向量索引，需考虑向量化模型版本、构建参数与向量插入的代价：大规模重建常采用离线批处理，实时新增向量可采用内存热表或小型 HNSW 以实现短期可检索，随后做定期合并。入库过程应由消息队列驱动并支持幂等写入（使用唯一 ID 去重），并在发生写入失败或索引切换时提供原子化切换与回滚流程。写入还需记录索引元数据，以便后续查询兼容性与回溯。为保证服务连续性，索引写入要具备速率控制和后端退压策略，并监控写入延迟、错误率与索引大小增长，结合 MCM 做容量与维护窗口规划。

3.2.7 非功能性需求分析

体系的非功能性需求涵盖性能、可用性、可扩展性、安全性与可运维性。在性能方面，需对不同场景设定明确的服务水平协议（SLA）。对于交互类请求，应设定 P95 延迟目标，例如小于 300ms，并且确保 P99 延迟小于 1 秒。对于触发爬取的场景，允许异步补全处理，但需要保证补全的时延在可接受的范围内，通常为数秒级。吞吐量方面，要求系统具备每秒并发检索的能力以及最大并发爬取的数目，并确保在高负载下依然能够保持稳定的响应能力和数据处理速度。此外，针对大规模数据处理，系统应具备动态扩展能力，以应对未来可能的流量增长。

在可用性方面，要求系统具备冗余机制、自动重试机制和快速回滚能力，确保系统在任何故障情况下能迅速恢复。整体的系统可用性应保持在 99.9% 以上，以保证用户的高可用需求。此外，系统还应具有全面的故障转移和灾难恢复机制，确保在发生严重故障时能够快速切换到备用系统，避免对业务造成较大影响。

可扩展性方面，系统需要支持组件的水平扩展能力，确保各个模块（如检索后端、爬虫池、模型推理服务和处理消费者）都可以根据负载自动进行扩展。同时，要求系统具备高效的分片副本策略，保证数据的分布和副本能够支持大规模并发的查询和数据处理需求。在一致性方面，要求系统的索引入库必须提供最终一致性保证，并且要尽可能减少索引数据的延迟，以保证用户的搜索体验不受影响。

安全性方面，系统必须具备完善的鉴权与授权机制，确保用户的身份和访问权限得到严格管理。此外，必须实施全面的审计日志功能，记录系统的所有

关键操作，以便追溯。对于敏感信息，系统需进行脱敏处理，同时满足相关的合规性要求，包括版权和隐私的保护。同时，系统需要对爬取行为进行实时合法性校验，确保数据来源的合法性和合规性。

可运维性要求方面，系统应具备完善的监控和告警机制，能够实时监控系统的各项指标，并在异常情况发生时自动触发告警。同时，系统应支持自动化运维工具，如滚动升级、灰度发布和容量预警，以确保系统在高负载和复杂环境下依然能够稳定运行。为了保障系统的高可靠性，必须建立全面的测试策略，包括单元测试、集成测试、压力测试、回放测试和混沌演练等。通过这些测试手段，能够有效评估系统在不同故障场景下的恢复能力和韧性，从而提高系统的整体稳定性。

3.3 系统总体设计

3.3.1 系统逻辑视图

系统逻辑视图从功能与角色角度描述各子系统之间的职责分工与交互关系，便于理解 Search Agent 在端到端流水线中的定位与控制边界。总体上，系统由五类核心逻辑组件组成：请求协调层、决策与路由层、模型推理与解析层、检索与爬取后端、以及数据处理与索引层。每类组件在逻辑视图中承担明确职责：MCP 负责接收上游请求并维持任务生命周期、配额与回调；Search Agent 负责将 prompt 转为可执行检索指令、在实时/离线通道间做路由决策、触发爬取并维护任务并发与超时策略；Prompt Parser 为 Search Agent 提供语义分词、意图识别、关键词提炼与置信度评估；实时检索（bin search）用于低延迟候选召回并可在条件满足时触发爬虫获取详情；爬虫池执行实时抓取并将原始结果发往数据处理模块；离线索引服务（面向 XHS、知乎等源）提供基于倒排与向量索引的深度召回；数据处理层执行实时解析、去噪、质量评估并将合格数据落入适配的索引库。

图3-4展示了一个典型的搜索代理系统的自上而下的分层逻辑视图。在该系统中，服务层、后端与中间件层以及存储层各自承担了不同的功能和任务，共同支撑整个搜索流程的高效运转。

首先，服务层负责接收来自客户端的请求，进行预处理和路由决策。请求通过前端进行验证和追踪，随后由 Prompt Parser 进行规范化并提取关键词和意图信息。路由器根据策略将请求分发至合适的检索后端，分为实时搜索和离线

搜索。如果需要，系统还会请求 Embedding Service 进行文本到向量的转换。爬取管理器负责决策是否触发新的爬取任务，并通过爬取调度器将任务队列传递至爬虫执行器，最终将抓取结果写入消息队列并触发异步补全。

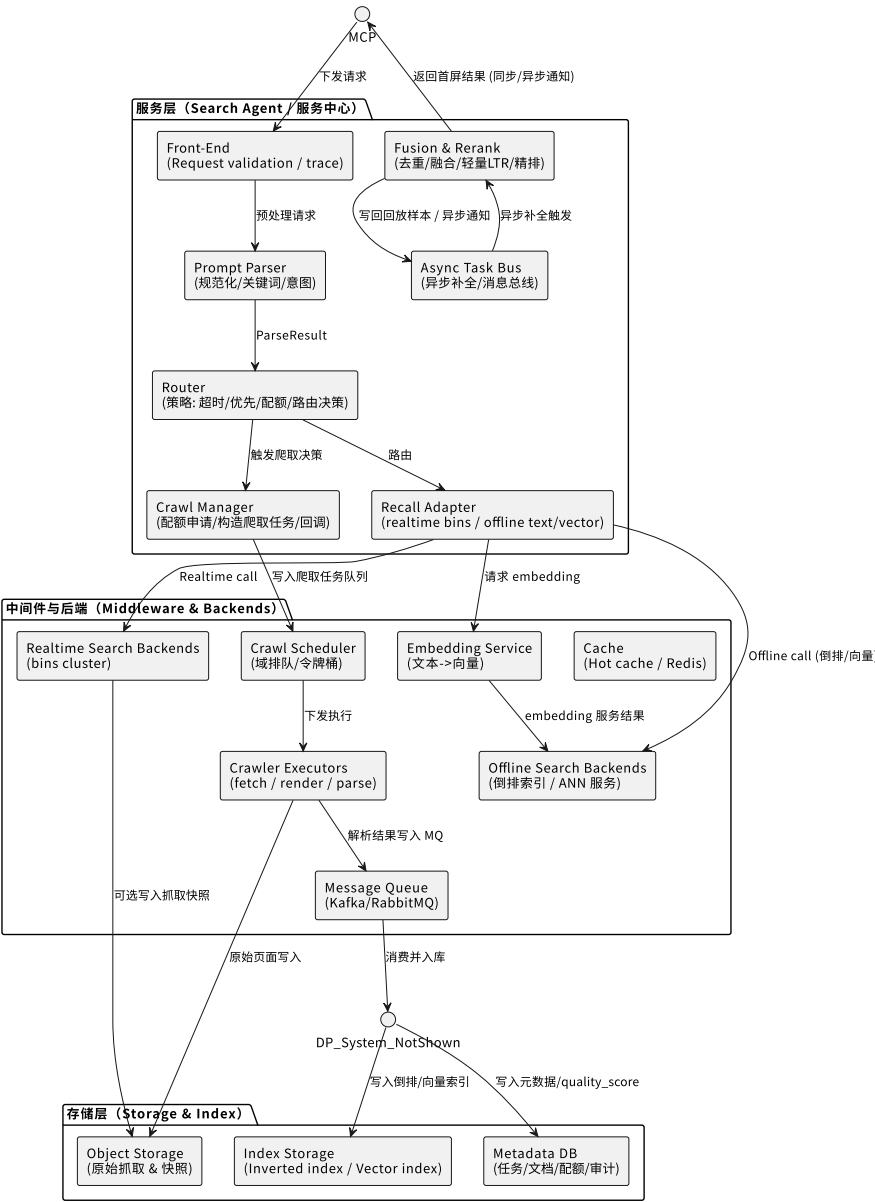


图 3-4: 系统逻辑视图

后端与中间件层主要负责搜索请求的具体处理，包括实时和离线的索引查询、嵌入服务的向量生成以及缓存管理等。该层中的消息队列提供了松耦合的数据传递通道，支持系统的高效异步操作。存储层包括对象存储、索引存储和元数据数据库，用于存储原始抓取内容、倒排索引、向量索引及相关的元数

据。通过这些组件的协作，系统能够高效地处理用户请求，进行数据抓取与处理，并提供精确的搜索结果。在逻辑交互上，整个请求生命周期从 MCP 收到带有 metadata 的 prompt 开始。MCP 将请求分发给 Search Agent，Search Agent 首先调用 Prompt Parser 进行语义解析，得到结构化输出。基于解析结果与全局策略，Search Agent 决定是先调用离线通道、实时通道、还是同时并行调用两者。离线通道通常直接调用索引服务的检验 API，实时通道则先向 bin search 发起请求以快速获取候选，若 snippet/候选不足且解析标识需展示详情则触发爬虫池。本模块设计上强调“先快后补”的理念：在触发爬取的场景下，Search Agent 会先返回可用的候选或占位摘要以保证交互响应，同时在后台等待爬取完成并在完成后通过 MCP 的回调或事件通知上游更新展示。

召回融合逻辑在逻辑视图中作为独立职责被强调：来自倒排索引与向量索引以及实时检索/爬取的候选需在融合层做去重、score normalization 与 TopK 合并，然后送入在线重排层。重排层采用分层重排策略：首先用轻量 feature-based 排序器做快速排序，再对 top-N 使用更强的 cross-encoder 或摘要生成模型对展示内容做最终加工。数据处理模块在此逻辑链路中既承担向索引的落库职责，也负责对爬取内容做实时清洗与质量判别，将质量元数据反馈给召回与重排模块以影响候选权重。

为了支持系统演进与运维，逻辑视图中还包括配置与控制子系统、以及策略管理子系统。这些横向能力通过统一的控制面下发策略并确保 Search Agent 在运行时具有可控的行为边界。图示描述建议绘制一张逻辑架构图：中心为 Search Agent，左侧为 MCP 与上游请求，右侧并列展示实时检索与离线索引，下方为爬虫池与数据处理，顶部为模型推理服务与配置；箭头标注主要数据流（prompt→parse→recall→crawl→process→index→response）与控制流。

3.3.2 系统过程视图

系统过程视图从时序与流程角度刻画关键业务场景的执行步骤，便于分析延迟、容错与故障恢复逻辑。以三类典型流程为主：同步检索请求流程、触发爬取的交互流程、与离线数据入库。

对于同步检索流程，当用户发起带 prompt 的请求时，MCP 作为入口进行身份与配额校验并将请求派发到 Search Agent；Search Agent 立即调用 Prompt Parser 进行语义解析，并行向模型服务发起轻量推理请求，解析返回关键词/意图并带有置信度；Search Agent 并行调用实时检索和/或离线索引服务，在收到

候选后执行快速的 TopK 合并，并将候选送入轻量重排器计算最终排序；若在预设延迟窗口内无法获得充分候选，则 Search Agent 可返回占位与部分结果，并在后台继续等待其他通道的补充。整个同步流程需在严格 SLA 内完成，过程中 Search Agent 维护 trace id 并将关键耗时点埋点以便 Prometheus/Grafana 展示。

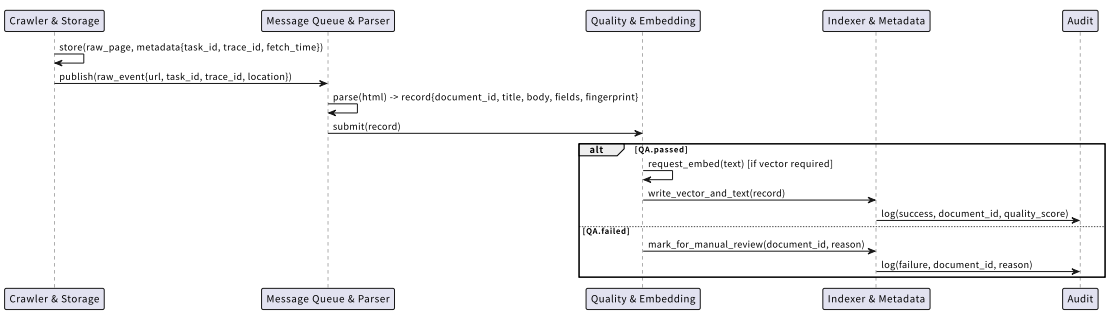


图 3-5: 数据爬取清洗解析入库时序图

触发爬取的交互流程比同步流程多出实时爬取与异步回调环节。流程开始如上，但当 Prompt Parser 或候选质量判断出需要页面级详情时，Search Agent 构造爬取任务并将任务通过消息队列或 MCP 下发到爬虫池，同时仍以最快速度返回检索候选或摘要以保证首屏体验。爬虫池接到任务后执行下载，在下载与解析阶段执行严格的超时控制；解析完成后将结构化结果写入消息队列或直接调用数据处理服务的入队接口。数据处理模块对爬取产物做实时解析、清洗、质量评估并在合格后将条目落库到对应索引；当入库或解析完成时，系统通过 MCP 的回调机制将更新推送给上游或前端，从而实现异步补全与用户界面的数据刷新。整个链路要求幂等、去重与失败补救。

离线数据入库与索引重建流程侧重于批量处理与一致性保障。数据采集端将原始数据批次写入对象存储并触发 ETL 作业；数据处理层使用批处理框架进行大规模清洗、向量化，并生成索引构建任务。索引构建采用隔离式构建，在临时索引环境构建完毕后使用别名/atomic switch 原子切换到新索引，减少对线上查询的影响。对于需要较短可搜索延迟的新增数据，系统会采用 delta 索引策略，先将新数据写入小规模近实时索引，并在合适时机合并回主索引

3.3.3 系统开发视图

开发视图描述了搜索系统内部的核心组件及其交互关系，展示了系统的逻辑层次和模块划分。应用层中的各个服务组件通过容器化部署在 Kubernetes 集

群中，负责处理来自客户端的请求，包括请求验证、语义解析、路由决策、检索调用等。服务之间通过 gRPC、HTTP 和消息队列进行通信，确保数据流转和任务调度。检索服务通过实时和离线索引服务提供搜索功能，而爬虫调度和执行模块则负责抓取和解析新的网页数据。数据处理和索引模块对爬取的数据进行清洗、质量评估和索引写入，确保数据的完整性和可搜索性。该架构具备高扩展性和灵活性，可以快速适应变化的需求。

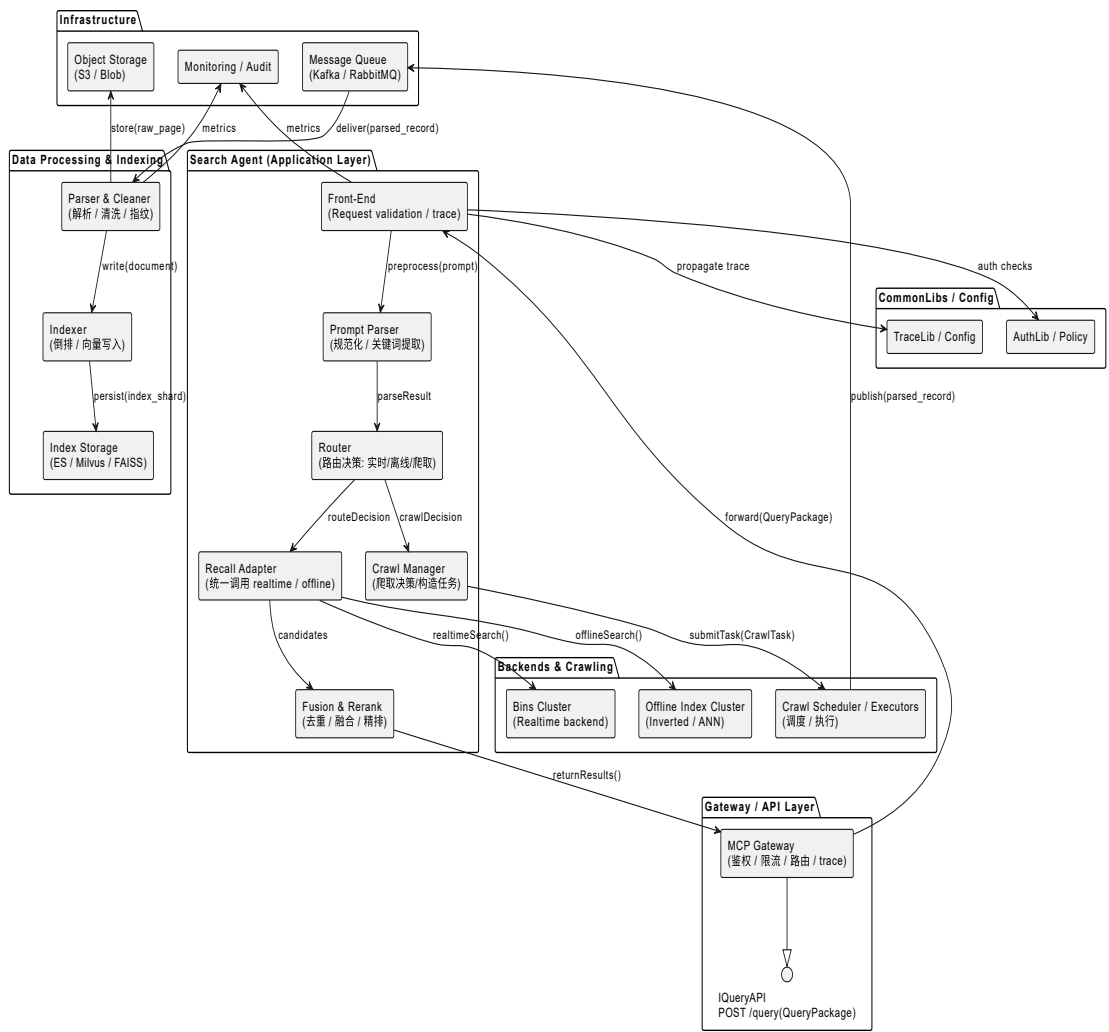


图 3-6: 系统开发视图

图 3-6展示了搜索系统的整体概述架构，通过不同的组件层次展示了系统的整体结构与模块间的交互流程。整个系统架构分为多个层次，包括网关/API层、应用层（Search Agent）、后端与爬取层、数据处理与索引层、基础设施层以及跨切面库层。系统首先由 MCP Gateway 负责接收请求并进行身份验证、配额校验以及路由决策。请求会通过 IQueryAPI 接口转发到应用层的相关模块进

行处理。

在应用层，Front-End 模块会验证请求并进行初步的追踪，随后由 Prompt Parser 模块对用户输入的 prompt 进行规范化处理，并提取关键词和意图信息。此时，Router 模块根据请求类型做出路由决策，将请求发送至不同的后端服务进行查询。如果请求需要实时数据，Recall Adapter 模块会调用实时搜索引擎（如 Bins Cluster）；如果请求涉及离线数据，模块则会向离线索引系统（Offline Index Cluster）发起查询。若在申请过程中，需要新的数据源或页面，Crawl Manager 模块会判断并构造爬取任务，通过消息队列提交到爬虫池，由爬虫执行器（Crawler）进行数据抓取与解析，随后通过消息队列将结果发送到数据处理模块。

数据处理与索引层负责对爬取或查询的结果进行解析、清洗和处理。Parser & Cleaner 模块会对抓取的数据进行解析并清洗，同时生成文档指纹。接着，Indexer 模块将处理后的数据写入倒排索引和向量索引，并将索引数据持久化到 Index Storage 中，确保系统能够高效检索和查询。这一过程中的数据也会被存储到对象存储中，确保数据的可靠性和持久性。

在基础设施层，Message Queue 提供了系统间异步通信的能力，确保模块之间能够高效且松耦合地进行数据交换。Object Storage 则用于存储原始的网页数据和其他静态资源。系统还通过 Monitoring 模块实时监控各项指标，记录日志和操作，帮助进行故障排查与性能优化。

最后，跨切面库层包括一些提供全局功能的库，如 AuthLib 负责处理系统的权限和安全策略，而 TraceLib 则用于请求的追踪和系统配置管理，确保系统可以进行全面的性能分析和调优。系统中的每个模块和组件都与其他模块通过定义良好的接口和消息队列进行交互，从而实现高效的数据处理和查询能力，确保系统在满足性能和可靠性要求的同时，还能够灵活地进行扩展和维护。

3.3.4 系统物理视图

物理视图展示了搜索系统的部署架构，涵盖从客户端到基础设施层的完整部署流程。系统的请求首先通过客户端发起，经由 API 网关进行身份验证、限流、路由决策和追踪处理后转发到 Kubernetes 集群中的应用层。应用层的多个微服务组件负责请求验证、Prompt 解析、路由决策、召回适配和爬取任务管理等功能。后端的检索集群、爬虫执行集群和数据存储系统则确保系统能够高效地处理查询请求和抓取新数据。在基础设施服务层，消息队列、对象存储、索

引存储和元数据数据库保证了系统的数据流转和持久化。监控和运维系统提供了实时的系统监控、日志管理和故障追踪，确保系统的高可用性和稳定性。

系统物理视图描述实际部署拓扑、资源分配、网络与安全边界及高可用配置。推荐将系统部署在 Kubernetes 集群上以获得弹性扩缩容与运维统一化管理。关键运行单元包括：Search Agent 服务（stateless，可水平扩展）、Prompt Parser、爬虫池、消息队列集群、数据处理，以及索引服务。

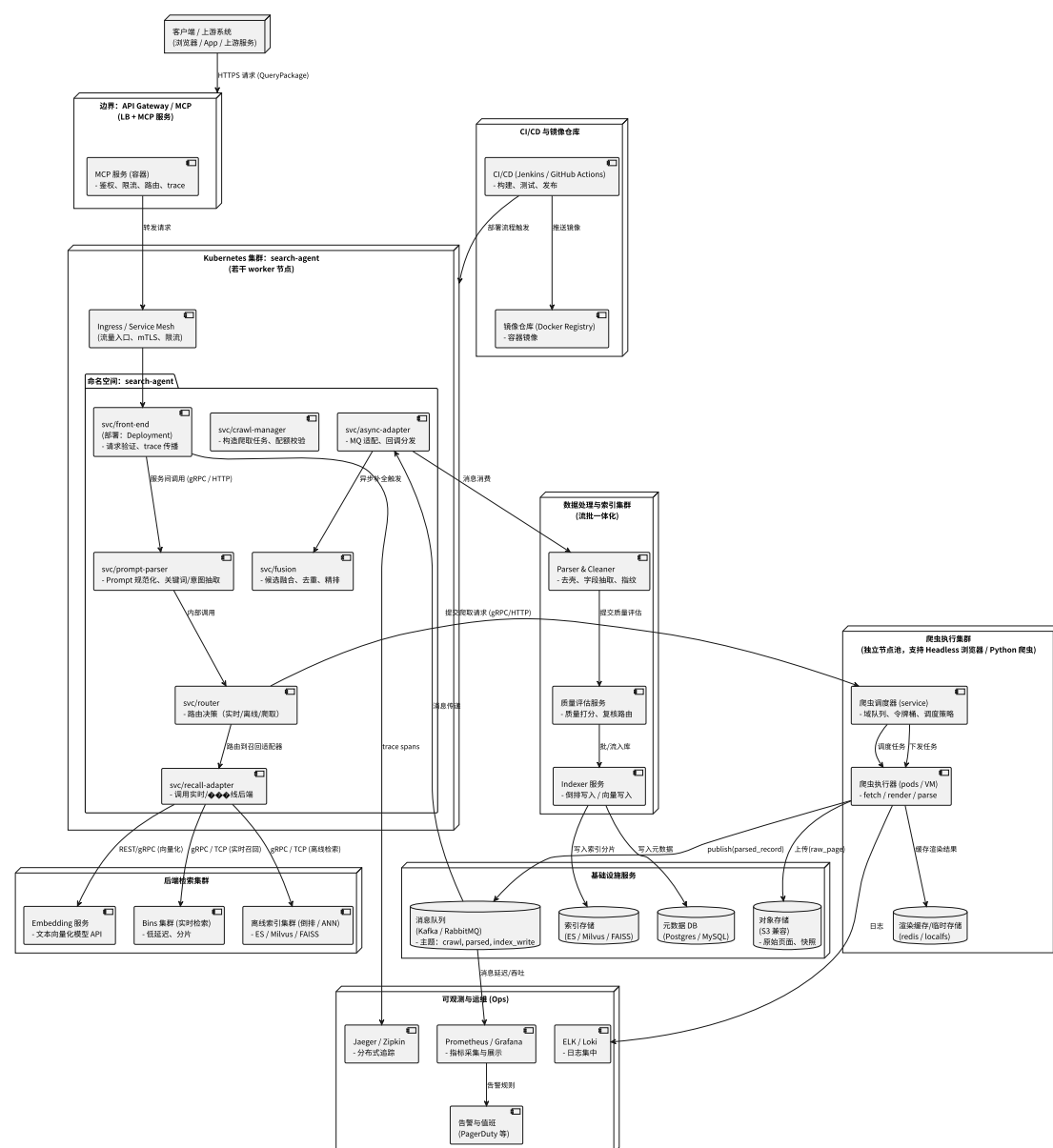


图 3-7: 系统物理视图

图 3-7为系统的物理部署图，该图展示了一个复杂的搜索系统的物理架

构，包括从客户端到基础设施服务的完整部署流程。整个系统的请求首先由客户端发起，经过 API 网关进行鉴权、限流、路由以及追踪等处理。网关将请求转发至 Kubernetes 集群中的核心微服务层，其中包括前端请求验证、Prompt 解析、路由决策、召回适配、爬取任务管理、候选融合等功能模块。

在 Kubernetes 集群中，微服务通过容器化的方式部署，每个服务模块承担不同的功能。前端服务负责请求验证并传播追踪信息，Prompt 解析服务将用户的查询转化为规范化的关键词和意图，路由决策服务则基于实时或离线检索需求决定请求的处理方式。召回适配器通过并行调用实时检索集群和离线索引服务，从后端检索系统获取候选结果。这些候选结果会被融合、去重并精排，最终返回给 API 网关。

当请求需要爬取新数据时，路由服务会触发爬取任务的创建，爬取任务会提交给专门的爬虫调度器，爬虫池中的执行器根据任务要求抓取网页，解析内容并上传到对象存储中，同时将处理后的数据通过消息队列传递至数据处理模块。数据处理模块对爬取的数据进行解析、清洗、质量评估后将合格的结果写入倒排索引或向量索引中，最终存储在索引存储系统中。

基础设施服务层包括消息队列、对象存储、索引存储和元数据数据库，确保各个服务之间的高效通信、数据持久化与备份。消息队列用于异步传输任务和数据，支持系统的高并发与解耦。对象存储用于保存原始页面与快照，而索引存储则用于持久化搜索索引，元数据数据库记录任务和数据的相关信息。此外，系统还提供了强大的可观测与运维支持，涵盖了 Prometheus 和 Grafana 用于监控、ELK 和 Loki 用于日志集中管理、Jaeger 和 Zipkin 用于分布式追踪。所有的操作都被监控并记录，及时触发警报以保障系统的稳定运行。CI/CD 流程和镜像仓库则确保了系统的快速迭代与部署，通过自动化构建和测试流程，确保系统组件的持续更新和高效交付。整个系统通过模块化的部署和容器化管理，实现了高可扩展性、灵活性与高可用性。

3.4 系统模块设计

3.4.1 系统层次与模块划分

系统采用分层模块化设计，以保证职能清晰、可扩展与易运维。最外层为接入与协调层，由 MCP 承担请求接入、鉴权、全局流量与配额控制、任务状态管理与回调；其下为控制与决策层的 Search Agent，负责 prompt 的接收、调

用解析模块、根据策略路径到实时或离线通道、触发爬取并维护任务并发与超时；解析与模型层提供语义分词、意图识别、query 重写与置信度评估，是 Search Agent 的语义中枢；召回层包含实时在线检索与离线检索，并以统一接口向上暴露候选集合；爬取层由独立的爬虫池和渲染池组成，负责下载、渲染与初步解析；数据处理层承接检索、二次清洗、质量评估与入库；索引层分为倒排索引服务与向量索引服务，分别承担文本与语义检索能力；最后是重排与展示层，负责多通道候选融合、TopK 合并、低延迟重排及摘要生成以供前端呈现。各层通过明确的 API 或消息 schema 解耦，关键通信使用 gRPC 或 Protobuf 以保证高效与向后兼容，同时消息驱动的队列用于解耦异步任务。

模块划分以职责单一为原则，每个模块内部再按功能细化子模块。例如 Search Agent 内部分为路由引擎、策略引擎、任务管理器与监控埋点组件；Prompt Parser 划分为轻量推理路径与精校验路径；数据处理层分为流式解析通道与批处理通道，分别承担实时清洗与离线深度处理。模块之间约定统一的元数据模型包含 trace_id、task_id、source、model_version、priority、timeout 等，以实现端到端可追溯。为支持灰度与多版本并行，系统在模块层面设计了版本控制与特性开关，并在 MCM 平台中集成发布审批、灰度流量控制与回滚机制。监控与告警覆盖所有关键模块的 SLI，包括延迟、错误率、吞吐、队列堆积、爬取成功率、解析质量分布等，并设计了自动化的降级与回退策略以降低事故影响。

Prompt 解析模块是将自然语言 prompt 转化为检索指令与任务元信息的核心。该模块设计为“多层混合架构”，上层为轻量化在线解析器，保证低延迟响应，面向交互路径提供快速的关键词抽取、意图判别与可控的 queryrewrite；下层为高质量离线解析器，用于周期性或异步地对线上解析结果进行回流校正与策略更新。在线解析器由两部分构成：规则引擎与蒸馏模型。规则引擎处理确定性逻辑，如指令式 prompt、安全或合规预检、白名单或黑名单命中，并对模型输出做约束。蒸馏模型为经过蒸馏或量化的 encoder 或 seq2seq 模型，侧重在短文本语义抽取，并通过缓存常见 prompt 解析结果与请求批处理来降低延迟与成本。

3.4.2 Prompt 解析模块设计

解析模块在整个 Search Agent 系统中扮演着至关重要的角色，负责将用户输入的查询转化为机器可以理解和处理的标准化结构，并输出 QueryPackage 供

后续的搜索引擎和数据处理模块使用。标准化的 `QueryPackage` 包括关键词列表、扩展查询、意图标签、触发爬取标志、置信度 `score` 以及上下文元数据，这些输出使得后续的计算、爬取、数据处理等环节能够高效、准确地完成。为了确保解析模块的高效性和准确性，它不仅要具备高可解释性，还要能够灵活适应不同业务需求，提供高质量的搜索服务。

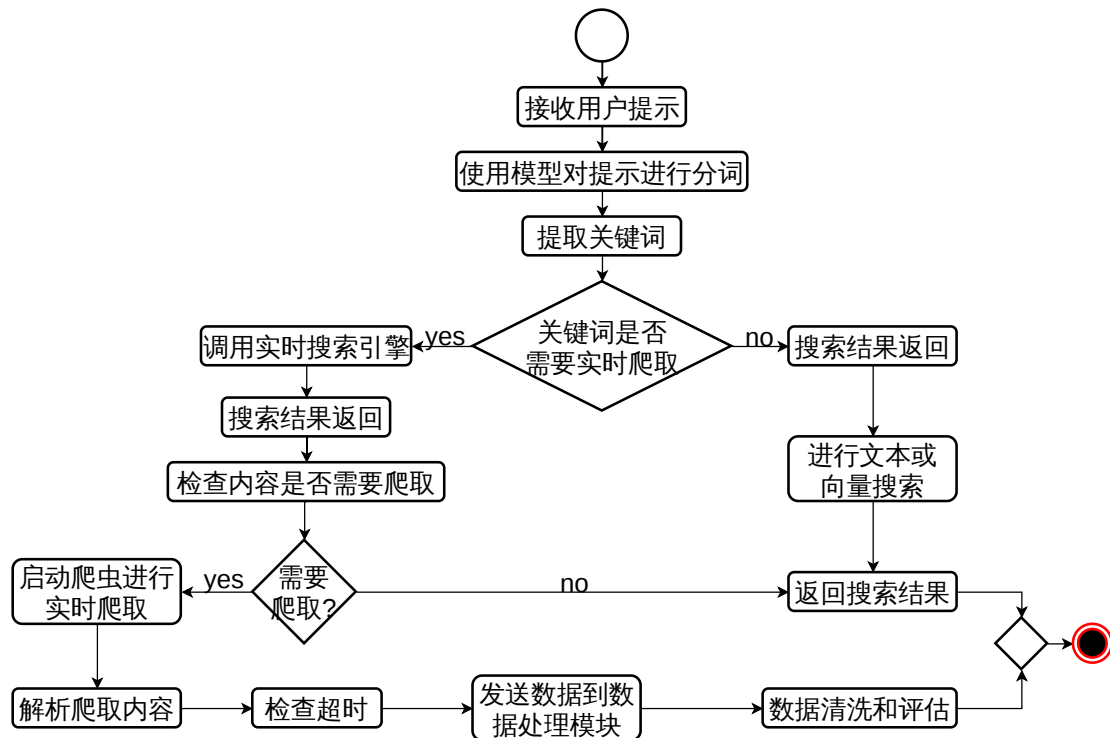


图 3-8: Prompt 解析模块设计流程图

首先，解析模块的输出结构必须具备高度的可解释性，以便在实际应用中进行调试和优化。`QueryPackage` 中的关键词列表是用户输入查询的核心信息，它从用户的原始文本中提炼出最具代表性和搜索价值的关键词，为后续的检索提供基础。扩展查询则是基于关键词的上下文和语义信息，生成可能与用户意图相关的额外查询内容。这一过程通常依赖于自然语言处理（NLP）模型和知识图谱，能够有效地扩展用户的查询范围，提高搜索引擎的覆盖面和准确度。

意图标签则通过分析用户查询的上下文信息，标注出用户的查询意图。例如，用户可能通过“最新的 Python 爬虫教程”来表达他们希望获得一份关于 Python 爬虫的教程资源，解析模块通过分析这种表达方式，会自动将查询标记为“学习教程”类型的意图标签。这一标签有助于后续的搜索引擎理解查询的具体需求，优化搜索结果的排序和呈现。

触发爬取标志则是通过分析查询的关键词和扩展查询，判断用户是否需要获取实时或更深入的内容。如果用户查询的是时效性很强或内容较为特殊的需求，解析模块会通过触发爬取标志来标识该查询需要进一步的爬虫抓取和数据获取。这是在线搜索流程中的重要环节，能够确保系统返回最新的、最相关的内容。

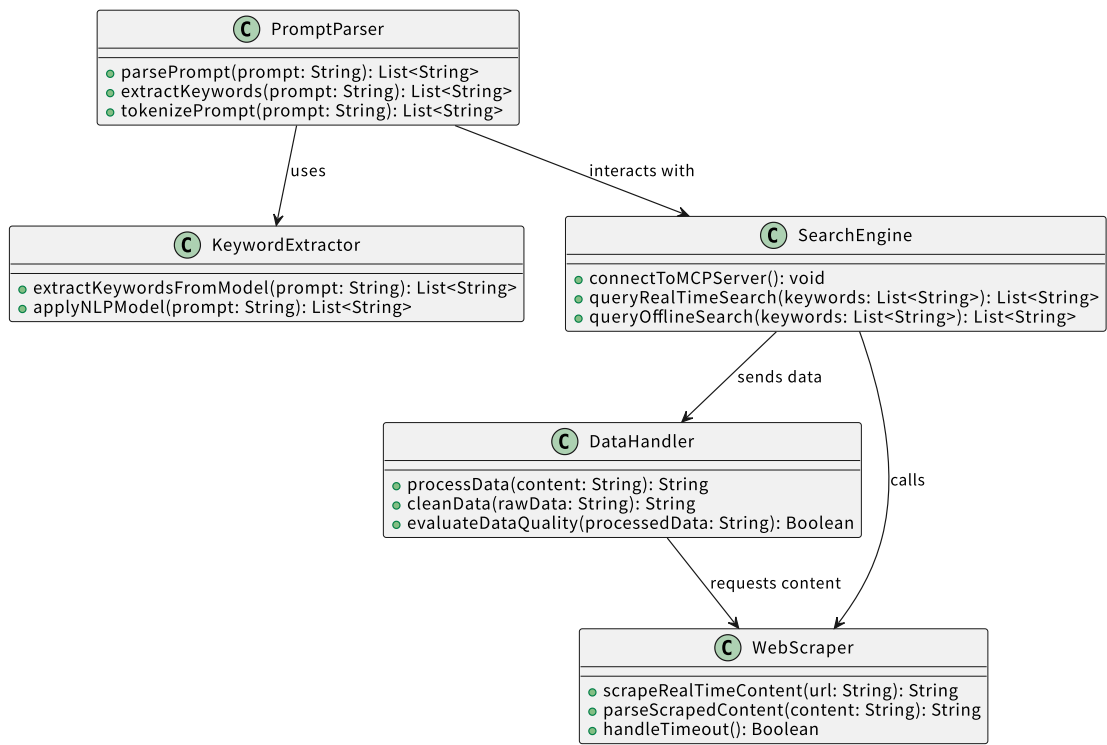


图 3-9: Prompt 解析模块设计设计类图

置信度 score 是解析模块输出的另一个重要组成部分，它表示模型对解析结果的信心。这个分数根据模型的预测结果和对用户查询的理解程度来计算。高置信度通常意味着解析结果准确且与用户查询高度一致，低置信度则可能意味着查询解析存在不确定性或模糊性。在低置信度场景下，解析模块会触发 **fallback** 逻辑，自动降级为规则解析或请求用户进一步澄清。规则解析是通过预设的硬规则来解析查询，例如通过固定的正则表达式匹配某些特定的查询模式，保证系统即使在复杂或模糊的查询情况下也能给出一定的解析结果。

为了保证解析模块的稳定性和一致性，系统维护了一套全面的预处理规范和后处理策略。预处理阶段包括分词、去噪、停用词策略、截断规则等，这些策略的目的是将用户输入的原始查询文本转化为标准化的处理格式，剔除掉无关信息并统一格式。例如，分词策略会将输入的文本进行词语切分，确保系统

能够准确地理解每个词语的含义；去噪策略则会清理掉一些无意义的字符或符号；停用词策略会去除常见的但对查询无实际意义的词语，截断规则则会处理长文本输入，避免系统处理过长的查询导致性能问题。

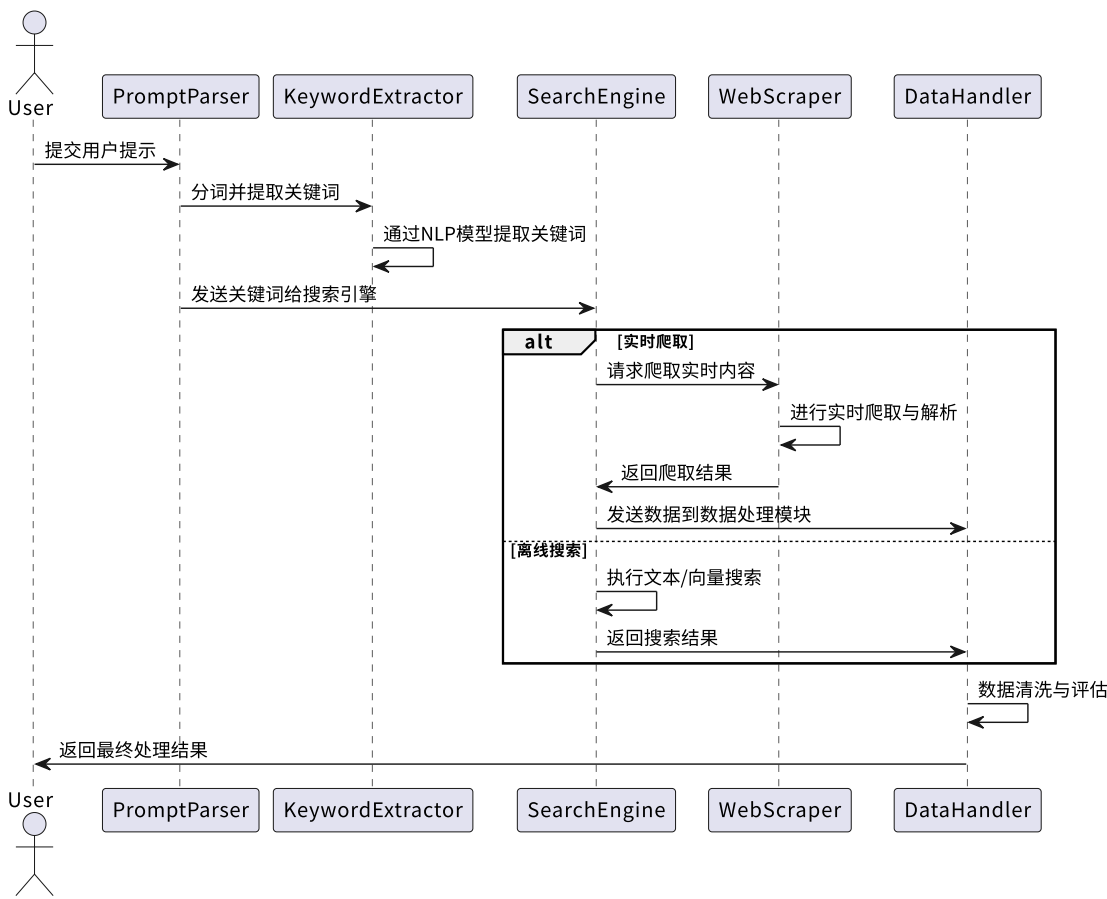


图 3-10: Prompt 解析模块设计时序图

这一高度可配置、可调节的设计使得解析模块不仅能高效地处理常规查询，还能够在面对复杂、模糊或变化的查询时，灵活调整其处理策略，以适应不同的查询类型和业务场景。解析模块的高可解释性和灵活的配置管理，确保了整个 Search Agent 系统在不同数据和业务需求下都能提供稳定、准确的查询解析服务，为搜索引擎的后续处理提供了坚实的基础。

在部署与运维方面，解析模块需要支持多版本并行、动态切换以及 A/B 实验。低延迟路径会部署在靠近 Search Agent 的推理池中，使用 ONNX 实现高效推理；高质量模型则部署在 GPU 池中，并以异步方式服务于离线校准或复杂请求的处理。在这种架构下，模型的响应速度和精确度能够得到最优化，同时支持对不同版本的模型进行实时切换，以便进行迭代和性能调优。为了保证系

统的安全与合规性，解析模块会对敏感信息进行实时识别与脱敏处理。日志层面，所有的 `prompt` 都会进行哈希化存储，这不仅有助于减少敏感信息的暴露，还能满足审计和合规性需求，确保系统的透明性和合规性。

性能监控是解析模块的另一个重要环节，模块通过监控解析延迟分位数、模型置信度分布、`fallback` 率以及解析引起的下游爬取触发率等关键指标，持续评估模型的表现并对其进行优化。这些监控指标为模型的优化与规则阈值调整提供了数据支持，能够帮助团队识别系统瓶颈和改进点。此外，实时的数据反馈也能够帮助调整模型的参数，从而确保系统在不同场景下都能保持较高的鲁棒性和效率。通过这些精细的监控机制，解析模块不仅提升了系统的响应速度和准确度，同时也加强了系统的稳定性和可靠性。

3.4.3 实时检索与爬取模块设计

实时检索与爬取模块承担两类任务：第一在极短时间内提供候选召回，第二在判定需要更详尽内容时触发 `Python` 爬取并完成实时解析。实时检索适配器设计为轻量客户端池，维持与 `bin search` 的连接池与并发控制，侧重在调用延迟与稳定性上做工程优化（并行请求、请求合并、重试与熔断）。热点定时检索作为后台任务维护高频词表并定期刷新结果到本地 `cache`，用于降低突发流量对后端的冲击。检索返回后，`Search Agent` 根据解析输出与 `snippet` 质量判断是否触发爬取。

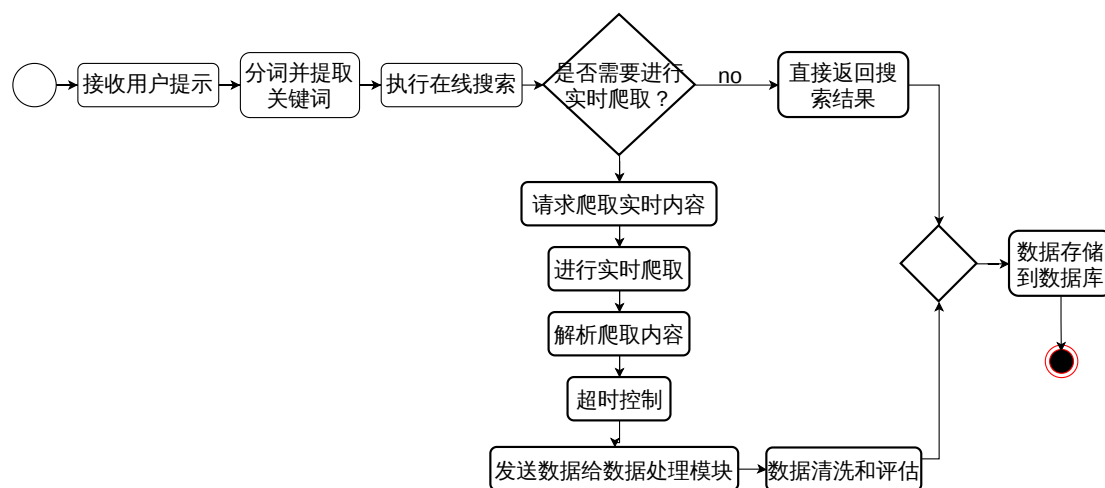


图 3-11: 实时检索与爬取模块流程图

爬取模块是系统中至关重要的组成部分，作为一个可弹性扩缩的爬虫池，它支持大规模、高效、灵活的内容抓取任务，确保能够实时响应用户的搜索请

求并提供精准的内容。整个爬取模块的设计分为三个主要层级：下载层、渲染层与解析层，每个层级的功能和设计都密切配合，以实现高效且稳定的爬取任务。

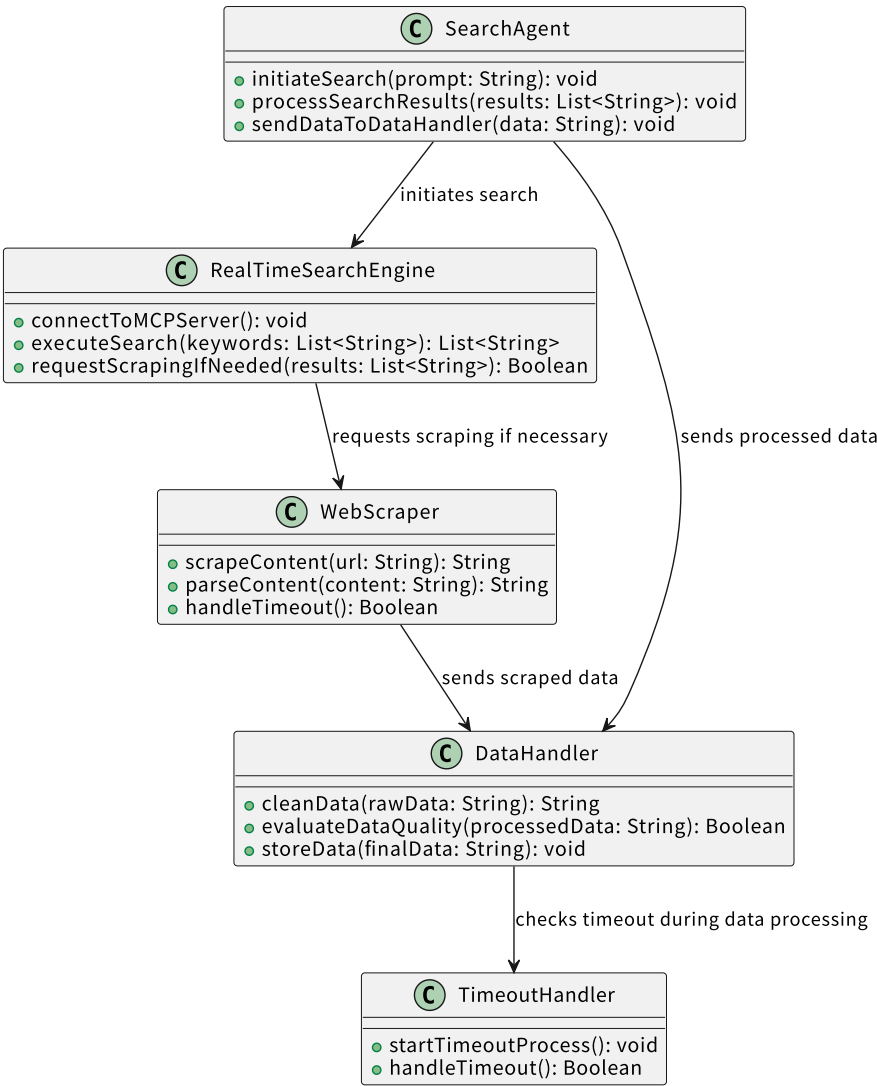


图 3-12: 实时检索与爬取模块设计类图

在下载层，系统首先尝试通过轻量级的 HTTP 请求来获取目标页面。这种方式速度较快且资源消耗低，是大多数页面抓取的首选方法。如果目标页面能够直接通过 HTTP 请求获取，下载层就能快速完成内容的抓取工作，从而提升整体爬取效率。然而，在某些情况下，目标页面的内容依赖于 JavaScript 渲染，或是存在关键数据缺失，这时，系统会自动探测并升级到渲染层。渲染层通过使用 Playwright 或 Puppeteer 等无头浏览器技术来处理 JS 渲染页面，模拟

真实浏览器的操作，以便获取那些需要 JavaScript 渲染才能显示的内容。通过这一层级，系统能够应对更复杂的页面抓取需求，确保无论是静态页面还是动态渲染页面都能被有效爬取。

为了控制成本并遵守目标站点的友好策略，渲染实例池采用了资源隔离的方式，通常在独立的 GPU/CPU 机器组或独立容器组内运行。这种设计可以有效避免不同任务之间的资源竞争，从而提高任务执行的稳定性与效率。在渲染任务的并发控制方面，系统设置了域名级和全局级的配额，以避免过度并发对目标站点造成压力。域名级配额控制可以根据不同网站的特点来设置不同的并发限制，从而确保爬虫行为不会引起目标站点的过度负载或被封禁。全局级配额则保证了爬虫池的整体资源不被过度消耗，确保每个任务都能获得足够的资源执行。

在解析层，系统的主要任务是从抓取到的页面中提取正文内容、结构化字段信息并进行初步质量打分。这一层主要使用基于规则的技术和轻量模型相结合的方法进行内容解析。基于规则的方法可以高效地提取页面中结构明确的内容，如标题、正文、图片等；而 NER 和 DOM 节点分类器则通过模型化的方式，对页面中的实体进行识别，从而为后续的质量评估和数据处理提供更准确的信息。通过这种结合方式，解析层不仅能保证高效的内容抽取，同时也能确保提取的内容具有较高的准确性和可信度。

爬取任务是通过消息队列驱动的，任务描述包含 URL、是否需要渲染、最大抓取时间、优先级以及回调主题。这种设计能够使得爬取任务具有较高的灵活性和可扩展性。任务在执行过程中必须保证幂等性，即同一个任务无论被执行多少次，最终的结果应保持一致。并且任务支持断点续传特性，当任务由于某些原因中断时，能够从中断的地方继续执行，避免重复抓取相同内容或浪费资源。任务完成后，解析结果会以结构化消息的形式写回数据处理队列，供后续的数据清洗、质量评估等环节使用。

超时与并发控制是实时爬取中非常重要的一部分。为了保证爬取任务的稳定性和及时性，每个任务都会设定严格的全链路超时，包括下载超时、渲染超时和解析超时。如果任务在规定时间内没有完成相应操作，系统会自动终止该任务，避免资源的浪费和爬取效率的低下。在网络不稳定或服务器响应较慢的情况下，系统还采用了指数退避和有限重试策略。当任务遇到临时的网络问题或请求失败时，系统会按一定的退避时间重试请求，避免频繁重试造成目标站点的负担，并提高任务成功率。

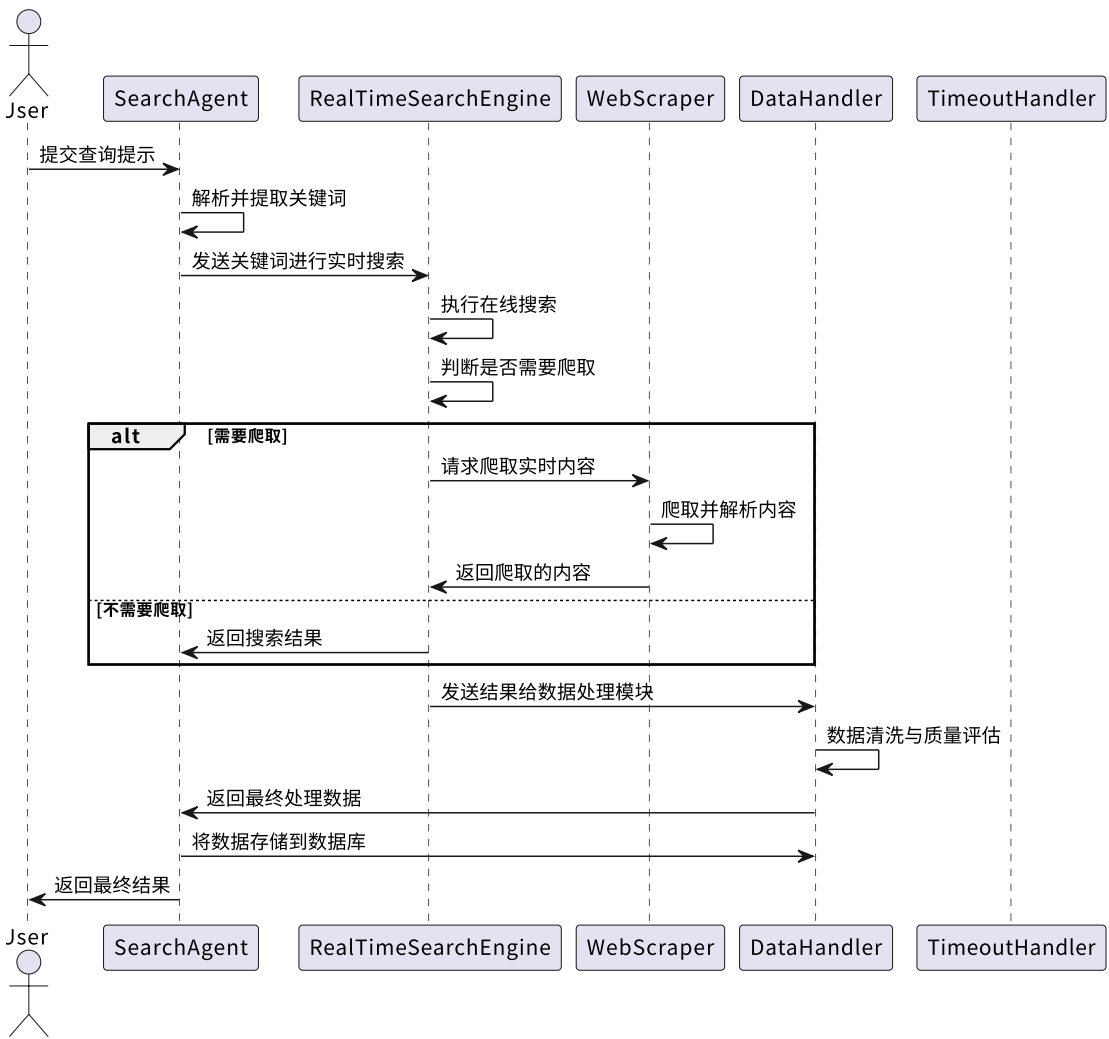


图 3-13: 实时检索与爬取模块时序图

为了避免对目标站点造成过大压力，系统会维护一个域名白名单、灰名单和黑名单，并根据这些名单对爬取策略进行调整。白名单中的站点会被认为是允许频繁爬取的目标，灰名单则会根据站点的响应情况逐步调整爬取频率，而黑名单中的站点则会被完全屏蔽，以避免对目标站点造成负面影响。系统还会在检测到爬取任务失败率过高或存在被封禁风险时，自动降级爬取策略，如降低爬取速度或减少并发请求等，以保护目标站点并提高爬取成功率。

在安全与合规方面，爬取模块在开始爬取前，会检查目标站点的 robots.txt 文件，以确保遵守目标站点的爬取规则。如果站点的 robots.txt 文件明确禁止爬虫访问，系统会自动停止对该站点的爬取任务。此外，对于可能涉及敏感信息的页面，系统会对解析结果进行脱敏处理，或在必要时触发人工复核流程，确保合规性和数据隐私得到保护。

爬取模块还具备全面的监控能力，涵盖了爬取成功率、平均耗时、渲染占比、解析错误率和队列堆积等关键指标。通过对这些指标的实时监控，系统能够快速发现并响应异常情况。若出现异常，系统会自动触发告警与限流策略，确保爬取任务的正常进行，并保障系统的稳定运行。

3.4.4 离线检索与索引模块设计

离线检索与索引模块在整个搜索系统中扮演着重要角色，专门面向垂直领域数据构建高质量的倒排索引与向量索引。这个模块通过精心设计的五个阶段：数据采集、批量清洗、向量化、索引构建与索引发布，确保系统能够高效、准确地处理和存储大量的垂直领域数据，并为后续的搜索请求提供优质的检索服务。

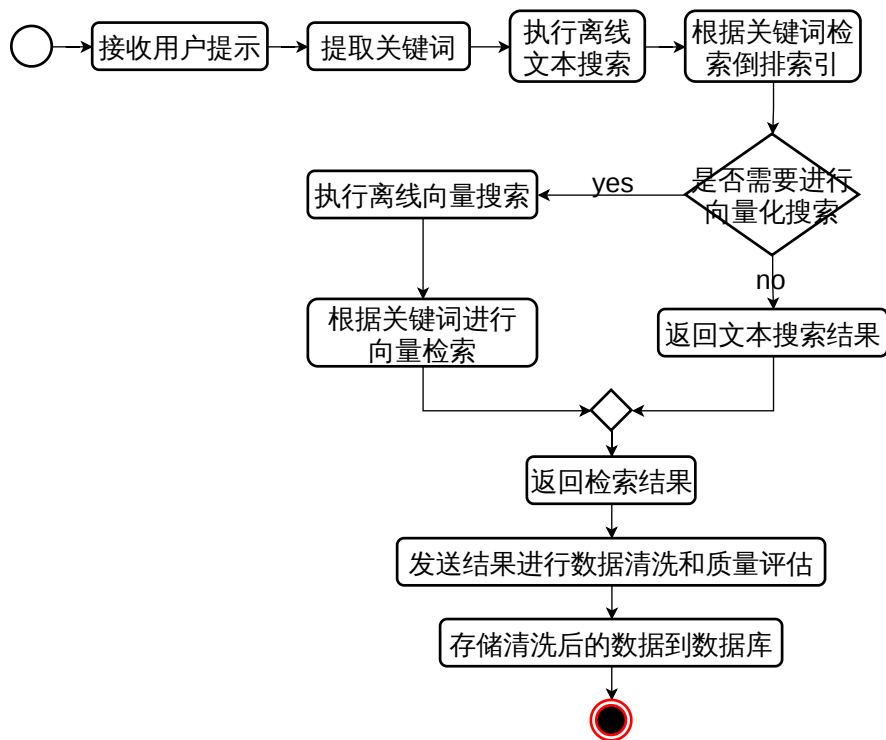


图 3-14: 离线检索与索引模块流程图

首先，数据采集是离线检索管线的第一步。数据采集的主要任务是从不同的数据源获取原始内容。在这个过程中，系统支持多种数据采集方式，包括通过 API 接口拉取数据、通过第三方授权导出数据或通过批量文件导入。这些方式能够灵活适应不同数据源和需求，确保可以从广泛的垂直平台获取所需的原始数据。无论采用哪种方式，数据采集过程都需要记录完整的元数据，以支持

血缘追踪。元数据包括采集的来源、时间戳、数据格式等重要信息，这些数据为后续的数据处理和质量评估提供了必要的上下文支持，并为系统的数据溯源提供了保障。

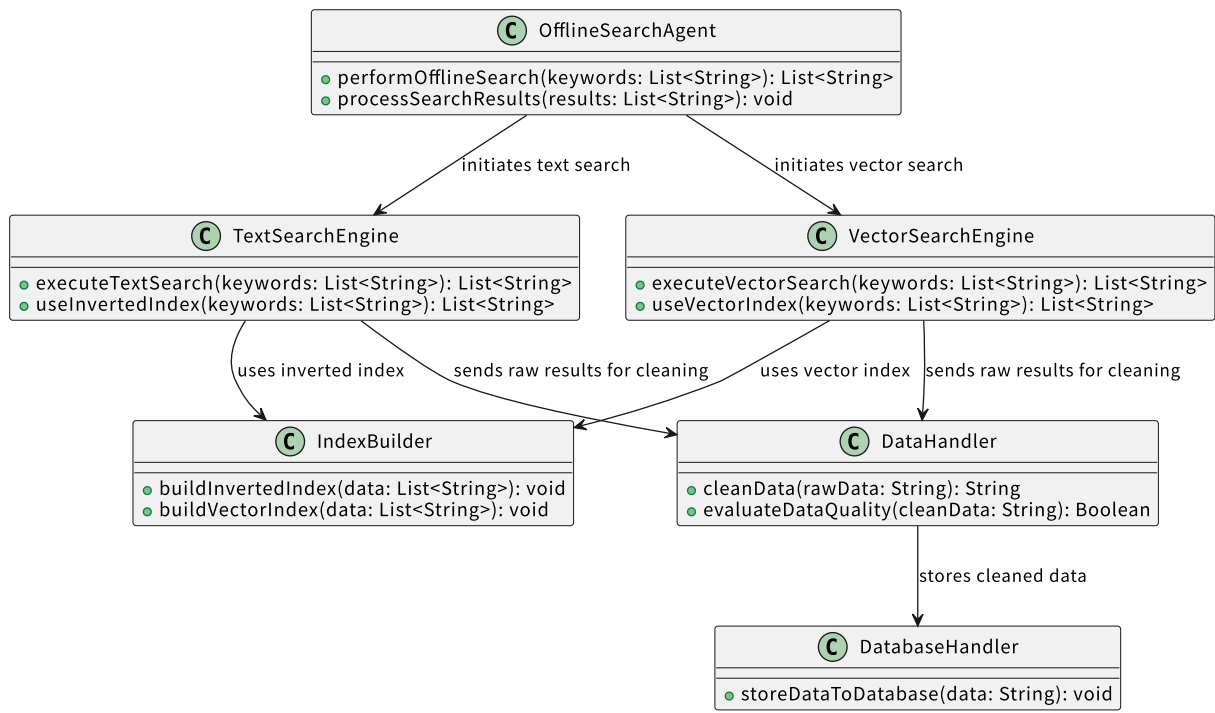


图 3-15: 离线检索与索引模块设计类图

在数据采集完成后，系统进入批量清洗阶段。数据清洗是离线检索流程中的核心步骤，确保输入的数据能够符合后续处理的标准。在批处理框架下，清洗过程包括多个子任务：去重、HTML 去壳、字段标准化、语言检测和初步质量评分。去重是通过消除重复数据，保证索引库中的每个数据项都是唯一且有效的；HTML 去壳则是去除 HTML 标签及其它冗余元素，使得文本数据更加简洁并便于后续的处理；字段标准化则是将不同来源的数据进行统一格式的转换，保证数据在各个字段上的一致性，方便后续索引构建；语言检测帮助系统识别并标注文本的语言类型，从而保证不同语言的数据能够按需处理；初步质量评分则是基于数据的完整性、有效性和一致性进行初步的质量评估，为后续的清洗和优化提供依据。清洗任务完成后，系统会产出可重放的中间结果，以便在数据处理过程中遇到问题时进行回放或重跑。

接下来是向量化阶段，系统使用统一的 `embedding` 模型与预处理规范对清洗后的数据进行向量化处理。向量化是指将原始文本数据转化为数字化的向量表示，这一过程是实现语义检索和相似度搜索的关键。为了确保向量的准确性

和一致性，系统采用经过训练的 embedding 模型，这些模型能够将文本中的语义信息有效地嵌入到向量空间中。每个索引记录都附带了 embedding 模型的版本信息，这为后续的模式回滚或重建提供了便捷的支持。当需要对某个版本的 embedding 模型进行更新或回滚时，系统能够通过记录的版本信息快速进行切换，保证数据处理的一致性和可追溯性。

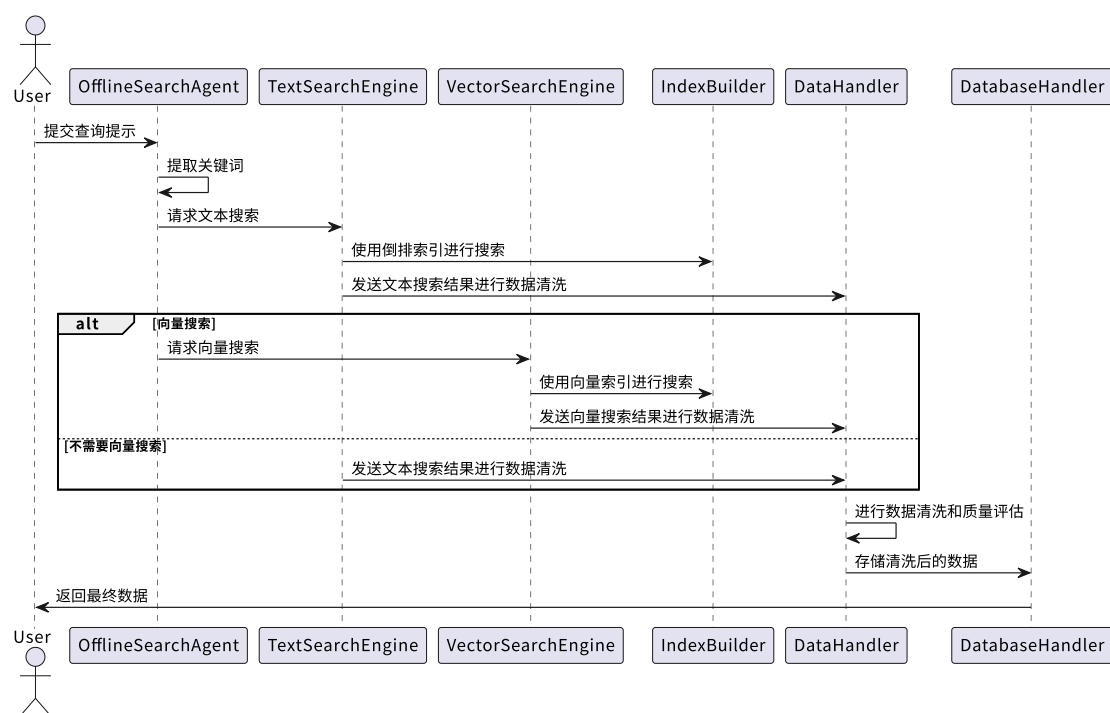


图 3-16: 离线检索与索引模块时序图

向量化完成后，系统进入索引构建阶段。为了高效地处理大规模的数据，系统使用了 Faiss 等专用库来构建向量索引。Faiss 是一个开源库，专门用于高效的相似度搜索和聚类，在构建向量索引时能够显著提高查询效率。在离线调优后，构建的索引参数将被记录并存储在系统数据库中，以便在需要时进行查询优化或参数调整。倒排索引的构建则采用了 Lucene，这是一个广泛使用的开源搜索引擎库，能够快速构建高效的文本倒排索引。通过倒排索引，系统能够快速定位到包含特定关键词的文档，提高查询的响应速度。

为了确保搜索系统的高可用性，倒排索引在构建完成后会通过别名切换技术实现无缝替换旧索引。这种方式保证了查询过程中，用户始终能够访问到最新构建的索引，而不会受到索引更新过程中的影响。通过原子切换，旧索引和新索引的切换是瞬时完成的，避免了系统出现短暂的停机或查询不可用的情况。无缝替换技术确保了索引更新过程中的透明性和稳定性，使得系统能够平

稳过渡到新的索引版本，从而保证了高效和持续的搜索服务。

增量更新策略采用 **delta** 索引与周期性合并，实现对实时或近实时数据的高效处理。小批量写入的数据首先存储在 **delta** 索引中，查询时系统并行访问主索引与 **delta** 索引，确保低延迟检索。达到预定阈值后，后台任务会自动将 **delta** 索引合并到主索引，并进行优化，以减少冗余数据并提高查询性能。为了保证索引的一致性和准确性，入库过程要求幂等写入和事务记录，确保数据完整性。

索引的元数据管理涵盖版本、构建参数、文档计数与快照信息，帮助运维人员进行回滚与容量规划。系统通过实时监控查询延迟、写入延迟、索引增长以及段合并时间分布，确保索引性能的稳定性。此外，支持离线评估，索引模块提供快照回放接口，使得历史索引可切换到测试环境中进行回归验证或 A/B 实验，进一步优化系统的稳定性和准确性。

3.4.5 召回融合与重排模块设计

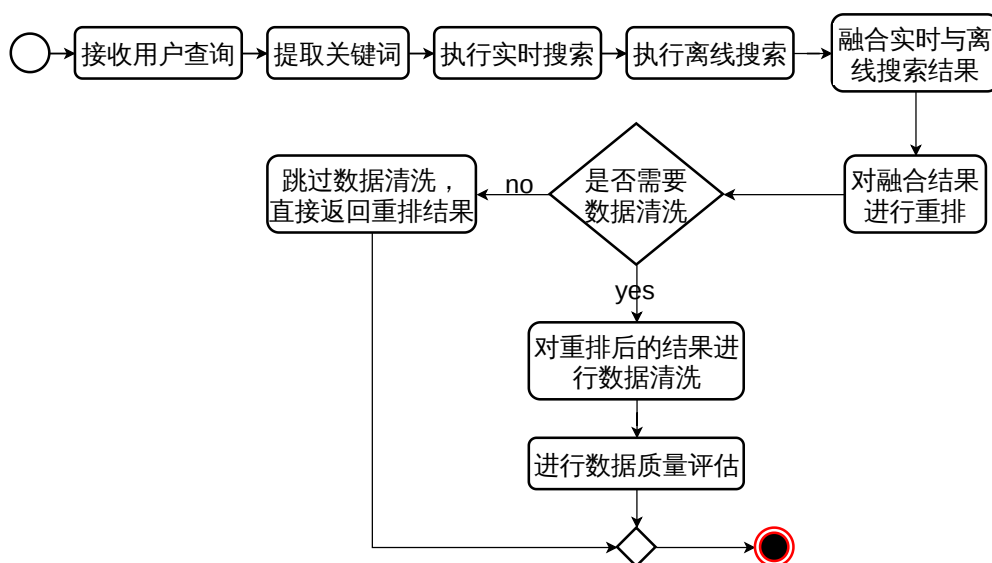


图 3-17: 召回融合与重排模块设计流程图

召回融合与重排模块负责将多通道候选（实时倒排、离线倒排、向量检索、爬取结果）合并、去重并给出最终排名，是影响最终用户感知质量的关键环节。融合阶段首先对不同通道的原始分数进行归一化或校准，以消除评分尺度差异。去重逻辑在候选合并前后均执行：先按 `doc_id` 简单去重，再对文本相

似度或内容指纹进行近似去重以处理文本近似的重复条目。合并策略依据业务权重控制不同通道的候选份额，并在合并后做候选裁剪。

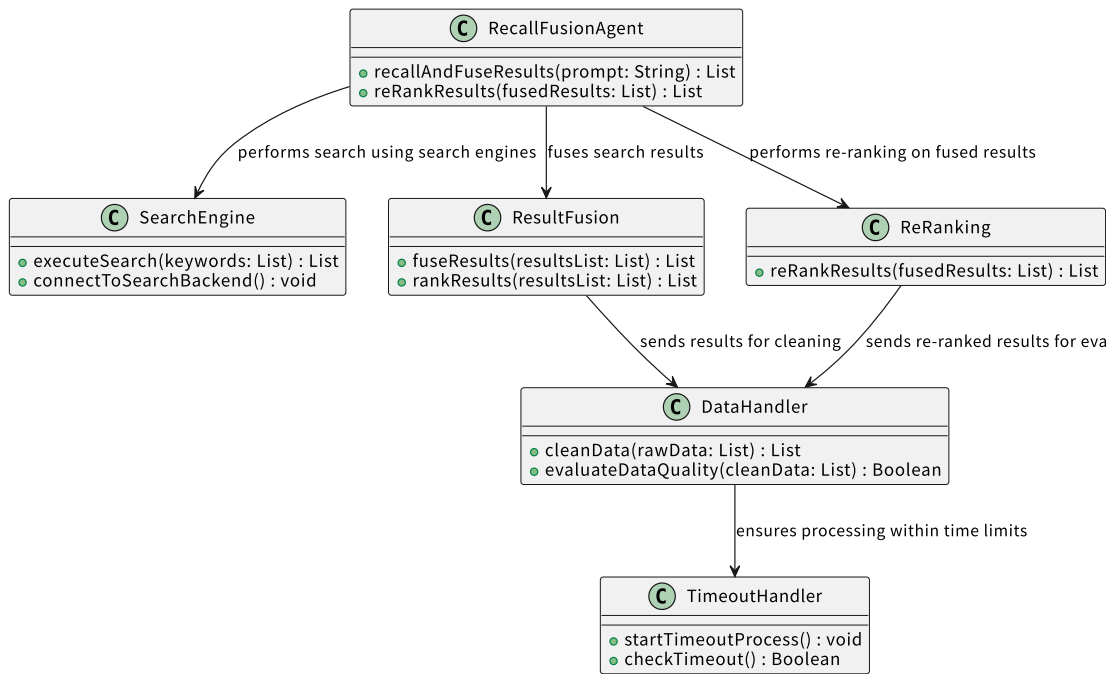


图 3-18: 召回融合与重排模块设计设计类图

召回融合与重排模块是整个搜索系统中至关重要的一环，负责将来自不同通道的候选结果进行合并、去重并最终给出排名。这一过程直接影响到用户对搜索结果的感知质量，是搜索引擎的关键组成部分，确保系统能够为用户提供相关性高、质量优的内容。随着互联网数据量的日益增长和搜索需求的多样化，如何高效地整合多源数据并确保排序的合理性，成为了搜索系统优化的一个关键挑战。

在召回融合的初步阶段，首先需要解决的是不同通道得分尺度的不一致性问题。由于不同搜索通道如实时倒排索引、离线倒排索引、向量检索、爬取结果等使用了不同的评分机制，它们的分数范围和评判标准可能存在较大差异。因此，系统需要对这些原始得分进行归一化或校准，以使得它们处于统一的评分标准下，从而保证在后续的融合和排序过程中，各通道的得分能够公平地参与到排名中。这一归一化或校准的过程，可以通过标准化、最小-最大规范化等方法进行，使得所有得分能够被压缩到同一范围内，或是通过其他调整手段对得分进行补偿，消除评分机制差异带来的不公平。

接下来，去重是召回融合过程中不可忽视的环节。多通道的搜索引擎可能

会返回相同或高度相似的内容，这就需要通过去重策略来确保最终展示的内容具有多样性和独特性。去重的工作分为两个阶段，首先是在候选结果合并之前，依据文档的唯一标识符进行简单去重，确保不同通道返回的相同文档不被重复计算。然而，仅凭文档 ID 的去重并不足以应对内容呈现形式不同的重复情况。很多时候，跨通道或跨平台的内容可能会在表达形式上有所不同，但实际上是重复的。为此，系统会在候选结果合并之后，再进行一次深度去重。这一过程通常通过计算文本相似度或内容指纹来识别和消除近似重复的文档。文本相似度计算能够通过比较文档内容的相似度，识别出重复的内容，而内容指纹技术则通过为每篇文档生成一个唯一的哈希值，即使文档的表现形式不同，也能有效地识别重复内容，防止无意义的冗余结果影响最终输出。

去重完成后，接下来就是合并和排序的过程。合并策略的设计至关重要，通常会依据不同业务的需求来确定各通道的权重。例如，在某些场景下，用户可能更加关注时效性强、更新频繁的实时搜索结果，因此，实时搜索结果可能会被赋予更高的权重。而在另一些情形下，离线搜索结果，如知乎或 XHS 等垂直领域的的数据，因其权威性和深度，可能会被赋予更高的比重。权重的设置能够根据实际业务需求的不同进行灵活调整，以确保最终的搜索结果最符合用户的需求和期望。加权之后，系统会对候选结果进行裁剪，这个过程会根据相关性评分、内容质量、用户行为等多个维度来筛选和剔除低质量的候选，确保最终展示给用户的结果是经过优化和精简的。

候选结果合并、去重和加权处理后，最终的排序阶段会依据预设的排序算法对所有候选结果进行排名。排序不仅仅依据候选结果的相关性得分，还可能结合用户的历史行为、查询上下文等因素进行个性化调整。通过综合考虑用户的兴趣、查询意图以及其他影响因素，系统能够为每一个用户提供更加精准、定制化的搜索结果。此外，系统还可以根据用户的点击率、反馈和行为数据持续优化召回融合与重排模块的策略，以不断提升搜索结果的相关性和用户满意度。

召回融合与重排模块不仅确保了不同搜索通道的合理融合，还通过去重、加权和裁剪等多重手段，提升了搜索结果的质量。这一模块通过优化候选结果的合并和排序流程，最终向用户呈现出高质量的搜索结果，提升了搜索引擎的整体效率和用户体验。随着数据量和用户需求的不断变化，召回融合与重排模块将继续在搜索系统中发挥着越来越重要的作用，成为优化搜索质量、提升用户体验的核心所在。

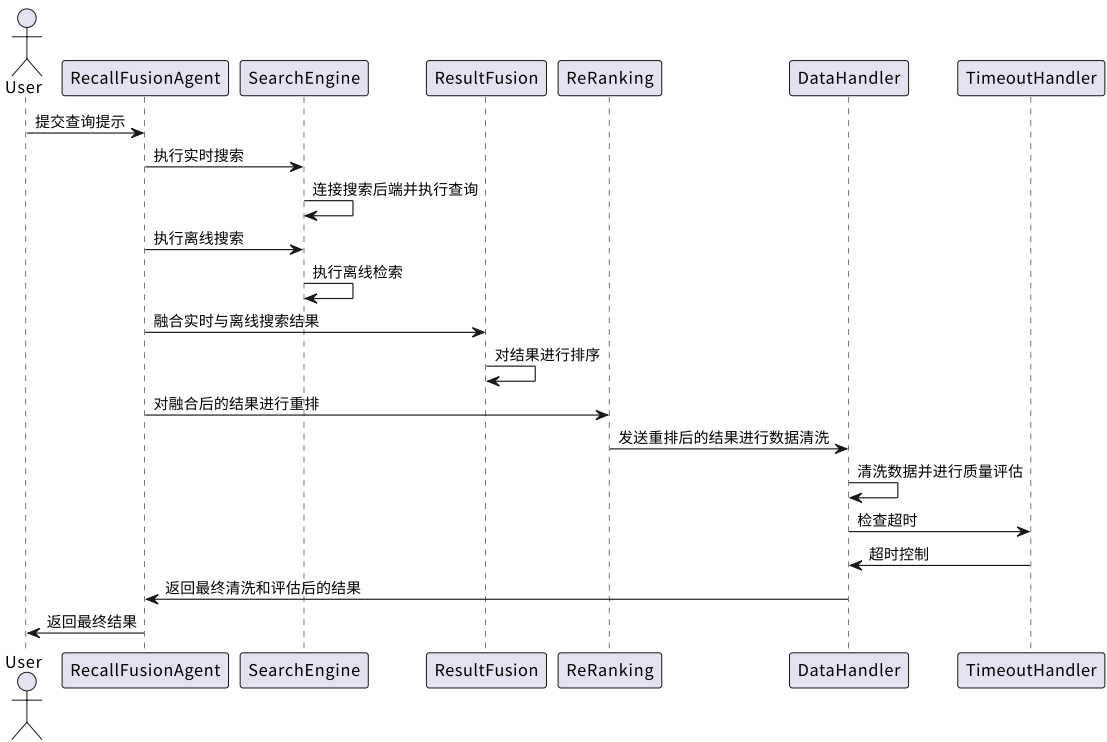


图 3-19: 召回融合与重排模块设计时序图

在线部署则需考虑模型推理并发、特征预计算与缓存。为降低在线延迟，许多特征可以预先离线计算并存储在特征仓库，实时请求只需取用。重排模块需监控关键指标，并在 CI/CD 中对模型上线进行离线回归与在线小流量灰度验证。最终，融合与重排输出应携带足够的解释性信息以便前端展示“结果为何被选中”的透明提示并支持审计。

3.5 数据库与索引存储设计

本节围绕系统中承担数据持久化、索引存储与检索支持的数据库与索引层进行系统性设计说明。设计目标是在保证检索质量与数据新鲜度的前提下，兼顾写入吞吐、查询延迟、可扩展性、可恢复性与合规安全。整体存储分为三类原始抓取/中间产物存储、结构化/元数据存储、以及检索索引存储。各类系统在逻辑上相互配合，通过消息队列与元数据引用完成端到端的数据流转与可追溯性。

原始数据与中间产物存储采用对象存储作为单一真相。所有爬取产物、第三方导出文件、以及离线批处理中间文件都以时间和任务维度组织在对象存储

中，并保留完整的元信息。对象存储的优势在于强持久性、成本可控及便于回放重跑。当出现解析模型或清洗逻辑问题时，可基于对象存储的原始快照重新执行流水线。为支持快速检索与短期回放，系统在对象存储之上还维护短期的 Redis 缓存，例如将近期抓取的正文或解析结果的嗅探摘要保存为缓存键，便于低延迟展示与幂等检查。

表 3-6: 用户查询表：存储用户输入的查询信息

字段名	数据类型	描述
query_id	INT	用户查询的唯一标识
user_id	INT	用户的唯一标识
query_text	TEXT	用户输入的查询文本
query_timestamp	DATETIME	查询时间
keywords	TEXT	由模型提取出的关键词

`user_queries` 表是系统中存储用户查询信息的核心表格。它记录了每个用户提交的查询内容以及与之相关的其他信息。首先，`query_id` 是每次查询的唯一标识，确保能够唯一地识别用户的每个查询。`user_id` 则是标识执行查询的用户的唯一标识符，通常是通过用户注册或认证系统分配的，用于区分不同的用户。`query_text` 存储用户输入的查询文本，即用户在搜索框中输入的具体内容。这些查询文本会被进一步传递给模型进行分词处理，提取出相关的关键词。

`query_timestamp` 字段记录了查询的时间，这对于日志分析、用户行为分析等非常重要。通过时间戳，可以了解用户活跃的时间段，以及是否存在某些季节性或突发的查询需求。`keywords` 则是通过模型对用户查询文本进行处理后提取出来的关键词，关键词的提取是系统进行后续搜索和内容筛选的关键。这些关键词不仅影响到搜索引擎的查询结果，还决定了是否需要爬取外部数据或进行更复杂的内容展示。

这个表格的设计支持快速存储和检索用户的查询信息，为后续的数据处理、爬取和质量评估提供基础数据。同时，它为系统的日志记录、用户行为分析以及查询分析提供了宝贵的参考。

`search_results` 表用于存储从搜索引擎返回的搜索结果。每次用户提交查询后，系统会根据查询文本和提取的关键词调用搜索引擎（如实时在线搜索或离线搜索）进行检索，返回的结果将被记录在此表格中。

首先，result_id 字段为每条搜索结果提供了一个唯一标识，确保每条记录能够被单独识别。query_id 则与 user_queries 表中的 query_id 关联，标明当前搜索结果对应的是哪次查询。这使得我们能够追踪到每个用户的查询以及该查询的返回结果。

表 3-7: 搜索结果表：存储搜索引擎返回的结果

字段名	数据类型	描述
result_id	INT	搜索结果的唯一标识
query_id	INT	关联的用户查询 ID
source_type	VARCHAR(50)	搜索源类型（如在线搜索、离线搜索）
content_id	INT	搜索到的内容的唯一标识
content_url	TEXT	搜索结果的 URL 或路径
content_summary	TEXT	搜索结果的摘要
content_timestamp	DATETIME	内容抓取的时间
status	VARCHAR(20)	搜索状态（如成功、失败、爬取中）

source_type 字段指示该搜索结果来自哪个搜索源，可能的值包括“在线搜索”和“离线搜索”，这有助于区分实时数据和历史数据的来源。content_id 用于关联 content_library 表中的内容记录，指明该搜索结果具体是哪个内容。content_url 存储了该搜索结果的具体链接或路径，可以是一个网页 URL，也可以是一个本地存储路径。

表 3-8: 爬取的数据表：存储从爬虫抓取的数据

字段名	数据类型	描述
crawled_id	INT	爬取数据的唯一标识
content_id	INT	关联的内容 ID
raw_content	TEXT	原始爬取的内容
cleaned_content	TEXT	清洗后的内容
crawl_timestamp	DATETIME	爬取时间
status	VARCHAR(20)	爬取状态（如成功、失败）

content_summary 字段提供了该搜索结果的简短摘要，帮助用户快速了解搜

索结果的概要信息。`content_timestamp` 记录了搜索结果的抓取时间，帮助系统追踪抓取的时间窗口，便于在内容更新时及时刷新结果。最后，`status` 字段记录了该搜索结果的处理状态，如“成功”、“失败”或“爬取中”，这些状态对于调试和监控爬虫任务非常重要。

`crawled_data` 表记录了通过爬虫从外部网站抓取的原始数据。随着在线搜索的进行，如果搜索引擎认为需要提供更详细的内容，系统会启动爬虫从指定的页面进行抓取。该表格的作用是存储这些爬取到的原始数据，并为后续的数据清洗和处理提供基础。

每条爬取数据都有一个唯一的标识符 `crawled_id`，以便于管理和追踪。`content_id` 用于与 `content_library` 表中的内容记录关联，帮助我们识别哪些内容是通过爬虫获取的。`raw_content` 存储了爬虫抓取到的原始内容，可能包含杂乱的信息、HTML 标签等。`cleaned_content` 则存储了清洗后的内容，去除冗余和无关信息，使其更加规范和有用。

`crawl_timestamp` 字段记录了数据的爬取时间，这对于判断数据的新鲜度和时效性至关重要。爬虫的状态通过 `status` 字段进行标识，常见的状态包括“成功”和“失败”。爬虫状态的管理对于优化爬虫流程、排查错误及防止爬虫陷入死循环等问题非常有帮助。

表 3-9: 内容库表：存储所有的内容，包括文本、图像等

字段名	数据类型	描述
<code>content_id</code>	INT	内容的唯一标识
<code>content_type</code>	VARCHAR(50)	内容的类型（如文本、图像）
<code>content_data</code>	TEXT	内容数据
<code>timestamp</code>	DATETIME	内容录入时间
<code>status</code>	VARCHAR(20)	内容状态（如正常、已删除）

`content_library` 表是整个系统中的核心内容存储库，所有的内容数据都会在这里进行归档。这个表格记录了所有通过搜索引擎、爬虫或其他方式获得的内容，无论是文本数据、图像还是其他格式的数据。该表为系统后续的搜索结果展示和数据分析提供了稳定的支持。

`content_id` 是该表的主键，用于唯一标识每一条内容记录。`content_type` 字段用于标记内容的类型，常见的类型包括文本、图像、视频等，这对于后续的内容处理非常重要。`content_data` 则存储了具体的内容数据，可能是纯文本、图

像的 URL 链接，或者是其他类型的数据。

timestamp 字段记录了该内容的录入时间，这对于内容的版本控制、时效性评估和更新非常重要。status 字段则指示内容的状态，如“正常”表示该内容有效，“已删除”表示该内容被移除，帮助系统在展示时能够过滤掉无效或过时的内容。

表 3-10: 处理后的数据表：存储经过清洗和质量评估后的数据

字段名	数据类型	描述
processed_id	INT	处理数据的唯一标识
raw_data_id	INT	关联的原始数据 ID
cleaned_data	TEXT	清洗后的数据
quality_score	FLOAT	数据质量分数
timestamp	DATETIME	处理时间
status	VARCHAR(20)	处理状态（如成功、失败）

processed_data 表用于存储经过清洗和质量评估的数据，旨在提升数据的准确性和可靠性。数据清洗是从爬虫获取原始数据到最终展示内容的关键一步，目的是删除不必要的噪声、格式化数据并规范化内容。这个表的设计确保了清洗后的数据能够更好地用于后续的分析和展示。

processed_id 字段是该表的主键，每条处理过的数据都有唯一的标识。raw_data_id 关联了原始数据的 ID，通过与 crawled_data 表的关联，可以追溯每条处理数据的来源。cleaned_data 存储了清洗后的数据，这些数据已被标准化并且去除了冗余信息，确保了数据的高效利用。

quality_score 字段是一个浮动值，用于表示处理数据的质量评分。质量评估通常基于数据的完整性、准确性和时效性等多方面的标准，这对于系统的可靠性至关重要。timestamp 字段记录了数据处理的时间，便于管理和版本控制。status 字段则指示处理状态，如“成功”表示数据已处理完成，“失败”则表示处理过程中出现了问题。

结构化元数据与任务状态存储采用关系型数据库与 NoSQL 结合的方式。核心元数据包括文档元信息表、爬取任务表、索引元数据表、质量评估表与审计日志表。WebContentDB 为检索条目的目录中心，记录 document_id、canonical_url、title、source、fetch_time、processing_version、quality_score、index_status 等关键字段；设计时需为 document_id 与 canonical_url 建立唯一约束以保证

去重幂等写入。爬取任务表记录 task_id、url、priority、attempts、last_status、worker_id、start_time、end_time 与 error_reason，用于任务恢复、溯源与运维。索引元数据表记录每次索引构建的版本号、embedding_model_version、构建参数、segment_info 与 snapshot_location，以支持回滚与 A/B 检验。审计日志表记录对关键数据或配置的审批与变更轨迹，保证合规审计要求。

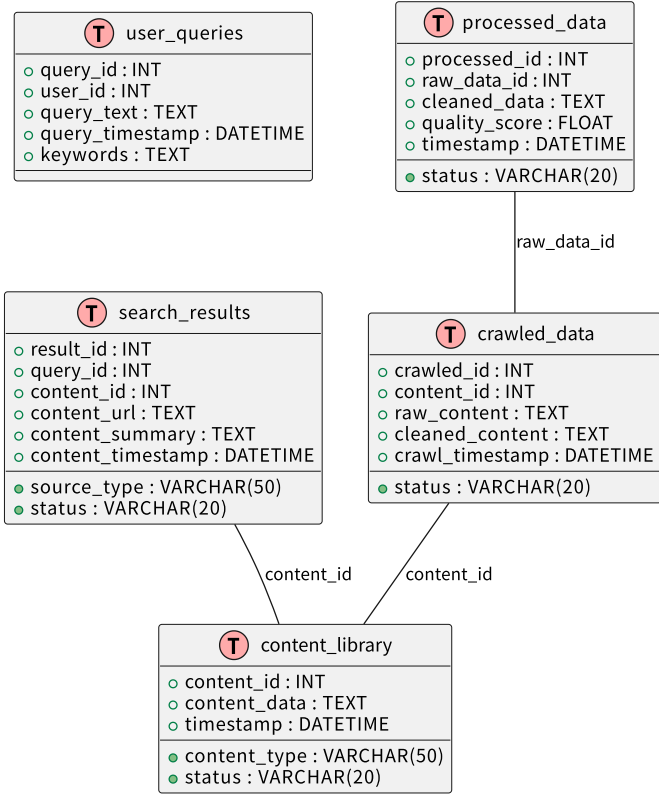


图 3-20: 关系型数据数据库设计 er 图

倒排索引与向量索引分别采用专门的索引系统来满足不同检索特性，确保系统在处理传统文本检索与基于语义的检索时都能具备高效性和准确性。倒排索引是传统信息检索的基础，适用于高频的短文本检索。它将文档中的每个词条与包含该词的文档建立倒排映射。推荐使用成熟的分布式搜索引擎来实现倒排索引系统。这些引擎提供了丰富的查询表达能力，支持高级的查询语言与过滤功能，能有效满足用户复杂检索需求。此外，它们具备强大的分词与分析器定制功能，可以根据特定应用场景对中文或其他语言的分词进行优化，从而提高检索的精度。分片与副本机制则是分布式系统中不可或缺的组成部分，它们保障了索引的扩展性与容灾能力，在处理海量数据时能平衡负载并提供高可用性。

为了支持中文语义与短文本 `prompt` 的检索，倒排索引的 `mapping` 设计非常关键。首先，需要为不同字段使用差异化的分词器，尤其是在处理中文时，可以采用细粒度的分词策略，避免语义丢失。同时，为了提高短语匹配的能力，针对需要短语匹配的字段，索引设计时应保留 `position` 信息，这样不仅能支持高效的短语检索，也能保证文档的语义完整性。此外，设置 `term vectors` 可以使检索引擎对每个词汇的出现位置、频率和邻接关系有更加详细的记录，这对于后续的重排特征计算至关重要，能够显著提高检索结果的相关性与准确性。

在倒排索引的分片策略方面，建议采用逻辑哈希分配分片的方式，以便在大规模数据集上能够均匀分布查询负载并支持横向扩展。此外，为了保障高可用性和容灾能力，系统应配置副本机制，确保在某一分片出现故障时，其他副本能够承担查询请求，保证系统的正常运行。对于高写入场景，倒排索引的 `segment merge` 策略需要进行调优，以避免合并操作时产生的 I/O 瓶颈。在实际应用中，`segment merge` 时的频繁 I/O 可能会导致系统出现性能瓶颈，特别是在实时更新场景下。为了减少这些影响，系统应优化合并策略，合理调整合并的阈值和频率，避免对系统吞吐产生过大压力。

针对实时通道的低延迟需求，倒排索引服务应启用 `near-real-time` 配置，以确保索引能够在较短时间内反映数据的更新。这意味着索引刷新过程应尽量减少对查询性能的影响。在此基础上，可以结合 `delta` 索引策略来进一步优化写入性能。`Delta` 索引允许系统将新增的文档或数据记录先写入一个小规模的近实时索引，当达到一定条件后，再将这些增量数据合并到主索引中。这样不仅能减少索引刷新频率带来的合并压力，还能有效提升实时查询的性能。

向量索引则专门用于语义搜索，尤其是在处理复杂查询如短文本或非结构化数据时，能够显著提高检索结果的相关性。向量索引通常基于先进的向量化技术，如 `Word2Vec`、`BERT` 等模型生成的向量表示，并依赖于高效的向量检索引擎来提供相似度搜索功能。与倒排索引相比，向量索引更侧重于通过语义相似度计算来完成检索，而不仅仅依赖于词汇的精确匹配。因此，向量索引的建库方式需要更多地考虑向量化的质量以及计算效率。为了提高检索性能，向量索引系统通常会采用高效的近似最近邻搜索算法，通过构建 `K` 近邻图或其他数据结构来加速大规模向量空间中的相似度查询。

为了支持向量索引的高效存储与检索，系统必须对向量数据进行有效的索引构建。常见的向量索引方式包括基于树的数据结构、基于量化的索引以及图搜索算法。在建库过程中，系统需要根据实际场景的需求选择合适的索引策

略。例如，对于实时性要求较高的场景，向量索引系统可以结合多层次索引机制，将高维向量空间划分为多个子空间，通过索引优化加速检索速度；而对于批量数据，系统可以采用静态批量索引方式，以保证高效的存储和查询性能。

3.6 本章小结

本章从模块设计延伸到数据库与索引存储的具体架构，系统性地回答了 Search Agent 在数据持久化、检索索引与可运维性方面的关键工程问题。数据处理模块建立了规则 + 模型的质量评估闭环，支持自动降级与人工复核；召回融合与重排模块采用分层设计兼顾效果与性能。数据库与索引存储设计部分则提出了面向工程实践的落地方案：对象存储作为原始快照、关系型/NoSQL 存储负责元数据与任务管理、倒排与向量索引分别承担文本与语义检索，并通过别名切换、备份快照与索引版本管理实现可回滚与灰度。

第四章 实时与离线混合检索系统的设计与实现

本章将详细介绍 Search Agent 系统的实现过程。该系统主要用于对用户输入的查询内容进行精准的关键词提炼和分析，通过结合自然语言处理模型对查询进行分词和意图识别，进而调用后端搜索引擎进行检索。质检系统不仅对实时搜索结果进行数据清洗、二次清洗和质量评估，还实现了对离线搜索数据的检索与分析，确保了系统输出结果的高质量与精准性。通过完善的数据处理与评估机制，系统能够有效优化搜索效果和用户体验。

4.1 Prompt 解析模块的实现

表 4-1: Prompt 解析模块接口描述（Search Agent）

接口名	功能描述	责任划分
parse_prompt	用户 Prompt 提取关键词	MCP 负责调用模型解析
realtime_search	在线搜索请求发起与管理	在线搜索后端负责检索
realtime_crawl	实时爬取并解析搜索结果	爬虫进程负责爬取解析
push_to_processing	检索内容实时推送	Agent 推送数据处理消费
offline_search	离线垂域文本/向量搜索	离线索引服务负责检索
index_build	倒排与向量库建库更新	系统负责建库维护
quality_assessment	检索质量评估与二次清洗	数据处理模块负责清洗

Prompt 解析模块的实现目标是在严格的延迟预算内，将用户自然语言提示稳定、可解释地转化为检索指令与任务元数据，并向上游/下游提供清晰的接口与审计信息。实现上采用“规则层 + 在线轻量模型 + 离线强模型”三层架构：规则层负责拦截显性命令、合规检查和简单模板匹配；在线轻量模型负责低延

迟的关键词提取、意图分类与 query rewrite；离线强模型在异步路径对在线解析结果做校正与质量回流，用于周期性更新规则与模型。

```

1 import java.util.*;
2 import java.util.concurrent.*;
3
4 public class PromptParser {
5     private final ModelClient modelClient;
6     private final McpClient mcpClient;
7     private final ExecutorService executor = Executors.
newCachedThreadPool();
8
9     public PromptParser(ModelClient m, McpClient c) { this.modelClient
= m; this.mcpClient = c; }
10
11     public void handlePrompt(String userId, String prompt) {
12         List<String> keywords = extractKeywords(prompt);
13         if (keywords.isEmpty()) { log("no keywords"); return; }
14
15         SearchRequest req = buildSearchRequest(userId, prompt,
keywords);
16         try {
17             SearchResponse resp = sendWithTimeoutAndRetry(req, 3, 2000
);
18             dispatchToRecallModule(resp);
19         } catch (Exception e) {
20             log("search failed: " + e.getMessage());
21         }
22     }
23
24
25     private SearchResponse sendWithTimeoutAndRetry(SearchRequest req,
int retries, long timeoutMs) throws Exception {
26         for (int i=0; i<retries; i++) {
27             CompletableFuture<SearchResponse> cf = CompletableFuture.
supplyAsync(() -> mcpClient.sendSearch(req), executor);
28             try {
29                 return cf.get(timeoutMs, TimeUnit.MILLISECONDS);
30             } catch (TimeoutException te) {
31                 cf.cancel(true);
32                 log("timeout, retry " + (i+1));
33             }
34         }
35         throw new Exception("all retries failed");
36     }
37 }

```

图 4-1: Prompt 解析模块的实现

输入输出接口统一用 Protobuf 描述，输入包含原始 prompt、用户上下文、请求优先级与 trace_id；输出包含关键词列表、候选重写查询、多意图子任务列表、触发爬取标志、置信度与解析元数据。在线模型部署在低延迟推理池，并结合局部缓存缓存热 prompt 的解析结果，支持批量化推理以提升并发吞吐。为

了保证可控性与可解释性，模型输出经过后处理规则校验：黑名单命中会覆写触发爬取标志，敏感实体将被脱敏或触发人工复核标识。

解析过程包含多项工程保障：预处理包括中文分词/normalize、拼写纠错与短语保留；实体链接通过轻量 NER 与外部知识库做快速校正；置信度校准通过温度缩放或后验校准器降低输出过度自信的风险。异常处理策略包括模型超时回退到规则、模型低置信触发澄清交互或调用离线增强路径。为支持持续迭代，解析模块埋点采集解析耗时分布、置信度分布、触发爬取率与 fallback 率，并将样本写入训练集，用于离线评估与模型重训练。版本管理、灰度发布与 A/B 实验通过 MCM 协调：模型新版本先在少量流量灰度，若关键 SLI 稳定再全量发布。

4.2 实时检索与爬取模块的实现

表 4-2: 实时检索与爬取模块接口描述（Search Agent 实时模块）

接口名	功能描述	责任划分
start_hotspot_polling	定时触发热点检索	调度服务负责触发
query_online_search	发起在线检索请求	Agent 调用在线后端
stream_search_results	实时推送检索结果流	在线后端负责推送
trigger_realtime_crawl	关键词触发实时爬取	爬虫服务负责爬取解析
crawl_parse_realtime	内容实时解析摘要	爬虫解析组件负责
crawl_timeout_control	爬取超时与重试控制	爬虫调度负责
push_to_data_processing	结果推送数据处理	Agent 推送数据处理消费

实时检索与爬取模块实现的核心在于“先快后补”的工程策略：实时检索提供低延迟候选，爬取模块在判定必要时异步抓取详情并通过数据处理管道补全入库。爬取模块由任务调度、下载渲染与解析三部分组成。任务调度器接收来自 Search Agent 的任务，做去重、优先级排队、域名限速与配额校验，并将合格任务放入 Kafka-like 队列。下载层先尝试轻量 HTTP 抓取，快速判断是否需要渲染；若需要，则进入渲染池，渲染池采取资源隔离与池化策略，限制每个域名并发与全局渲染并发以控制成本与避免被封禁。解析层结合规则与模型提取结构化字段，并产生 parse_confidence、extraction_errors 等质量元数据。任

务全过程严格的超时控制与幂等设计保证系统在高并发或目标站点异常时以可控方式降级。

```

1 import java.util.*;
2 import java.util.concurrent.*;
3
4 public class RealtimeRetrievalCrawler {
5     private final ScheduledExecutorService scheduler = Executors.
newScheduledThreadPool(2);
6     private final ExecutorService crawlerPool = Executors.
newFixedThreadPool(4);
7     private final OnlineSearchClient onlineClient;
8
9     public RealtimeRetrievalCrawler(OnlineSearchClient o, Crawler c,
DataProcessor d){
10         this.onlineClient=o; this.crawler=c; this.dataProcessor=d;
11     }
12
13     public void startHotspotPolling(long initialDelaySec, long
periodSec){
14         scheduler.scheduleAtFixedRate(this::performHotspotSearch,
initialDelaySec, periodSec, TimeUnit.SECONDS);
15     }
16
17     private void performHotspotSearch(){
18         try {
19             String query = buildHotspotQuery();
20             List<Hit> hits = queryOnlineSearch(query);
21             streamSearchResults(hits);
22             hits.stream().filter(this::needsDetail).forEach(h ->
triggerRealtimeCrawl(h.url, h));
23         } catch (Exception e){ log("hotspot fail: "+e.getMessage()); }
24     }
25
26     private void triggerRealtimeCrawl(String url, Hit hit){
27         int retries = 2; long timeoutMs = 3000;
28         for(int i=0;i<=retries;i++){
29             Future<Parsed> f = crawlerPool.submit(() -> crawler.
fetchAndParse(url));
30             try {
31                 Parsed p = f.get(timeoutMs, TimeUnit.MILLISECONDS);
32                 dataProcessor.processCrawlResult(hit, p);
33                 return;
34             } catch (TimeoutException te){
35                 f.cancel(true); log("crawl timeout, retry="+i);
36             } catch (Exception e){
37                 log("crawl error: "+e.getMessage()); return;
38             }
39         }
40         log("crawl failed after retries: "+url);
41     }
42
43 }

```

图 4-2: 实时检索与爬取模块的实现

实时检索适配器实现连接池管理、并发控制、请求合并与熔断策略；对热

点词设置本地缓存与提前预热任务以应对突发流量。调用流程是：Search Agent 将 QueryPackage 发给 bin search 适配器，适配器并行调用倒排与向量接口，快速合并 TopK 候选并返回给 Search Agent；同时根据触发逻辑决定是否下发爬取任务。爬取输出通过消息队列下发到实时解析消费者，消费者完成快速校验后将初步结构化数据写入中间存储并触发数据处理流水线。为保证合规与安全，爬取前会检查 robots.txt、站点白/灰/黑名单并记录审计日志；对含敏感信息的解析结果触发自动脱敏或入人工复核队列。监控维度包括爬取成功率、域名级失败率、渲染占比、队列积压与爬取引起的 downstream lag，异常阈值触发自动限流或暂停爬取策略。为充分利用资源，系统支持基于优先级的动态扩缩容，低优先级任务在高负载时被排队或降级。

4.3 离线检索与索引模块的实现

离线检索与索引模块负责大规模垂域数据的采集、清洗、向量化与建库，目标是提供高质量、语义丰富且可持续更新的索引服务。实现管线分为采集层、批量清洗层、向量化/索引构建层与发布层。采集层支持 API 拉取、合作方数据导入与授权导出，所有原始文件与采集快照落到对象存储并记录详细元数据以便重跑。批量清洗基于 Spark/Flink 或 K8s batch jobs 执行，包含 HTML 去壳、噪声过滤、时间/作者标准化、标签提取与重复检测（MinHash/SimHash），并输出高质量的结构化候选集与质量分。

表 4-3: 离线检索与索引模块接口描述（Search Agent 离线模块）

接口名	功能描述	责任划分
schedule_batch_crawl	定时批量抓取数据	调度服务与采集模块负责
fetch_domain_snapshot	获取指定数据快照	数据采集与存储负责
offline_text_search	基于关键词文本检索	离线搜索服务负责
offline_vector_search	基于向量的相似检索	向量检索服务负责
build_inverted_index	构建倒排索引持久化	索引构建服务负责
build_vector_index	构建向量索引持久化	向量索引服务负责
incremental_index_update	增量索引更新与合并	索引维护服务负责
index_health_metrics	索引状态与统计监控	监控与运维负责

```

1 import java.util.*; import java.util.concurrent.*;
2
3 public class OfflineRetrievalIndexer {
4     private final ScheduledExecutorService scheduler = Executors.
newSingleThreadScheduledExecutor();
5     private final ExecutorService worker = Executors.
newFixedThreadPool(4);
6     private final Crawler crawler; private final TextIndexer textIdx;
7     private final VectorIndexer vecIdx; private final EmbeddingService
embService;
8     private final Storage storage;
9
10    private void performBatchCrawl(){
11        List<String> domains = storage.listTargetDomains();
12        chunk(domains, 20).forEach(batch -> worker.submit(() -> {
13            List<Document> snaps = new ArrayList<>();
14            for(String d: batch){
15                try { snaps.addAll(crawler.fetchDomainSnapshot(d)); }
16                catch(Exception e){ log("crawl fail "+d); }
17            }
18            if(!snaps.isEmpty()){
19                storage.persistRawSnapshots(snaps);
20                worker.submit(() -> buildIndexes(snaps));
21            }
22        }));
23    }
24
25    private void buildIndexes(List<Document> docs){
26        try {
27            textIdx.bulkAdd(tokenizeDocs(docs));
28            List<float[]> vecs = embService.batchEmbed(docs);
29            vecIdx.bulkUpsert(docs, vecs);
30            storage.updateIndexMeta("last_build", System.
currentTimeMillis());
31        } catch(Exception e){ log("build error: "+e.getMessage()); }
32    }
33
34    public void applyIncremental(List<Document> changes){
35        worker.submit(() -> {
36            textIdx.bulkAdd(tokenizeDocs(changes));
37            vecIdx.bulkUpsert(changes, embService.batchEmbed(changes
));
38            storage.recordIncremental(changes);
39        });
40    }
41 }

```

图 4-3: 离线检索与索引模块的实现

向量化阶段使用版本化的 embedding 模型做批量推理，并记录 embedding_model_version 及预处理参数。向量索引采用 Faiss，工程上采用“热表 + 主表”的混合策略：新数据先写入热内存索引以保证短期可检索，定期批量并入主索引并进行压缩以降低长期存储。倒排索引使用 Elasticsearch，构建时通

过合理映射与分词器确保查询质量，构建完成后通过 `snapshot` 与 `alias` 原子切换发布到线上以保证无缝替换。增量更新采用 `delta` 索引与批量合并策略：增量写入走消息队列聚合批次写入，达到阈值触发后台合并并做 `segment` 优化。

索引构建与发布流程注重可回滚与可验证：每次构建记录构建参数、`checksum` 与样例查询回归结果；构建后在灰度流量上验证召回与重排指标，满足门槛后才切换别名。索引监控包括构建时长、`segment` 合并时间、查询延迟与内存占用，异常时支持回滚到上一个 `snapshot`。为支持研究与 A/B 实验，系统支持同时运行多套索引版本并按路由进行流量切分。

4.4 召回融合与重排模块的实现

召回融合与重排实现的目标是将来自多通道的候选统一评分、去重并给出最终排序，同时在延迟约束下最大化检索质量。实现分为三层：归一化与去重层、融合与候选裁剪层、分层重排层。归一化首先对不同通道原始分数进行标准化，并附带来源信任度与质量元数据。去重采用两阶段策略：第一阶段基于 `doc_id` 做精确去重；第二阶段基于 `SimHash/MinHash` 或 `embedding cosine` 做近似去重，以过滤相似内容但保留语义多样性。

表 4-4: 召回融合与重排模块接口描述（Search Agent 召回融合与重排）

接口名	功能描述	责任划分
<code>aggregate_candidates</code>	多源召回候选合并	召回与融合服务负责
<code>deduplicate_results</code>	候选结果去重处理	融合服务负责
<code>score_candidates</code>	候选初筛打分	基线打分服务负责
<code>rerank_with_model</code>	基于模型的重排序	重排模型服务负责
<code>diversity_filter</code>	多样性与去偏处理	融合/重排服务负责
<code>finalize_results</code>	阈值过滤与结果截断	结果返回服务负责
<code>log_and_metrics</code>	日志记录与质量评估	监控与评估负责

融合层执行多通道候选合并与份额控制，并根据业务策略裁剪候选池，在此阶段会计算重排所需的基础特征并从特征仓库或缓存读取静态特征以降低在线计算成本。重排层采用分层架构：轻量级在线 `LTR` 作为主重排器，保证毫秒级响应；对 `top-N` 可选启用 `cross-encoder` 做精排与一致性打分，但仅对高价值

请求或在可接受延迟条件下启用。为缩短 cross-encoder 延迟，可采用蒸馏后的双塔在在线路径提供近似 cross 评分。

```

1 import java.util.*;
2 import java.util.stream.*;
3
4 public class RecallFusionRerank {
5     private final RerankModel model; private final MetricsSink metrics
6     ;
7     public RecallFusionRerank(RerankModel m, MetricsSink ms){ this.
8     model=m; this.metrics=ms; }
9
10    private List<Candidate> aggregateCandidates(Map<String,List<
11    Candidate>> src){
12        return src.values().stream().flatMap(List::stream).collect(
13        Collectors.toList());
14    }
15    private List<Candidate> deduplicate(List<Candidate> in){
16        Map<String,Candidate> byId = new LinkedHashMap<>();
17        for(Candidate c: in) byId.putIfAbsent(c.uniqueKey(), c);
18        return new ArrayList<>(byId.values());
19    }
20    private List<Scored> baselineScore(List<Candidate> cands){
21        List<Scored> out = new ArrayList<>();
22        for(Candidate c: cands) out.add(new Scored(c, baseline(c)));
23        out.sort(Comparator.comparingDouble(Scored::score).reversed
24        ());
25        return out;
26    }
27    private List<Scored> modelRerank(List<Scored> scored){
28        return model.rerank(scored);
29    }
30    private List<Scored> diversityFilter(List<Scored> in){
31        Set<String> seenDomain = new HashSet<>(); List<Scored> out=new
32        ArrayList<>();
33        for(Scored s: in){
34            if(out.size()>=100) break;
35            String d = s.candidate.domain();
36            if(!seenDomain.contains(d) || Math.random()<0.2){ out.add(
37            s); seenDomain.add(d); }
38        }
39        return out;
40    }
41 }

```

图 4-4: 召回融合与重排模块的实现

系统在实现上重视特征工程与可观测性：大量静态特征提前离线计算并存入特征服务；实时特征在线计算并缓存。重排模型训练基于离线标注并包含数据流水线。上线前通过离线回放验证 ranking lift，并在灰度期间以线上指标检验模型效果。为支持解释性与审计，重排输出包含 top contributing features 与来源 attribution，用于前端展示与问题排查。最后，融合与重排模块与监控链路深

度集成：监控重排延迟分位、重排提升效果与跨通道贡献度，以便在异常时触发自动降级保障服务可用性。

4.5 本章小结

本章重点阐述了 Search Agent 系统各核心模块的具体实现过程，体现了从理论设计到工程落地的关键转化。本章在实现过程中充分考虑系统性能与扩展性，通过模块解耦与接口规范设计，确保系统具备良好的工程可维护性。

第五章 系统测试

5.1 系统测试概述

系统测试的目标是验证 Search Agent 在真实工程场景下的功能正确性、性能指标、可靠性与可用性，确保从 prompt 到检索/爬取、解析、清洗、入库与返回的端到端流水线在各种正常与异常条件下满足设计要求。本章测试策略采用“分层 + 闭环”方法：先通过单元测试保证最小功能单元的正确性，再通过功能测试验证各核心模块在集成环境下的行为，最后通过非功能测试评估系统的工程可用性。为了兼顾工程效率与现实覆盖，测试采用混合数据源：一方面使用合成与回放历史真实请求来进行自动化回归；另一方面构造标注测试集与人工审查场景来评估检索质量与解析准确性。所有测试组件都纳入 CI/CD 流水线，关键阶段需通过灰度发布并触发自动化回归验证。

测试环境分为单元测试环境、集成测试环境与性能/压力释放环境。单元与集成测试在 Kubernetes 测试命名空间中以 mock 外部依赖方式进行，以便快速反馈并保证可重复性；性能与压力测试在独立大规模集群中运行，能模拟真实并发、网络延迟与外部系统行为。测试过程强调可观测性：所有测试运行时需采集 Prometheus 指标、日志与链路追踪，并将结果自动汇总到测试报告系统。质量门控策略将基于预先定义的 SLI 决定是否允许发布或回滚。下文按测试类型详细展开测试设计与实现细节。

5.2 单元测试

单元测试着重验证系统最小功能单元的正确性与边界行为，是保证系统稳定性的第一道防线。针对 Search Agent 的不同模块，单元测试覆盖点包括：Prompt 解析模块的输入预处理、模型输出后处理与规则融合逻辑；Search Agent 的路由决策器；实时适配器与爬取任务构造器的任务描述格式与幂等性校验；数据处理模块的解析规则、字段标准化与去重算法；索引客户端的批量写入拼接与错误重试逻辑；召回融合与重排模块的分数归一化、特征计算与模

型推理调用契约等。单元测试采用 Mock 与 Stub 技术隔离外部依赖。

```

1 import org.junit.jupiter.api.*;
2 import static org.junit.jupiter.api.Assertions.*;
3 import java.util.*;
4
5 public class SearchAgentUnitTests {
6
7     @Test void offlineIndexerIncrementalUpdatesIndexes() {
8         OfflineRetrievalIndexer.TextIndexer ti=docs->{};
9         OfflineRetrievalIndexer.VectorIndexer vi=(docs,vecs)->{};
10        OfflineRetrievalIndexer.EmbeddingService es=docs-> List.of(new
float[1]);
11        OfflineRetrievalIndexer.Storage st=new OfflineRetrievalIndexer
.Storage(){
12            public List<String> listTargetDomains(){ return List.of
(); }
13            public void persistRawSnapshots(List<
OfflineRetrievalIndexer.Document> s) {}
14            public void recordIncremental(List<OfflineRetrievalIndexer
.Document> c) {}
15            public long countDocs(){ return 0; }
16            public Object getMeta(String k){ return null; }
17            public void updateIndexMeta(String k,Object v){}
18        };
19        OfflineRetrievalIndexer.Crawler cr=d-> List.of(new
OfflineRetrievalIndexer.Document(){ { id="d1"; text="x"; domain="x"
; } });
20        OfflineRetrievalIndexer idx=new OfflineRetrievalIndexer(cr,ti,
vi,es,st);
21        idx.applyIncremental(List.of(new OfflineRetrievalIndexer.
Document(){ { id="d2"; text="y"; domain="y"; } }));
22        assertTrue(true);
23    }
24
25    @Test void recallFusionRerankProducesTopK() {
26        RecallFusionRerank.RerankModel m = cand-> cand;
27        RecallFusionRerank.MetricsSink ms = (k,v)->{};
28        RecallFusionRerank rf = new RecallFusionRerank(m, ms);
29        Map<String,List<RecallFusionRerank.Candidate>> src = Map.of("a
", List.of(new RecallFusionRerank.Candidate(){ { id="1"; url="u";
title="t"; source="s"; } }));
30        List<RecallFusionRerank.Candidate> out = rf.process(src, 5);
31        assertEquals(1, out.size());
32    }
33 }

```

图 5-1: 单元测试代码

实现细节方面，所有模块需编写覆盖率高的测试用例，覆盖正常输入、边界输入、异常输入（非法字符、恶意注入）与错误路径（模型超时、后端返回 5xx、解析异常）。Prompt 解析的单元测试不仅断言关键词与 intent 的正确性，还需检验置信度阈值分支、fallback 路径与规则覆盖情况。重排模块的单元测试需确保特征计算的数值稳定、score normalization 在多个通道输入下的稳定性与

去重逻辑的正确性。数据库层的单元测试采用内存化数据库或 Docker Compose 启动轻量 DB，验证事务、唯一约束与回滚逻辑。测试自动化覆盖 CI：每次

表 5-1: 系统单元测试结果表

功能模块	测试内容率 (%)	
Prompt 解析模块	关键词提取准确性与输入鲁棒性测试	95%
MCP 通信与超时处理	响应超时、重试与错误回退场景测试	96%
实时检索（在线）	热点检索与检索结果实时推送正确性	93%
实时爬取与解析	实时爬取、解析正确性超时重试测试	90%
离线检索与索引	批量抓取、持久化与索引构建正确性	92%
索引增量更新	增量合并、一致性与回滚测试	94%
召回融合与重排	多源合并、去重与重排序准确性测试	91%
数据处理与清洗	初次清洗、二次清洗与质量评估测试	89%
向量检索性能	向量查询召回率与响应时延性能测试	88%
端到端集成测试	Prompt 解析到数据落库的全流程验证	90%

PR 都触发单元测试，测试失败阻止合并；同时在模型或规则变更提交时触发特定解析单元测试套件，以避免解析误回归。单元测试结果与覆盖率阈值作为代码质量核查项，并定期对测试用例做漏测补充（基于错误回放与事故 RCA）。

5.3 功能测试

功能测试在集成层面验证模块间接口契约、数据流与端到端功能场景。功能测试环境尽量接近生产，使用真实或高拟真度的外部依赖仿真，并利用历史请求回放作为主要测试驱动。以下分模块详细描述功能测试策略与评估准则。

5.3.1 Prompt 解析模块功能测试

Prompt 解析模块的功能测试旨在验证解析质量、稳定性与与下游契约的一致性。测试采用标注语料库与回放真实交互日志两条主线：标注语料覆盖常见场景（问答型、命令型、检索型、模糊描述）、多意图示例、噪声输入、以及触发爬取的样例。每个输入标注预期输出，作为黄金标准。测试时将黄金输入送入在线解析服务，校验输出字段的精确度、召回率与置信度分布，并对比历

表 5-2: Prompt 解析模块功能测试报告

测试功能	前置条件	测试步骤	预期结果
关键词提取	模型可用	输入含实体的 prompt	返回关键词列表
空输入处理	接口可访问	传入空的字符串	返回明确错误码
噪声鲁棒性	支持长文本	输入长文并夹杂噪声	抽出核心关键词
路径判定	触发词规则载入	输入含“详情”等词	标记为 true
MCP 超时	可模拟延迟	模拟首次调用超时	重试或降级
敏感词过滤	敏感词表已加载	输入含敏感词	敏感词被屏蔽
响应性能	并发测试环境	并发多请求测延迟	平均响应 <200ms

史模型/规则输出以评估迭代改进带来的改动。回放日志测试用于覆盖长尾与真实业务分布。将历史 prompt（匿名与脱敏）回放至解析模块并记录解析后下游影响指标：解析触发爬取比率、解析导致的检索通道路由变化、以及后续检索质量。对解析引起的误触率做重点关注，因为误触会显著增加爬虫池负载与外部风险。测试过程中通过断言与阈值设定（如误触率 < 0.1%）判断是否通过。

功能测试还需验证解析模块的回退与版本管理路径：包括当在线模型超时或返回低置信度时，规则回退到何种逻辑、与离线强模型的回流校正是否能正确更新规则与训练样本、以及解析输出的 trace 链路与审计记录是否完整。最后，压力下的功能稳定性测试检验缓存命中、批处理推理、以及降级策略是否按预期触发。

5.3.2 实时检索与爬取模块功能测试

实时检索与爬取功能测试覆盖从 Search Agent 发起检索、判定触发爬取、到爬虫执行、解析并将结果下发的完整闭环。测试场景包括正常流、必须爬取的场景、目标站点 JS 渲染场景、以及故障与合规降级场景。功能测试需要部署仿真第三方站点：静态页面、需要渲染的页面、含反爬机制的页面的样例，以验证爬虫策略的自动检测与处理。重点验证点包括爬取任务从生成到完成的 lifecycle（去重、入队、执行、回调）、任务幂等性（重复提交不导致重复入库）、断点续传（对大页面或网络抖动后的恢复能力）、与超时与重试策略的正确触发。测试会模拟目标站点临时不可用，观察调度器是否按退避策略重

表 5-3: 实时检索与爬取模块功能测试报告

测试功能	前置条件	测试步骤	预期结果
定时检索	调度服务已启动	触发定时任务	返回候选
在线检索	在线后端可用	发起检索请求	返回候选列表
结果推送	支持流式推送	接收并转发检索流	推送到消费端
关键词爬取	爬虫服务可用	触发词候选触发爬取	爬虫开始抓取
爬取超时	可模拟延迟	模拟首次超时场景	按重试策略重试
实时解析	解析组件在线	爬取后执行解析流程	提取标题/正文等
结果推送	数据处理端可用	发送解析结果	消息入队并被消费

试、队列堆积是否触发速率限制、以及 Search Agent 是否在短时间窗口内进行合适的降级（仅返回摘要或链接）。对于渲染场景，测试要验证渲染池的资源隔离、渲染超时的中止与后续解析失败标记。

同时，需要验证爬取输出进入数据处理的契约：解析字段是否按 schema 填充、parse_confidence 的分布是否合理、异常解析是否被正确标记并进入人工复核流程。合规测试包括在爬取前核验 robots.txt、在被禁止场景下是否正确跳过并记录审计日志、以及在触及敏感信息时是否正确脱敏或触发复核。对于触发爬取的同一 URL 频繁请求场景，要验证去重与短期缓存逻辑能有效抑制冗余抓取。

5.3.3 离线检索与索引模块功能测试

离线检索与索引的功能测试重点在于大规模数据管线的正确性、索引一致性与增量更新流程。测试流程从模拟数据采集开始：通过导入样本批次到对象存储，触发 ETL 流程并验证清洗、去重、标签化及向量化的输出。测试要断言清洗规则对噪声 HTML、缺失字段与非中文文本的处理是否符合设计，验证去重算法在常见重复变体上的召回与误判率。向量化测试涉及 embedding 模型版本一致性检验、向量索引构建 correctness、以及增量插入逻辑。倒排索引测试需要验证映射兼容性，并在索引切换时通过 alias 原子替换检查查询不中断。索引发布前需要进行回归查询测试：对一组标准查询集合计算 Recall@K、nDCG 与 baseline 比较，确保新索引在核心指标上不降级。

此外，增量更新场景测试检验 delta 索引并行查询的正确性，以及合并窗口

表 5-4: 离线检索与索引模块功能测试报告

测试功能	前置条件	测试步骤	预期结果
抓取调度	调度与爬虫可用	触发定时批量抓取	生成并持久化快照
快照获取	目标域数据存在	请求指定域快照	返回完整域快照
倒排索引	文本分词器可用	批量文档建索引	文本检索正确命中
向量索引	向量化服务在线	批量生成向量并入库	向量检索相似结果
增量索引	索引维护可用	提交增量变更集	更新数据生效
文本检索	倒排索引可查询	执行关键词查询	返回相关文档
向量检索	向量索引可查询	使用 query 向量检索	返回 TopK 结果

与合并后索引一致性。最后，测试索引回滚流程：在发现新索引性能问题时，验证别名回退、数据完整性校验以及重新建索引的可重复性与时间成本。

5.3.4 召回融合与重排模块功能测试

召回融合与重排的功能测试聚焦多通道候选合并、去重、特征计算、模型推理与最终排序质量。测试输入由多路候选源产生：实时倒排、离线倒排、向量召回与爬取解析结果，测试要验证归一化策略在合并后的稳健性，确保没有单一通道过度主导结果。去重测试检验两阶段去重在 doc_id、URL 变体与语义近似重复上的表现，避免重要但类似的条目被误去除。重排功能测试需覆盖

表 5-5: 召回融合与重排模块功能测试报告

测试功能	前置条件	测试步骤	预期结果
候选合并	多个候选源	提供多源列表	所有候选无丢失
去重策略	存在重复候选	输入重复候选去重	重复被去除
基线打分	基线打分器可用	对候选执行基线打分	返回分数
模型重排	重排模型在线	将打分候选送入模型	返回重排序列表
多样性过滤	存在单域刷量	执行多样性过滤算法	结果域分布更均衡
阈值截断与	阈值配置完成	执行截断 TopK 截取	返回 TopK
日志上报	指标系统接入	运行全流程检查指标	产出召回指标

特征完整性、特征缺失下的容错性，以及排序模型推理稳定性。测试用离线标注集评价最终排序指标与人工标注结果的相关性，同时在多种负载与延迟限制

下校验分层重排逻辑：当 `cross-encoder` 超时或不可用时系统是否能无缝降级到 `LightGBM` 并保持合理效果。还需验证排序解释性输出与溯源字段是否正确，以支持审计与人工校验。

重排上线前通过离线回放、A/B 测试与小流量灰度进行验证：离线回放用于快速筛查模型回归，A/B 实验与灰度检验用于在线测量 CTR、用户满意度及系统资源影响，同时统计显著性与长期趋势用于决策是否全量发布。

5.4 非功能测试

非功能测试是确保系统在实际运行中能够满足性能、稳定性、伸缩性、安全性和恢复能力的关键环节。对于本项目中开发的 `Search Agent` 系统，非功能测试尤为重要，因为该系统涉及到实时在线搜索和离线搜索的双重模式，并且需要处理大规模数据的爬取与实时解析。性能测试作为非功能测试的一部分，主要关注系统在高负载下的响应时间和吞吐量。通过流量生成器模拟真实的请求分布，并监控 P50、P95 和 P99 延迟，系统的吞吐量（QPS）、爬取并发数、消息队列的延迟（lag）、索引入库的吞吐量以及 CPU、内存、IOPS 等资源的消耗情况，性能测试帮助我们评估系统在实际工作负载下的表现。

压力测试在这一过程中发挥了重要作用，它通过逐步增加负载直至系统达到瓶颈，能够帮助定位系统的限流点，并为后续的容量规划提供指导。性能优化通常侧重于几个方面，如提升模型推理吞吐量、控制重排深度、优化缓存命中率 and 改善批量写入性能等。通过这些优化措施，我们能够有效提升系统的处理能力，确保系统在高并发请求下的稳定运行。除了性能测试，系统的稳定性测试同样重要。长时间运行的浸泡测试用于检测系统在接近生产环境的负载下是否存在内存泄漏、慢查询积累等问题。此外，混沌测试则通过模拟网络延迟、数据库短暂不可用、节点重启等极端情况，验证系统的自动恢复能力和降级策略。

恢复能力测试评估了系统在发生故障后能否迅速恢复，并确保数据不会丢失或损坏。备份恢复演练是其中的关键环节，它通过模拟故障场景并评估恢复时间目标（RTO），确保在真实环境中发生故障时，系统能够快速恢复，减少停机时间。系统的安全性和合规性也是非功能测试的重要组成部分。在安全性测试方面，我们通过漏洞扫描、依赖库安全审计、入侵检测等手段，确保系统能够有效抵御潜在的安全威胁，同时确保敏感数据得到妥善保护。合规性测试

则验证了审计日志的完整性，确保访问控制策略能够有效隔离运维与开发人员的操作，并确保爬取行为符合相关法律法规和目标站点的爬取规则。

除了性能、稳定性和安全性，伸缩性和运维能力也是本系统需要重点验证的内容。系统采用了自动伸缩策略，通过模拟流量波动和资源瓶颈的场景，评估了系统的扩容时间和服务恢复时间。金丝雀发布流程和回滚路径的验证保证了系统在负载突增时能够快速响应，并且服务能够在不影响用户体验的前提下进行升级或回滚。运维测试则检验了系统的自动化管理能力，通过实时监控和告警功能，在发生异常时能够快速定位问题并进行处理，减少人为干预，确保系统的高可用性和稳定性。

效能测试用于评估不同配置对系统资源消耗和质量指标的影响。通过测试不同配置的系统在性能、资源消耗、吞吐量等方面的表现，能够为系统的优化和扩展提供依据，并帮助团队制定合理的容量规划和成本预算。最终，通过这些非功能测试的综合验证，能够确保 **Search Agent** 系统在面对高并发、大规模数据处理以及复杂业务需求时，依然能够高效稳定地运行，并提供优质的用户体验。这些测试结果为后续系统的优化、扩展和性能提升提供了宝贵的数据支持，确保了系统能够在不断变化的环境中保持可靠性和可扩展性。

5.5 本章小结

本章围绕 **Search Agent** 系统的测试与验证工作展开，从多个维度对系统性能与功能进行系统性评估。本章还结合监控系统对关键指标进行实时观测，对系统运行状态进行量化评估。测试结果表明，系统在功能完整性与性能指标方面均达到了预期目标，能够满足实际应用需求。通过本章测试，不仅验证了系统设计与实现的正确性，也为后续优化与实际部署提供了重要依据。

第六章 总结与展望

6.1 论文工作总结

本论文面向工程化检索与数据采集场景，设计并实现了一套以 Search Agent 为核心的端到端检索流水线，旨在将用户自然语言提示通过模型驱动的解析，工程化地对接实时在线与离线垂域检索后端，完成检索、可选的实时爬取、解析、清洗、质量评估与索引入库的全流程。论文的工作可从方法论贡献与工程实现两大层面进行概括。

在方法论层面，本论文系统化地提出并实现了 prompt-to-query 的工程化路线：结合规则与轻量模型的混合解析架构，输出可解释、带置信度的 QueryPackage，包括关键词、意图标签、触发爬取标识等，并通过在线轻量化推理满足交互延迟约束，同时为离线强模型提供回流校正样本。针对混合检索问题，本论文阐述了并实现倒排检索与向量检索并行召回、跨通道 score normalization 与多阶段融合策略，提出在候选层面采用两阶段去重与配置化通道配额，以在保证语义覆盖的同时控制冗余。重排方面，论文采用分层重排思想：以 feature-based 的轻量 LTR 保证在线延迟需求，必要时对 top-K 启用 cross-encoder 做精排，并通过蒸馏与特征缓存减小二阶段模型对实时性能的影响。此外，论文在实时爬取与解析方面提出了“先快后补”的工程策略：在判断需要页面详情时异步触发爬取，不阻塞首屏响应，并设计了域名级限速、渲染降级、严格超时与断点续传策略，以平衡新鲜度、响应性与外部合规性。

在工程实现层面，论文构建了端到端的消息驱动流水线与模块化服务框架：以 MCP 为协调层实现同步与异步任务管理、路由与回调；Search Agent 作为决策层实现 prompt 路由、触发爬取与候选融合；Prompt Parser 提供语义解析服务；实时适配器与爬虫池完成低延迟召回与页面抓取；数据处理模块负责实时解析、二次清洗与质量评估；索引层分别实现倒排索引与向量索引的增量与批量建库策略。为保证系统的工程可用性，论文还讨论了监控、日志、告警、审计、灰度发布与回滚等运维措施，并提出了具体的存储与索引策略以支持回放重跑与审计。

在评估与验证方面，论文制定了完整的指标体系：包含系统性能指标、检索质量指标、数据工程指标及用户感知指标，并以离线回放、A/B 小流量灰度与压力测试等方式进行验证。总体实验与工程验证表明：模型驱动的 `prompt-to-query` 能有效提高检索的语义相关性；实时爬取在受控超时与并发策略下可以显著提升结果的新鲜度；跨通道融合与分层重排在延迟约束下提升了最终排序效果。同时论文也识别出系统在索引更新成本、实时爬取稳定性、模型输出解释性等方面的局限与挑战。

最后，论文在实现细节上给出了可部署性的最佳实践：包括解析模块的版本化与灰度发布流程、爬取池的资源隔离策略、索引构建的 `alias` 原子切换、消息队列驱动的异步补全以及完整的监控、告警与审计链路。这些工程化细节使得提出的方法不仅具备学术价值，也具有明确的工业落地潜力。

6.2 后续工作展望

尽管本论文实现了一个功能完备且工程化的 `Search Agent` 系统，但在若干重要方向上仍存在可延展和优化的空间。以下从技术深化、系统工程、合规与应用扩展等角度阐述后续工作方向：

1. 未来可在 `Prompt` 解析与语义理解方面引入在线学习和微调机制，提升对新兴实体和用户表达的适应能力。此外，实时爬取可通过优化反爬机制与渲染策略，提升资源利用效率。在向量索引与增量更新上，研究高效的在线索引结构和增量更新算法，以减少重建成本。同时，在召回融合与重排方面，探索端到端学习框架和在线学习能力，提升系统的灵活性和实时性。

2. 加强隐私保护与合规审查，引入差分隐私等技术确保数据安全，并完善自动化合规机制。未来还将扩展多模态检索与生成模型集成，增强系统在不同场景下的适应性。此外，通过构建成熟的 `MLOps` 体系，实现模型与数据版本治理、自动化回放与性能回归检测，以支撑大规模生产环境中的稳定运行。

致 谢

感谢在实验室度过的两年时光，老师无论在学术还是人生的指导上都对我起到了很大的帮助；师兄师姐小伙伴们的鼓励支持和陪伴是我坚持下去的动力。

参考文献

- [1] MAO Q, QIN H, NEARY R, et al. A Comparative Study of Retrieval Methods in Azure AI Search[J], 2025.
- [2] WANG M, TAN B, GAO Y, et al. Balancing the Blend: An Experimental Analysis of Trade-offs in Hybrid Search[J], 2025.
- [3] YANG J, MO S, SHI J, et al. ARCADE: A Real-Time Data System for Hybrid and Continuous Query Processing across Diverse Data Modalities[J], 2025.
- [4] AL-AKASHI F H A, INKPEN D. A New Adaptive Indexing for Real-Time Web Search[J]. International Journal of Intelligent Information Technologies, 2022, 18(1).
- [5] ZHU Y, YUAN H, WANG S, et al. Large Language Models for Information Retrieval: A Survey[J]. ACM Trans. Inf. Syst., 2025, 44(1).
- [6] LUO Z, SHI X, LIN X, et al. Evaluation Report on MCP Servers[J], 2025.
- [7] KALRA J S, ZHAO X, KIM T E, et al. MoR: Better Handling Diverse Queries with a Mixture of Sparse, Dense, and Human Retrievers[J], 2025.
- [8] RAJ M, MISHRA N. Exploring new Approaches for Information Retrieval through Natural Language Processing[J], 2025.
- [9] XU Z, MO F, HUANG Z, et al. A Survey of Model Architectures in Information Retrieval[J], 2026.
- [10] PRAJAPATI T. LiveVectorLake: A Real-Time Versioned Knowledge Base Architecture for Streaming Vector Updates and Temporal Retrieval[J], 2025.
- [11] WANG Y, LI Z, LIU S, et al. LEANN: A Low-Storage Vector Index[J], 2025.

- [12] HOVEYDA M, OOSTERHUIS H, de VRIES A P, et al. Adaptive Orchestration of Modular Generative Information Access Systems[J], 2025 : 3899 – 3910.
- [13] LI X, SU L, JIA P, et al. Agent4Ranking: Semantic Robust Ranking via Personalized Query Rewriting Using Multi-agent LLM[J], 2023.
- [14] HUANG J, HUANG X, ZHAO F, et al. QP-OneModel: A Unified Generative LLM for Multi-Task Query Understanding in Xiaohongshu Search[J], 2026.
- [15] ZHANG K, BALALAU O, MANOLESCU I. Structured Discourse Representation for Factual Consistency Verification[J], 2025 : 820 – 838.
- [16] SHACHAR O, KATZ U, GOLDBERG Y, et al. NER Retriever: Zero-Shot Named Entity Retrieval with Type-Aware Embeddings[J], 2025.
- [17] MENG C, ARABZADEH N, ALIANNEJADI M, et al. Query Performance Prediction: From Ad-hoc to Conversational Search[J], 2023 : 2583 – 2593.
- [18] ZHANG H, WANG S, LI H, et al. Uncovering the Propensity Identification Problem in Debaised Recommendations[J], 2024 : 653 – 666.
- [19] ZHANG H, WANG S, LI H, et al. Uncovering the Propensity Identification Problem in Debaised Recommendations[J], 2024 : 653 – 666.
- [20] CHEN C-C, TAKAMURA H, KOBAYASHI I, et al. Professionalism-Aware Pre-Finetuning for Profitability Ranking[J], 2024 : 3674 – 3678.
- [21] CORMODE G, MARKOV I L. Federated Calibration and Evaluation of Binary Classifiers[J]. Proc. VLDB Endow., 2023, 16(11): 3253 – 3265.
- [22] STORMCRAWLER A. Quick Start[EB]. 2023.
- [23] CHEN W, YIN M, KU M, et al. TheoremQA: A Theorem-driven Question Answering Dataset[J], 2023 : 7889 – 7901.
- [24] LIU W, YU X, MAO J, et al. Quantifying the Effectiveness of Advertising: A Bootstrap Proportion Test for Brand Lift Testing[J], 2023 : 1627 – 1636.
- [25] YANG E, YATES A, LAWRIE D, et al. RoutIR: Fast Serving of Retrieval Pipelines for Retrieval-Augmented Generation[J], 2026.

-
- [26] SUN Q, BI T, LI S, et al. SymbioticRAG: Enhancing Document Intelligence Through Human-LLM Symbiotic Collaboration[J], 2025.
- [27] DONG J-K, HUANG I, WU C-T, et al. ETOM: A Five-Level Benchmark for Evaluating Tool Orchestration within the MCP Ecosystem[J], 2025.
- [28] LAI Y, WU J, WANG Z, et al. AdaRewriter: Unleashing the Power of Prompting-based Conversational Query Reformulation via Test-Time Adaptation[J], 2025.
- [29] FANG Y-L, JIAN D-S, LI X, et al. LOCA: Logical Chain Augmentation for Scientific Corpus Cleaning[J], 2025.
- [30] GERMAN D M, ADAMS B, HASSAN A E. Continuously mining distributed version control systems: an empirical study of how Linux uses Git[J]. *Empirical Software Engineering*, 2016, 21(1): 260–299.
- [31] MIZOUCHI T, SHIMARI K, ISHIO T, et al. PADLA: A Dynamic Log Level Adapter Using Online Phase Detection[J], 2019: 135–138.
- [32] afROSENSCHÖLD C-J F, AWAYSHEH F M, AWAD A. Next Generation Cloud-native In-Memory Stores: From Redis to Valkey and Beyond[J]. *arXiv preprint*, 2025, arXiv:2510.19805.
- [33] TAY B H, ANANDA A L. A Survey of Remote Procedure Calls[J]. *ACM SIGOPS Operating Systems Review*, 1990, 24(3): 68–79.
- [34] ZHU Z, CHAMBERLAIN J, WU K, et al. Approximation-First Timeseries Monitoring Query At Scale[J]. *arXiv preprint*, 2025, arXiv:2505.10560.
- [35] MAJORS C, FONG-JONES L, MIRANDA G. *Observability Engineering: Achieving Production Excellence*[M]. [S.l.]: O'Reilly Media, 2022.
- [36] AMANBAYEV A, TSAN B, DANG T, et al. Filtered Approximate Nearest Neighbor Search in Vector Databases: System Design and Performance Analysis[J], 2026.
- [37] CHAVHAN S, DHARMIK R C. Feature Selection for Ranking using Heuristics based Learning to Rank using Machine Learning[J]. *International Journal of Next-Generation Computing*, 2022, 13(1): 141–164.

- [38] BUSA-FEKETE R, ZIMMERT J, GYÖRGY A, et al. A Scalable Crawling Algorithm Utilizing Noisy Change-Indicating Signals[J], 2025.
- [39] KREUTZ C, HAAK F, ENGELMANN B, et al. BATS: BenchmArking Text Simplicity □[J], 2024 : 11968 – 11989.
- [40] BAI F, KANG J, STANOVSKY G, et al. Schema-Driven Information Extraction from Heterogeneous Tables[J], 2024 : 10252 – 10273.
- [41] KUMAR A, TUMRE S, PATIL S. Benchmarking Near-Duplicate Detection in the era of Pay-walled News[J], 2025 : 745 – 748.
- [42] SHAH A, SHAH H, BAFNA V, et al. VERITAS-NLI : Validation and Extraction of Reliable Information Through Automated Scraping and Natural Language Inference[J], 2024.
- [43] KAHOLON N K, SINGH W. A systematic review of web scraping: Techniques, LLM-enhanced approaches, performance metrics, and legal – ethical issues[J]. Data & Knowledge Engineering, 2026, 164 : 102598.
- [44] CAO W, HALLGARTEN M, LI T, et al. Pseudo-Simulation for Autonomous Driving[J], 2026.
- [45] BIN X, CUI J, YAN W, et al. Real-time Indexing for Large-scale Recommendation by Streaming Vector Quantization Retriever[J], 2025.
- [46] JI Y, WANG S, CHEN Z, et al. Updatable Balanced Index for Fast On-device Search with Auto-selection Model[J], 2025.
- [47] MOHONEY J, SARDA D, TANG M, et al. Quake: Adaptive Indexing for Vector Search[J], 2025.